

# **NOVA Microhypervisor Interface Specification**

Udo Steinberg  
[udo@hypervisor.org](mailto:udo@hypervisor.org)

April 29, 2026

**Copyright © 2006–2011 Udo Steinberg, Technische Universität Dresden**  
**Copyright © 2012–2013 Udo Steinberg, Intel Corporation**  
**Copyright © 2014–2016 Udo Steinberg, FireEye, Inc.**  
**Copyright © 2019–2026 Udo Steinberg, BlueRock Security, Inc.**

This specification is provided "as is" and may contain defects or deficiencies which cannot or will not be corrected. The author makes no representations or warranties, either expressed or implied, including but not limited to, warranties of merchantability, fitness for a particular purpose, or non-infringement that the contents of the specification are suitable for any purpose or that any practice or implementation of such contents will not infringe any third party patents, copyrights, trade secrets or other rights.

The specification could include technical inaccuracies or typographical errors. Additions and changes are periodically made to the information therein; these will be incorporated into new versions of the specification, if any.

# Contents

|            |                                          |           |
|------------|------------------------------------------|-----------|
| <b>I</b>   | <b>Introduction</b>                      | <b>1</b>  |
| <b>1</b>   | <b>System Architecture</b>               | <b>2</b>  |
| <b>II</b>  | <b>Basic Abstractions</b>                | <b>3</b>  |
| <b>2</b>   | <b>Kernel Objects</b>                    | <b>4</b>  |
| 2.1        | Protection Domain . . . . .              | 4         |
| 2.1.1      | Object Space . . . . .                   | 4         |
| 2.1.2      | Host Space . . . . .                     | 4         |
| 2.1.3      | Guest Space . . . . .                    | 4         |
| 2.1.4      | DMA Space . . . . .                      | 5         |
| 2.1.5      | PIO Space . . . . .                      | 5         |
| 2.1.6      | MSR Space . . . . .                      | 5         |
| 2.2        | Execution Context . . . . .              | 6         |
| 2.2.1      | Host Execution Context . . . . .         | 6         |
| 2.2.2      | Guest Execution Context . . . . .        | 6         |
| 2.3        | Scheduling Context . . . . .             | 7         |
| 2.4        | Portal . . . . .                         | 7         |
| 2.5        | Semaphore . . . . .                      | 7         |
| 2.6        | Device Context . . . . .                 | 7         |
| <b>3</b>   | <b>Hardware Resources</b>                | <b>8</b>  |
| 3.1        | System Time Counter . . . . .            | 8         |
| <b>III</b> | <b>Application Programming Interface</b> | <b>9</b>  |
| <b>4</b>   | <b>Data Types</b>                        | <b>10</b> |
| 4.1        | Capability . . . . .                     | 10        |
| 4.1.1      | Null Capability . . . . .                | 10        |
| 4.1.2      | Object Capability . . . . .              | 10        |
| 4.1.2.1    | Object Space Capability . . . . .        | 10        |
| 4.1.2.2    | Host Space Capability . . . . .          | 10        |
| 4.1.2.3    | Guest Space Capability . . . . .         | 10        |
| 4.1.2.4    | DMA Space Capability . . . . .           | 11        |
| 4.1.2.5    | PIO Space Capability . . . . .           | 11        |
| 4.1.2.6    | MSR Space Capability . . . . .           | 11        |
| 4.1.2.7    | Protection Domain Capability . . . . .   | 11        |
| 4.1.2.8    | Execution Context Capability . . . . .   | 11        |
| 4.1.2.9    | Scheduling Context Capability . . . . .  | 12        |
| 4.1.2.10   | Portal Capability . . . . .              | 12        |
| 4.1.2.11   | Semaphore Capability . . . . .           | 12        |
| 4.1.2.12   | Device Context Capability . . . . .      | 12        |
| 4.1.3      | Memory Capability . . . . .              | 12        |
| 4.1.4      | PIO Capability . . . . .                 | 13        |
| 4.1.5      | MSR Capability . . . . .                 | 13        |
| 4.2        | Capability Selector . . . . .            | 14        |
| 4.2.1      | Object Capability Selector . . . . .     | 14        |

|          |                               |           |
|----------|-------------------------------|-----------|
| 4.2.2    | Host Capability Selector      | 14        |
| 4.2.3    | Guest Capability Selector     | 14        |
| 4.2.4    | DMA Capability Selector       | 14        |
| 4.2.5    | PIO Capability Selector       | 14        |
| 4.2.6    | MSR Capability Selector       | 14        |
| 4.3      | User Thread Control Block     | 15        |
| 4.3.1    | Regular Layout                | 15        |
| 4.3.2    | Architectural Layout          | 15        |
| 4.4      | Message Transfer Descriptor   | 16        |
| 4.4.1    | Regular IPC                   | 16        |
| 4.4.2    | Architectural IPC             | 16        |
| 4.5      | Scheduling Context Descriptor | 17        |
| <b>5</b> | <b>Hypercalls</b>             | <b>18</b> |
| 5.1      | Definitions                   | 18        |
| 5.1.1    | Hypercall Numbers             | 18        |
| 5.1.2    | Status Codes                  | 18        |
| 5.2      | Communication                 | 19        |
| 5.2.1    | IPC Call                      | 19        |
| 5.2.2    | IPC Reply                     | 20        |
| 5.3      | Object Creation               | 21        |
| 5.3.1    | Create Protection Domain      | 21        |
| 5.3.2    | Create Execution Context      | 23        |
| 5.3.3    | Create Scheduling Context     | 25        |
| 5.3.4    | Create Portal                 | 26        |
| 5.3.5    | Create Semaphore              | 27        |
| 5.3.6    | Create Device Context         | 29        |
| 5.4      | Object Control                | 30        |
| 5.4.1    | Control Protection Domain     | 30        |
| 5.4.2    | Control Execution Context     | 32        |
| 5.4.3    | Control Scheduling Context    | 33        |
| 5.4.4    | Control Portal                | 34        |
| 5.4.5    | Control Semaphore             | 35        |
| 5.5      | Platform Management           | 36        |
| 5.5.1    | Control Hardware              | 36        |
| 5.5.2    | Assign Device                 | 39        |
| 5.5.3    | Assign Interrupt              | 40        |
| <b>6</b> | <b>Bootimg</b>                | <b>42</b> |
| 6.1      | NOVA Microhypervisor          | 42        |
| 6.1.1    | NOVA Image                    | 42        |
| 6.1.2    | NOVA Integrity Measurement    | 42        |
| 6.1.3    | NOVA Spaces                   | 43        |
| 6.1.3.1  | NOVA Object Space             | 43        |
| 6.1.3.2  | NOVA Host Space               | 43        |
| 6.1.3.3  | NOVA PIO Space                | 43        |
| 6.1.3.4  | NOVA MSR Space                | 43        |
| 6.2      | Root Protection Domain        | 44        |
| 6.2.1    | Root Image                    | 44        |
| 6.2.2    | Root Integrity Measurement    | 45        |
| 6.2.3    | Root Spaces                   | 45        |
| 6.2.3.1  | Root Object Space             | 45        |
| 6.2.3.2  | Root Host Space               | 45        |
| 6.2.3.3  | Root PIO Space                | 45        |
| 6.3      | Hypervisor Information Page   | 46        |

## IV Application Binary Interface

49

|                                            |           |
|--------------------------------------------|-----------|
| <b>7 ABI aarch64</b>                       | <b>50</b> |
| 7.1 Boot State                             | 50        |
| 7.1.1 NOVA Microhypervisor                 | 50        |
| 7.1.1.1 Multiboot v2 Launch                | 50        |
| 7.1.1.2 Multiboot v1 Launch                | 50        |
| 7.1.1.3 Legacy Launch                      | 50        |
| 7.1.2 Root Protection Domain               | 51        |
| 7.2 Memory Map                             | 51        |
| 7.3 Class Of Service                       | 51        |
| 7.4 Protected Resources                    | 51        |
| 7.4.1 Physical Memory                      | 51        |
| 7.5 Event-Specific Capability Selectors    | 52        |
| 7.5.1 Architectural Events                 | 52        |
| 7.5.2 Microhypervisor Events               | 53        |
| 7.6 Architecture-Dependent Structures      | 54        |
| 7.6.1 Platform Features                    | 54        |
| 7.6.2 Hypervisor Information Page          | 54        |
| 7.6.3 User Thread Control Block            | 55        |
| 7.6.4 Message Transfer Descriptor          | 56        |
| 7.6.5 Memory Attribute Descriptor          | 57        |
| 7.6.6 Interrupt Semaphore Descriptor       | 57        |
| 7.6.7 Device Topology Descriptor           | 58        |
| 7.6.7.1 SMMUv3                             | 58        |
| 7.6.7.2 SMMUv2                             | 58        |
| 7.7 Calling Convention                     | 59        |
| 7.8 Supplementary Functionality            | 62        |
| <b>8 ABI x86-64</b>                        | <b>63</b> |
| 8.1 Boot State                             | 63        |
| 8.1.1 NOVA Microhypervisor                 | 63        |
| 8.1.1.1 Multiboot v2 Launch                | 63        |
| 8.1.1.2 Multiboot v1 Launch                | 63        |
| 8.1.2 Root Protection Domain               | 63        |
| 8.2 Memory Map                             | 64        |
| 8.3 Class Of Service                       | 64        |
| 8.4 Protected Resources                    | 64        |
| 8.4.1 Physical Memory                      | 64        |
| 8.4.2 I/O Ports                            | 65        |
| 8.4.3 Model-Specific Registers             | 65        |
| 8.5 Event-Specific Capability Selectors    | 66        |
| 8.5.1 Architectural Events                 | 66        |
| 8.5.2 Microhypervisor Events               | 68        |
| 8.6 Architecture-Dependent Structures      | 69        |
| 8.6.1 Platform Features                    | 69        |
| 8.6.2 Hypervisor Information Page          | 69        |
| 8.6.3 User Thread Control Block            | 70        |
| 8.6.3.1 Encoding: Segment Access Rights    | 71        |
| 8.6.3.2 Encoding: Interruption Information | 71        |
| 8.6.4 Message Transfer Descriptor          | 72        |
| 8.6.5 Memory Attribute Descriptor          | 73        |
| 8.6.6 Interrupt Semaphore Descriptor       | 73        |
| 8.6.7 Device Topology Descriptor           | 73        |
| 8.7 Calling Convention                     | 74        |

|          |                                 |           |
|----------|---------------------------------|-----------|
| <b>V</b> | <b>Appendix</b>                 | <b>78</b> |
| <b>A</b> | <b>Acronyms</b>                 | <b>79</b> |
| <b>B</b> | <b>Bibliography</b>             | <b>83</b> |
| <b>C</b> | <b>Consoles</b>                 | <b>85</b> |
| C.1      | Memory-Buffer Console . . . . . | 85        |
| C.2      | Framebuffer Console . . . . .   | 86        |
| C.3      | UART Consoles . . . . .         | 86        |
| <b>D</b> | <b>Download</b>                 | <b>87</b> |

## Notation

The key words **must**, **must not**, **required**, **should**, **should not**, **recommended**, **may** and **optional** in this document are to be interpreted as described in RFC 2119 [1].

Throughout this document, the following symbols are used:

- ~ Indicates that the value of this parameter or field is **unknown**. The microhypervisor cannot ensure that the value does not leak information across protection domain boundaries.
- ⚡ Indicates that the value of this parameter or field is **undefined**. The microhypervisor ensures that the value does not leak information across protection domain boundaries. Future versions of this specification may define a value for the parameter or field.
- Indicates that the value of this parameter or field is **ignored**. Future versions of this specification may define a meaning for the parameter or field.
- ≡ Indicates that the value of this parameter or field is **unchanged**. The microhypervisor preserves the value across hypercalls.





## **Part I**

# **Introduction**

# 1 System Architecture

The NOVA OS Virtualization Architecture [2] (NOVA) facilitates the coexistence of multiple legacy guest operating systems and a user-mode host framework on a single platform. The core system leverages hardware virtualization technology provided by modern x86 or Arm platforms and comprises the NOVA microhypervisor and one or more Virtual-Machine Monitors (VMMs).

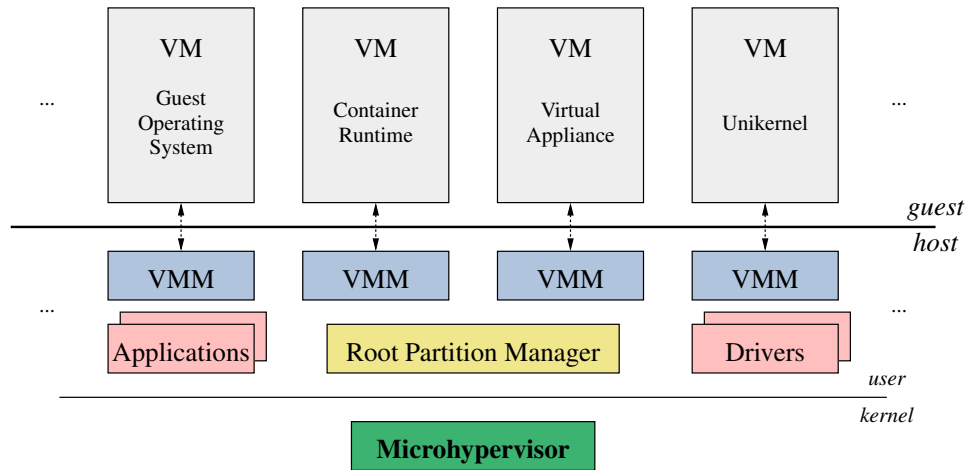


Figure 1.1: System Architecture

Figure 1.1 shows the structure of the system. The microhypervisor is the only component executing in privileged host/kernel mode. It isolates the various user-mode components, including the virtual-machine monitors, from one another by placing them in different protection domains in unprivileged host/user mode. Each legacy guest operating system runs in its own virtual-machine environment in guest mode and is therefore isolated from the other components.

Besides spatial and temporal isolation, the microhypervisor also provides mechanisms for partitioning and delegation of platform resources, such as CPU time, physical memory, I/O ports and hardware interrupts and for establishing communication channels and signaling between different protection domains.

The virtual-machine monitors handle virtualization events and implement virtual devices that enable legacy guest operating systems to function in the same manner as they would on bare-metal hardware. Providing this functionality outside the microhypervisor in the VMMs reduces the size of the trusted computing base significantly for all components that do not require virtualization support.

The architecture and interfaces of the VMM and the user-mode host framework are not described in this document.

## **Part II**

# **Basic Abstractions**

## 2 Kernel Objects

### 2.1 Protection Domain

1. The **Protection Domain (PD)** is a unit of protection and spatial isolation.
2. Access to a **Protection Domain** is controlled by a **PD Capability** ( $CAP_{OBJ_{PD}}$ ).
3. Different types of spaces can be created for a **PD** – see Section 4.2 for details regarding availability and size.
4. Spaces store **Capabilities (CAPs)** for kernel objects or platform resources that **Execution Contexts** in that **Protection Domain** can access.

The following subsections describe each space in more detail.

#### 2.1.1 Object Space

1. Only one **Object Space** ( $SPC_{OBJ}$ ) can be created per **Protection Domain**.
2. Access to an **Object Space** is controlled by an **Object Space Capability** ( $CAP_{OBJ_{OBJ}}$ ).
3. Each hypercall invoked by a **Host Execution Context** explicitly specifies an **Object Capability Selector** to designate the kernel object on which it operates.
4. The **Object Capability Selector** ( $SEL_{OBJ}$ ) serves as index into the **EC's Object Space** and selects a slot that contains either a **Null Capability** ( $CAP_0$ ) or an **Object Capability** ( $CAP_{OBJ}$ ) that refers to a kernel object with associated access permissions.

#### 2.1.2 Host Space

1. Only one **Host Space** ( $SPC_{HST}$ ) can be created per **Protection Domain**.
2. Access to a **Host Space** is controlled by a **Host Space Capability** ( $CAP_{OBJ_{HST}}$ ).
3. Each memory operation issued by a **Host Execution Context** implicitly uses the page number of the accessed **Host-Virtual Address (HVA)** as **Host Capability Selector**:  $SEL_{HST} = HVA \gg 12$ .
4. The **Host Capability Selector** ( $SEL_{HST}$ ) serves as index into the **EC's Host Space** and selects a slot that contains either a **Null Capability** ( $CAP_0$ ) or a **Memory Capability** ( $CAP_{MEM}$ ) that refers to a 4 KiB page frame in physical memory with associated access permissions.

#### 2.1.3 Guest Space

1. Multiple **Guest Spaces** ( $SPC_{GST}$ ) can be created per **Protection Domain**.
2. Access to a **Guest Space** is controlled by a **Guest Space Capability** ( $CAP_{OBJ_{GST}}$ ).
3. Each memory operation issued by a **Guest Execution Context** implicitly uses the page number of the accessed **Guest-Physical Address (GPA)** as **Guest Capability Selector**:  $SEL_{GST} = GPA \gg 12$ .
4. The **Guest Capability Selector** ( $SEL_{GST}$ ) serves as index into the **EC's Guest Space** and selects a slot that contains either a **Null Capability** ( $CAP_0$ ) or a **Memory Capability** ( $CAP_{MEM}$ ) that refers to a 4 KiB page frame in physical memory with associated access permissions.

### 2.1.4 DMA Space

1. Multiple **DMA Spaces** ( $SPC_{DMA}$ ) can be created per **Protection Domain**.
2. Access to a **DMA Space** is controlled by a **DMA Space Capability** ( $CAP_{OBJ_{DMA}}$ ).
3. Each **DMA** operation issued by a device implicitly uses the page number of the accessed **DMA-Virtual Address** (DVA) as **DMA Capability Selector**:  $SEL_{DMA} = DVA \gg 12$ .
4. The **DMA Capability Selector** ( $SEL_{DMA}$ ) serves as index into the device's **DMA Space** and selects a slot that contains either a **Null Capability** ( $CAP_0$ ) or a **Memory Capability** ( $CAP_{MEM}$ ) that refers to a 4 KiB page frame in physical memory with associated access permissions.

### 2.1.5 PIO Space

1. Multiple **PIO Spaces** ( $SPC_{PIO}$ ) can be created per **Protection Domain**.
2. Access to a **PIO Space** is controlled by a **PIO Space Capability** ( $CAP_{OBJ_{PIO}}$ ).
3. Each **PIO** access (IN/OUT instruction) issued by an **Execution Context** (EC) implicitly uses the number of the accessed I/O port as **PIO Capability Selector**.
4. The **PIO Capability Selector** ( $SEL_{PIO}$ ) serves as index into the EC's **PIO Space** and selects a slot that contains either a **Null Capability** ( $CAP_0$ ) or a **PIO Capability** ( $CAP_{PIO}$ ) that refers to the I/O port  $SEL_{PIO}$  with associated access permissions.

### 2.1.6 MSR Space

1. Multiple **MSR Spaces** ( $SPC_{MSR}$ ) can be created per **Protection Domain**.
2. Access to an **MSR Space** is controlled by an **MSR Space Capability** ( $CAP_{OBJ_{MSR}}$ ).
3. Each **MSR** access (RDMSR/WRMSR instruction) issued by an **Execution Context** (EC) implicitly uses the number of the accessed **MSR** as **MSR Capability Selector**.
4. The **MSR Capability Selector** ( $SEL_{MSR}$ ) serves as index into the EC's **MSR Space** and selects a slot that contains either a **Null Capability** ( $CAP_0$ ) or an **MSR Capability** ( $CAP_{MSR}$ ) that refers to the **MSR**  $SEL_{MSR}$  with associated access permissions.

## 2.2 Execution Context

1. The **Execution Context (EC)** is an abstraction for an activity within a **PD**.
2. Access to an **Execution Context** is controlled by an **EC Capability** ( $CAP_{OBJ_{EC}}$ ).
3. An **EC** is permanently bound to one **CPU**.
4. An **EC** contains architecture-dependent state, such as
  - Central Processing Unit (**CPU**) registers
  - Floating Point Unit (**FPU**) registers (optionally)

The following subsections provide more details for each type of **Execution Context**.

### 2.2.1 Host Execution Context

1. There exist two types of **Host Execution Context** ( $EC_{HST}$ ):
  - Local Threads – these may have **PTs** (but no **SCs**) bound to it.
  - Global Threads – these may have an **SC** (but no **PTs**) bound to it.
2. A **Host Execution Context** has a **UTCB** that enables it to perform regular **IPC**.
3. Upon creation, a **Host Execution Context** is permanently bound to the following **required** spaces of its **PD**:
  - The first **Object Space**\*
  - The first **Host Space**\*
  - The first **PIO Space**<sup>†</sup>
4. A **Host Execution Context** cannot be reassigned to any spaces.

### 2.2.2 Guest Execution Context

1. There exists one type of **Guest Execution Context** ( $EC_{GST}$ ):
  - Virtual CPUs – these may have an **SC** (but no **PTs**) bound to it.
2. A **Guest Execution Context** does not have a **UTCB**.
3. Upon creation, a **Guest Execution Context** is permanently bound to the following **required** spaces of its **PD**:
  - The first **Object Space**\*
  - The first **Host Space**\*
4. Upon any architectural/microhypervisor event, a **Guest Execution Context** may be (re)assigned to the following spaces of its **PD**, by setting the **SPACES** bit in the **MTD** and providing selectors that refer to capabilities with **ASSIGN** permission for these spaces in the respective **UTCB** fields during **ipc\_reply**:
  - Any **Guest Space**<sup>†</sup>
  - Any **PIO Space**<sup>†</sup>
  - Any **MSR Space**<sup>†</sup>

The handler of the **STARTUP** event must perform the initial assignment of these spaces.

---

\*Because only one such space can be created for a **PD**, it is also the only such space.

<sup>†</sup>Only on architectures, where this type of space is **available**.

## 2.3 Scheduling Context

1. The **Scheduling Context (SC)** is a unit of prioritization and temporal isolation.
2. Access to a **Scheduling Context** is controlled by an **SC Capability** ( $CAP_{OBJ_{SC}}$ ).
3. An **SC** is permanently bound to one **CPU**.
4. An **SC** is permanently bound to the **EC** for which it was created.
5. Donation allows another **EC** to consume the budget of the **SC** for the duration of the donation.
6. A scheduling context comprises the following state:
  - Reference to bound **EC** (2.2)
  - Class Of Service (**COS**)
  - Priority – numerically higher priorities always preempt numerically lower priorities
  - Budget – time after which the **SC** can be preempted by an **SC** with the same priority

## 2.4 Portal

1. A **Portal (PT)** represents a dedicated entry point into the **PD** for which the portal was created.
2. Access to a **Portal** is controlled by a **PT Capability** ( $CAP_{OBJ_{PT}}$ ).
3. A **PT** is permanently bound to the **EC** for which it was created.
4. A portal comprises the following state:
  - Reference to bound **EC** (2.2)
  - **Message Transfer Descriptor (MTD)** (4.4)
  - Entry Instruction Pointer (**IP**)
  - Portal Identifier (**PID**)

## 2.5 Semaphore

1. A **Semaphore (SM)** provides a means to synchronize execution and interrupt delivery by selectively blocking and unblocking **Execution Contexts (ECs)**.
2. Access to a **Semaphore** is controlled by a **SM Capability** ( $CAP_{OBJ_{SM}}$ ).

## 2.6 Device Context

1. A **Device Context (DC)** provides management information for a hardware device.
2. Access to a **Device Context** is controlled by a **DC Capability** ( $CAP_{OBJ_{DC}}$ ).
3. A **Device Context** comprises the following state:
  - Topology of the device, such as **PCI Segment/Bus/Device/Function (SBDF)**
  - Reference to the component that performs **DMA** translation for the device, such as **IOMMU** or **SMMU**
  - Reference to the component that performs **MSI** translation for the device, such as **IOMMU** or **GITS**
  - Auxiliary information

## 3 Hardware Resources

### 3.1 System Time Counter

The system time is represented by an unsigned 64-bit [System Time Counter \(STC\)](#) with the following properties:

1. The [STC](#) starts with a power-on value of 0.
2. Subsequent reads of the [STC](#) return a higher value that reflects the platform uptime.
3. While the platform is in a shallow sleep state, the [STC](#) retains its current value.
4. While the platform is running, the [STC](#) monotonically increments at a fixed frequency, which is conveyed in the [Hypervisor Information Page \(HIP\)](#).
5. The [STC](#) and its frequency are synchronized across all [CPUs](#). Applications can use both values to convert between system time and wall clock time.
6. Applications can obtain the current [STC](#) value as follows:

**Arm:** By reading CNTVCT\_EL0 via the MRS instruction [3].

**x86:** By reading IA32\_TSC via the RDTSC instruction [4, 5].



## **Part III**

# **Application Programming Interface**

# 4 Data Types

## 4.1 Capability

**Capabilities** are communicable, unforgeable tokens of authority. They consist of a reference to a resource coupled with access permissions. **Capabilities** are opaque and immutable for applications – they cannot be inspected or modified directly; instead applications refer to a **Capability** (**CAP**) via a **Capability Selector** (**SEL**).

### 4.1.1 Null Capability

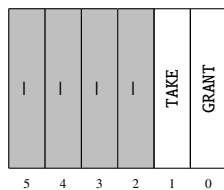
A **Null Capability** ( $CAP_0$ ) does not refer to anything and carries no permissions.

### 4.1.2 Object Capability

An **Object Capability** ( $CAP_{OBJ}$ ) is stored in the **Object Space** of a **PD** and refers to a kernel object.

#### 4.1.2.1 Object Space Capability

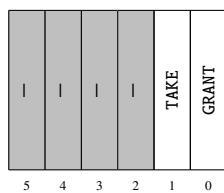
An **Object Space Capability** ( $CAP_{OBJ_{OBJ}}$ ) refers to an **Object Space** and carries the following permissions:



**GRANT** If set, **ctrl\_pd** can use that **Object Space** as destination.  
**TAKE** If set, **ctrl\_pd** can use that **Object Space** as source.

#### 4.1.2.2 Host Space Capability

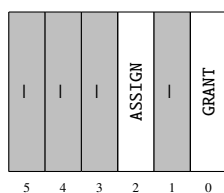
A **Host Space Capability** ( $CAP_{OBJ_{HST}}$ ) refers to a **Host Space** and carries the following permissions:



**GRANT** If set, **ctrl\_pd** can use that **Host Space** as destination.  
**TAKE** If set, **ctrl\_pd** can use that **Host Space** as source.

#### 4.1.2.3 Guest Space Capability

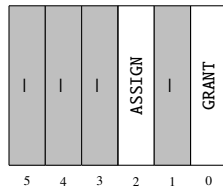
A **Guest Space Capability** ( $CAP_{OBJ_{GST}}$ ) refers to a **Guest Space** and carries the following permissions:



**GRANT** If set, **ctrl\_pd** can use that **Guest Space** as destination.  
**ASSIGN** If set, **ipc\_reply** can assign an  $EC_{GST}$  to that **Guest Space**.

#### 4.1.2.4 DMA Space Capability

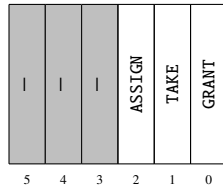
A **DMA Space Capability** ( $CAP_{OBJ_{DMA}}$ ) refers to a **DMA Space** and carries the following permissions:



GRANT If set, **ctrl\_pd** can use that **DMA Space** as destination.  
 ASSIGN If set, **assign\_dev** can assign a device to that **DMA Space**.

#### 4.1.2.5 PIO Space Capability

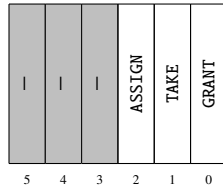
A **PIO Space Capability** ( $CAP_{OBJ_{PIO}}$ ) refers to a **PIO Space** and carries the following permissions:



GRANT If set, **ctrl\_pd** can use that **PIO Space** as destination.  
 TAKE If set, **ctrl\_pd** can use that **PIO Space** as source.  
 ASSIGN If set, **ipc\_reply** can assign an **EC<sub>GST</sub>** to that **PIO Space**.

#### 4.1.2.6 MSR Space Capability

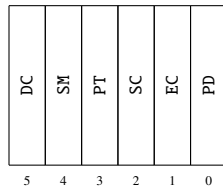
An **MSR Space Capability** ( $CAP_{OBJ_{MSR}}$ ) refers to an **MSR Space** and carries the following permissions:



GRANT If set, **ctrl\_pd** can use that **MSR Space** as destination.  
 TAKE If set, **ctrl\_pd** can use that **MSR Space** as source.  
 ASSIGN If set, **ipc\_reply** can assign an **EC<sub>GST</sub>** to that **MSR Space**.

#### 4.1.2.7 Protection Domain Capability

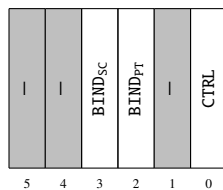
A **PD Capability** ( $CAP_{OBJ_{PD}}$ ) refers to a **Protection Domain (PD)** and carries the following permissions:



PD If set, **create\_pd** permitted.  
 EC If set, **create\_ec** permitted.  
 SC If set, **create\_sc** permitted.  
 PT If set, **create\_pt** permitted.  
 SM If set, **create\_sm** permitted.  
 DC If set, **create\_dc** permitted.

#### 4.1.2.8 Execution Context Capability

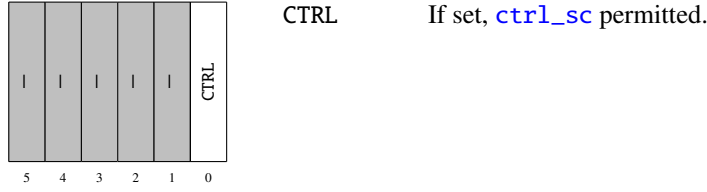
An **EC Capability** ( $CAP_{OBJ_{EC}}$ ) refers to an **Execution Context (EC)** and carries the following permissions:



CTRL If set, **ctrl\_ec** permitted.  
 BIND<sub>PT</sub> If set, **create\_pt** can bind a **Portal (PT)** to the **EC**.  
 BIND<sub>SC</sub> If set, **create\_sc** can bind a **Scheduling Context (SC)** to the **EC**.

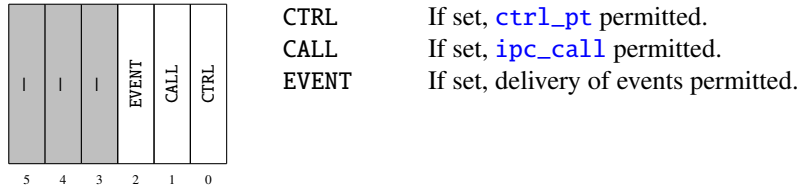
#### 4.1.2.9 Scheduling Context Capability

An **SC Capability** ( $CAP_{OBJ_{SC}}$ ) refers to a **Scheduling Context (SC)** and carries the following permissions:



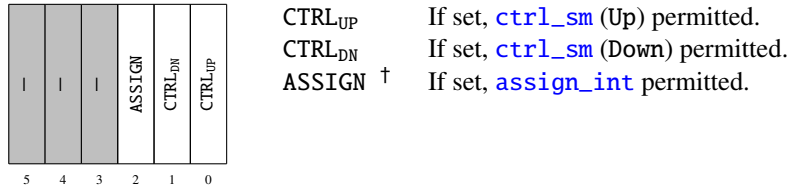
#### 4.1.2.10 Portal Capability

A **PT Capability** ( $CAP_{OBJ_{PT}}$ ) refers to a **Portal (PT)** and carries the following permissions:



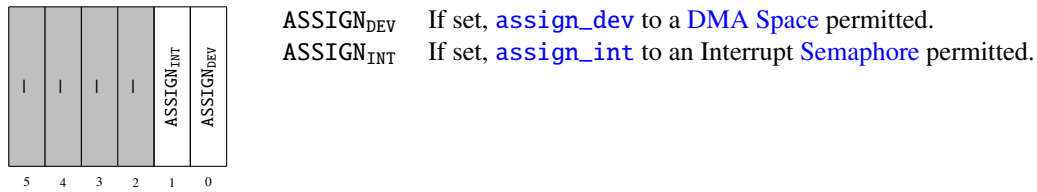
#### 4.1.2.11 Semaphore Capability

An **SM Capability** ( $CAP_{OBJ_{SM}}$ ) refers to a **Semaphore (SM)** and carries the following permissions:



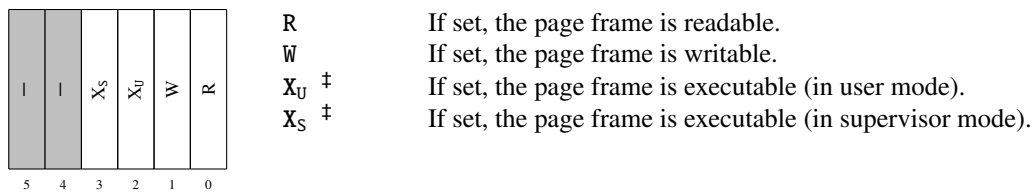
#### 4.1.2.12 Device Context Capability

A **DC Capability** ( $CAP_{OBJ_{DC}}$ ) refers to a **Device Context (DC)** and carries the following permissions:



### 4.1.3 Memory Capability

A **Memory Capability** ( $CAP_{MEM}$ ) is stored in a **Host Space**, **Guest Space** or **DMA Space** of a **PD**, refers to a 4 KiB page frame, and carries the following permissions:

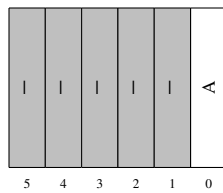


<sup>†</sup>This permission bit is only defined for interrupt semaphores.

<sup>‡</sup>If the hardware supports only combined execute permissions (X) for both modes, then  $X = X_U \vee X_S$ .

#### 4.1.4 PIO Capability

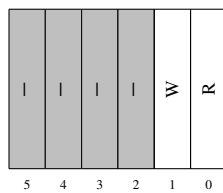
A **PIO Capability** ( $CAP_{PIO}$ ) is stored in a **PIO Space** of a **PD**, refers to an I/O port, and carries the following permissions:



**A** If set, the I/O port is accessible (via IN/OUT).

#### 4.1.5 MSR Capability

An **MSR Capability** ( $CAP_{MSR}$ ) is stored in an **MSR Space** of a **PD**, refers to a Model-Specific Register (**MSR**), and carries the following permissions:



**R** If set, the **MSR** is readable (via RDMSR).  
**W** If set, the **MSR** is writable (via WRMSR).

## 4.2 Capability Selector

A [Capability Selector \(SEL\)](#) is an application-visible unsigned number that serves as an index into a space of an [Execution Context](#) to select a [Capability \(CAP\)](#). The following subsections provide more details.

### 4.2.1 Object Capability Selector

1. [Object Spaces \(SPC<sub>OBJ</sub>\)](#) are unavailable if  $SBW_{OBJ} = 0$ . No systems currently report this.
2. Otherwise, a valid [Object Capability Selector \(SEL<sub>OBJ</sub>\)](#) is in range  $[0, 2^{SBW_{OBJ}})$ .

### 4.2.2 Host Capability Selector

1. [Host Spaces \(SPC<sub>HST</sub>\)](#) are unavailable if  $SBW_{HST} = 0$ . No systems currently report this.
2. Otherwise, a valid [Host Capability Selector \(SEL<sub>HST</sub>\)](#) is in range  $[0, 2^{SBW_{HST}})$ .  $\implies$  valid [HVA](#) range

### 4.2.3 Guest Capability Selector

1. [Guest Spaces \(SPC<sub>GST</sub>\)](#) are unavailable if  $SBW_{GST} = 0$ . Systems without [CPU](#) virtualization may report this.
2. Otherwise, a valid [Guest Capability Selector \(SEL<sub>GST</sub>\)](#) is in range  $[0, 2^{SBW_{GST}})$ .  $\implies$  valid [GPA](#) range

### 4.2.4 DMA Capability Selector

1. [DMA Spaces \(SPC<sub>DMA</sub>\)](#) are unavailable if  $SBW_{DMA} = 0$ . Systems without I/O virtualization may report this.
2. Otherwise, a valid [DMA Capability Selector \(SEL<sub>DMA</sub>\)](#) is in range  $[0, 2^{SBW_{DMA}})$ .  $\implies$  valid [DVA](#) range

### 4.2.5 PIO Capability Selector

1. [PIO Spaces \(SPC<sub>PIO</sub>\)](#) are unavailable if  $SBW_{PIO} = 0$ . Non-x86 systems may report this.
2. Otherwise, a valid [PIO Capability Selector \(SEL<sub>PIO</sub>\)](#) is in range  $[0, 2^{SBW_{PIO}})$ .  $\implies$  valid [PIO](#) range

### 4.2.6 MSR Capability Selector

1. [MSR Spaces \(SPC<sub>MSR</sub>\)](#) are unavailable if  $SBW_{MSR} = 0$ . Non-x86 systems may report this.
2. Otherwise, a valid [MSR Capability Selector \(SEL<sub>MSR</sub>\)](#) is in range  $[0, 2^{SBW_{MSR}})$ .  $\implies$  valid [MSR](#) range

## 4.3 User Thread Control Block

A **User Thread Control Block (UTCb)** has a size of one memory page (4 KiB) and is mapped in the **Host Space** of its associated **Host Execution Context**. Because a **UTCb** is allocated and owned by the microhypervisor, it cannot be delegated using `ctrl_pd`.

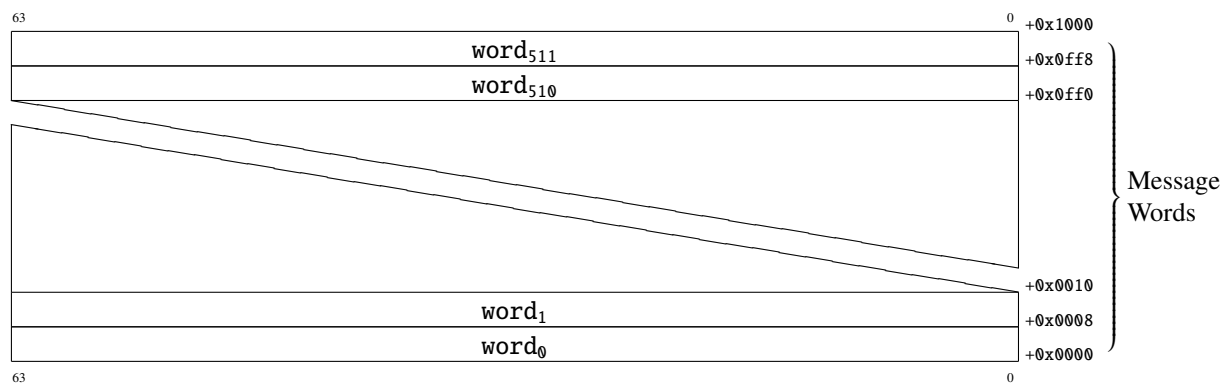
To ensure proper visibility of loads and stores with relaxed memory ordering, application programs are expected to access a **UTCb** only from the **Host Execution Context** to which that **UTCb** is bound.

### 4.3.1 Regular Layout

During regular **IPC** (see 4.4.1), the **UTCb** is used for data transfer.

The data transfer from one **UTCb** to another **UTCb** is defined as follows:

- The data transfer is performed by the **CPU** on which the caller **EC** and callee **EC** execute.
- The data transfer uses the regular layout with 512 message words (see below).
- The data is copied from low words to high words, beginning with `word0`.
- The granularity of the loads and stores used for copying is **undefined**.
- Loads from and stores to the **UTCb** are **non-atomic** and use **relaxed** memory ordering.



### 4.3.2 Architectural Layout

During architectural **IPC** (see 4.4.2), the **UTCb** is used for state transfer.

The state transfer between the architectural registers and a **UTCb** is defined as follows:

- The state transfer is performed by the **CPU** on which the affected **EC** and callee **EC** execute.
- The state transfer uses the architectural layout (**Arm**, **x86**).
- The state is copied between architectural registers and the **UTCb** in an **undefined** order.
- The granularity of the loads and stores used for copying is **undefined**.
- Loads from and stores to the **UTCb** are **non-atomic** and use **relaxed** memory ordering.

## 4.4 Message Transfer Descriptor

### 4.4.1 Regular IPC

For regular [Inter-Process Communication \(IPC\)](#), the [Message Transfer Descriptor \(MTD\)](#) is provided by the sender, passed to the receiver, and uses the following layout:

|    |                        |
|----|------------------------|
| -  | UTCB Message Words - 1 |
| 31 | 9 8 0                  |

The [MTD](#) controls the data transfer (see 4.3.1) as shown in Figure 4.1:

- During [ipc\\_call](#), it specifies the number of message words to transfer from the [UTCB](#) of the caller [EC](#) (sender) to the [UTCB](#) of the callee [EC](#) (receiver).
- During [ipc\\_reply](#), it specifies the number of message words to transfer from the [UTCB](#) of the callee [EC](#) (sender) to the [UTCB](#) of the caller [EC](#) (receiver).

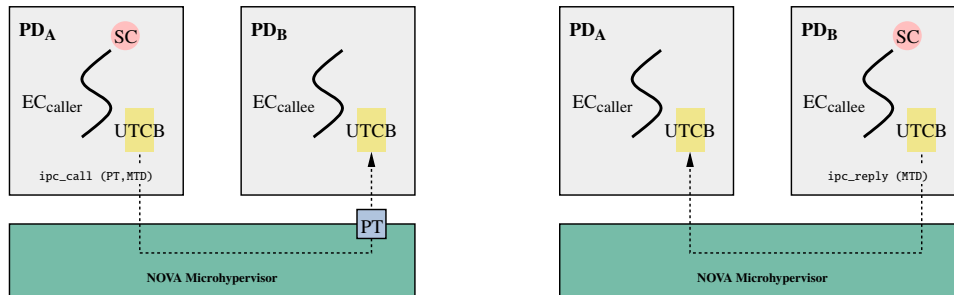


Figure 4.1: Regular [IPC](#)

### 4.4.2 Architectural IPC

For exceptions and intercepts, the [Message Transfer Descriptor \(MTD\)](#) is provided by the architectural event-specific portal ([Arm, x86](#)) or sender, passed to the receiver, and uses an architectural bitfield layout ([Arm, x86](#)):

- If a bit is 0, then the microhypervisor does **not** transmit the architectural state associated with that bit.
- If a bit is 1, then the microhypervisor transmits the architectural state associated with that bit.

The [MTD](#) controls the state transfer (see 4.3.2) as shown in Figure 4.2:

- During an exception/intercept, it specifies the subset of registers to transfer from the architectural state of the affected [EC](#) (sender) to the [UTCB](#) of the callee [EC](#) (receiver).
- During [ipc\\_reply](#), it specifies the subset of registers to transfer from the [UTCB](#) of the callee [EC](#) (sender) to the architectural state of the affected [EC](#) (receiver).

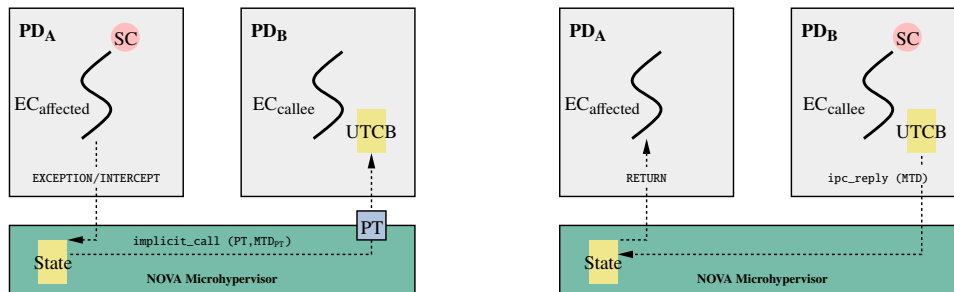
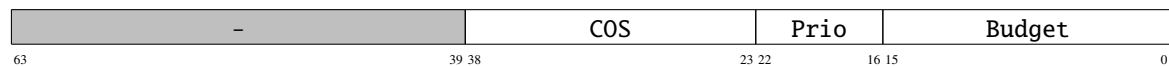


Figure 4.2: Architectural [IPC](#)



## 4.5 Scheduling Context Descriptor

The [Scheduling Context Descriptor \(SCD\)](#) describes the configuration of a [Scheduling Context \(SC\)](#).



The fields are defined as follows:

### **Budget**

Specifies the scheduling budget in milliseconds – must be > 0.

### **Prio**

Specifies the scheduling priority – must be > 0.

### **COS**

Specifies the Class Of Service – valid values depend on architectural [COS](#) support ([Arm](#), [x86](#)):

- If [COS](#) is not supported, then this field must be 0.
- If [COS](#) is supported, then this field must be < [COS](#)<sub>NUM</sub>.

# 5 Hypercalls

## 5.1 Definitions

### 5.1.1 Hypercall Numbers

Each hypercall is identified by a unique number. The following hypercalls are currently defined:

| Number | Hypercall                  | Section               |
|--------|----------------------------|-----------------------|
| 0x0    | <a href="#">ipc_call</a>   | <a href="#">5.2.1</a> |
| 0x1    | <a href="#">ipc_reply</a>  | <a href="#">5.2.2</a> |
| 0x2    | <a href="#">create_pd</a>  | <a href="#">5.3.1</a> |
| 0x3    | <a href="#">create_ec</a>  | <a href="#">5.3.2</a> |
| 0x4    | <a href="#">create_sc</a>  | <a href="#">5.3.3</a> |
| 0x5    | <a href="#">create_pt</a>  | <a href="#">5.3.4</a> |
| 0x6    | <a href="#">create_sm</a>  | <a href="#">5.3.5</a> |
| 0x7    | <a href="#">create_dc</a>  | <a href="#">5.3.6</a> |
| 0x8    | <a href="#">ctrl_pd</a>    | <a href="#">5.4.1</a> |
| 0x9    | <a href="#">ctrl_ec</a>    | <a href="#">5.4.2</a> |
| 0xa    | <a href="#">ctrl_sc</a>    | <a href="#">5.4.3</a> |
| 0xb    | <a href="#">ctrl_pt</a>    | <a href="#">5.4.4</a> |
| 0xc    | <a href="#">ctrl_sm</a>    | <a href="#">5.4.5</a> |
| 0xd    | <a href="#">ctrl_hw</a>    | <a href="#">5.5.1</a> |
| 0xe    | <a href="#">assign_dev</a> | <a href="#">5.5.2</a> |
| 0xf    | <a href="#">assign_int</a> | <a href="#">5.5.3</a> |

### 5.1.2 Status Codes

Hypercalls return a status code to indicate success or failure. The following status codes are currently defined:

| Number | Status Code                    | Description                               |
|--------|--------------------------------|-------------------------------------------|
| 0x0    | SUCCESS                        | Operation Successful                      |
| 0x1    | TIMEOUT                        | Operation Timeout                         |
| 0x2    | ABORTED                        | Operation Abort                           |
| 0x3    | OVRFLOW                        | Operation Overflow                        |
| 0x4    | BAD_HYP                        | Invalid Hypercall                         |
| 0x5    | BAD_CAP                        | Invalid Capability                        |
| 0x6    | BAD_PAR                        | Invalid Parameter                         |
| 0x7    | BAD_FTR                        | Invalid Feature                           |
| 0x8    | BAD_CPU                        | Invalid CPU Number                        |
| 0x9    | BAD_DEV                        | Invalid Device ID                         |
| 0xa    | MEM_OBJ                        | Insufficient Memory (Object Creation)     |
| 0xb    | MEM_CAP                        | Insufficient Memory (Capability Creation) |
| ≥0xc   | <i>reserved for future use</i> |                                           |

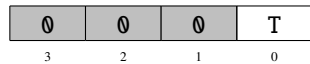
## 5.2 Communication

### 5.2.1 IPC Call

#### Parameters:

```
status = ipc_call (SELOBJ pt,          // Portal
                  MTD& mtd)           // Message Transfer Descriptor
```

#### Flags:



#### Description:

Sends a message from **EC<sub>CURRENT</sub>** (caller) to the **EC** (callee) to which the specified **Portal (PT)** is bound.

Prior to the hypercall:

- **SPC<sub>OBJ<sub>CURRENT</sub></sub>[pt]** must refer to a **PT Capability (CAP<sub>OBJ<sub>PT</sub></sub>)** with permission **CALL**.

If the hypercall completed successfully:

- If **T=0 (No Timeout)**: If the callee **EC** was still busy handling a prior **ipc\_call**, then the caller **EC** has helped run that prior **ipc\_call** to completion, i.e. until the callee **EC** became available again.
- The microhypervisor has transferred a message from the **UTCB** of the caller **EC** to the **UTCB** of the callee **EC**. The content of that message is defined by the **MTD mtd**, which has been passed from the caller **EC** to the callee **EC**.
- The hypercall returns once the callee **EC** has invoked an **ipc\_reply**. Upon return, the **UTCB** of the caller **EC** and the **mtd** parameter have been updated by the reply message.
- The Current Scheduling Context (**SC<sub>CURRENT</sub>**) has been donated to the callee **EC** upon **ipc\_call** and returned back upon **ipc\_reply**, thereby accounting the entire handling of the request to **SC<sub>CURRENT</sub>**.

#### Status:

##### SUCCESS

- The hypercall completed successfully.

##### BAD\_CAP

- **SPC<sub>OBJ<sub>CURRENT</sub></sub>[pt]** did not refer to a **PT Capability (CAP<sub>OBJ<sub>PT</sub></sub>)** or that capability had insufficient permissions.

##### BAD\_CPU

- Caller **EC** and callee **EC** are on different **CPU**s.

##### TIMEOUT

- If **T=1 (Timeout)**: The callee **EC** is still busy handling a prior **ipc\_call**.

##### ABORTED

- The callee **EC** is dead and the operation aborted.

## 5.2.2 IPC Reply

### Parameters:

`pid = ipc_reply (MTD & mtd)` // Message Transfer Descriptor

### Flags:

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 3 | 2 | 1 | 0 |

### Description:

Sends a reply message from `ECCURRENT` (callee) back to the caller `EC` (if one exists) and subsequently waits for the next incoming message.

If the hypercall completed successfully:

- If a caller `EC` exists:
  - The microhypervisor has transferred a reply message from the `UTCB` of the callee `EC` back to the `UTCB` of the caller `EC`.
  - The content of that reply message is defined by the `MTD mtd`, which has been passed from the callee `EC` back to the caller `EC`.
  - The Current Scheduling Context (`SCCURRENT`) that had been donated to the callee `EC` upon `ipc_call` has been returned back to the caller `EC`.
- `ECCURRENT` blocks until the next incoming message arrives on any `Portal (PT)` bound to it.

### Status:

This hypercall does not return directly.

Instead, when the next message arrives via a subsequent `ipc_call` to any `Portal (PT)` bound to the callee `EC`:

- The microhypervisor passes the Portal Identifier (`PID`) of the called `PT` to the callee `EC`.
- The `UTCB` of the callee `EC` and the `mtd` parameter have been updated by the incoming message.
- Execution of the callee `EC` continues at the Instruction Pointer (`IP`) configured in the called `PT`.

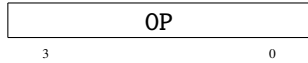
## 5.3 Object Creation

### 5.3.1 Create Protection Domain

Parameters:

```
status = create_pd (SEL_OBJ sel,          // Created PD or Space
                   SEL_OBJ pd)           // Owner PD
```

Flags:



Description:

Creates a new **Protection Domain (PD)** or an empty new space for a **Protection Domain**.

Prior to the hypercall:

- $SPC_{OBJ\_CURRENT}[sel]$  must refer to a **Null Capability ( $CAP_0$ )**.
- $SPC_{OBJ\_CURRENT}[pd]$  must refer to a **PD Capability ( $CAP_{OBJ\_PD}$ )** with permission PD.

If the hypercall completed successfully:

- If **OP=0 (Protection Domain)**:
  - A new **Protection Domain (PD)** has been created.
  - $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_PD}$  for the created **Protection Domain** with the **permissions** inherited from  $SPC_{OBJ\_CURRENT}[pd]$ .
- If **OP=1 (Object Space)**:
  - A new **Object Space ( $SPC_{OBJ}$ )** has been created for the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .
  - $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_OBJ}$  for the created **Object Space** with all **defined** permissions.
- If **OP=2 (Host Space)**:
  - A new **Host Space ( $SPC_{HST}$ )** has been created for the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .
  - $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_HST}$  for the created **Host Space** with all **defined** permissions.
- If **OP=3 (Guest Space)**:
  - A new **Guest Space ( $SPC_{GST}$ )** has been created for the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .
  - $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_GST}$  for the created **Guest Space** with all **defined** permissions.
- If **OP=4 (DMA Space)**:
  - A new **DMA Space ( $SPC_{DMA}$ )** has been created for the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .
  - $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_DMA}$  for the created **DMA Space** with all **defined** permissions.
- If **OP=5 (PIO Space)**:
  - A new **PIO Space ( $SPC_{PIO}$ )** has been created for the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .
  - $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_PIO}$  for the created **PIO Space** with all **defined** permissions.
- If **OP=6 (MSR Space)**:
  - A new **MSR Space ( $SPC_{MSR}$ )** has been created for the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .
  - $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_MSR}$  for the created **MSR Space** with all **defined** permissions.
- The resources for the created object were accounted to the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .

Status:

#### SUCCESS

- The hypercall completed successfully.

**ABORTED**

- The **PD** designated by  $SPC_{OBJ_{CURRENT}}[pd]$  is being destructed concurrently.
- If  $OP > 0$ : The space could not bind to the **PD** designated by  $SPC_{OBJ_{CURRENT}}[pd]$ .

**BAD\_CAP**

- $SPC_{OBJ_{CURRENT}}[sel]$  did not refer to a **Null Capability** ( $CAP_0$ ).
- $SPC_{OBJ_{CURRENT}}[pd]$  did not refer to a **PD Capability** ( $CAP_{OBJ_{PD}}$ ) or that capability had insufficient permissions.

**BAD\_PAR**

- The requested operation (OP) is invalid.

**BAD\_FTR**

- The requested space type is not supported by the hardware architecture.

**MEM\_OBJ**

- The **Protection Domain** referred to by  $SPC_{OBJ_{CURRENT}}[pd]$  had insufficient memory resources for object creation.

**MEM\_CAP**

- The **Protection Domain** to which  $SPC_{OBJ_{CURRENT}}$  belongs had insufficient memory resources for capability creation.

### 5.3.2 Create Execution Context

#### Parameters:

```
status = create_ec (SEL_OBJ sel,          // Created EC
                   SEL_OBJ pd,          // Owner PD
                   SEL_HST hvp,         // Host-Virtual Page Number
                   SEL_EVT evt,         // Event Selector Base
                   UINT_16 cpu,         // CPU Number
                   UINT_PTR sp)         // Initial Stack Pointer
```

#### Flags:

|   |   |   |   |
|---|---|---|---|
| 0 | F | T | G |
| 3 | 2 | 1 | 0 |

#### Description:

Creates a new [Execution Context \(EC\)](#).

Prior to the hypercall:

- $SPC_{OBJ_{CURRENT}}[sel]$  must refer to a [Null Capability \(CAP<sub>0</sub>\)](#).
- $SPC_{OBJ_{CURRENT}}[pd]$  must refer to a [PD Capability \(CAP<sub>OBJ<sub>PD</sub></sub>\)](#) with permission EC.
- If **G=0**: All spaces [required](#) for a [Host Execution Context](#) must have been created for the owner [PD](#).
- If **G=1**: All spaces [required](#) for a [Guest Execution Context](#) must have been created for the owner [PD](#).

If the hypercall completed successfully:

- If **G=0 (Host Execution Context)**:
  - A new [Host Execution Context \(EC<sub>HST</sub>\)](#) has been created.
  - The microhypervisor has allocated a [UTCB](#) for [EC<sub>HST</sub>](#) and mapped it at [HVA hvp << 12](#).
  - If **T=0 (Local Thread)**: [Portals \(PTs\)](#) may subsequently be bound to [EC<sub>HST</sub>](#), which will run whenever any of those bound portals is called.
  - If **T=1 (Global Thread)**: [EC<sub>HST</sub>](#) will generate a [STARTUP](#) event the first time a [Scheduling Context \(SC\)](#) is bound to it.
- If **G=1 (Guest Execution Context)**:
  - A new [Guest Execution Context \(EC<sub>GST</sub>\)](#) has been created.
  - The microhypervisor has allocated a [vAPIC](#) page for [EC<sub>GST</sub>](#) and mapped it at [HVA hvp << 12](#). On non-Intel architectures, the parameter [hvp](#) was ignored.
  - [EC<sub>GST</sub>](#) will generate a [STARTUP](#) event the first time a [Scheduling Context \(SC\)](#) is bound to it.
  - If **T=0 (Virtual CPU)**: The virtual CPU uses no time adjustment.
  - If **T=1 (Virtual CPU)**: The virtual CPU uses time offsetting.
- The created [EC](#) will be able to use [FPU](#) instructions only if **F=1 (FPU)**. Otherwise any [FPU](#) access by that [EC](#) will generate an architecture-specific exception.
- The created [EC](#) has its [Stack Pointer \(SP\)](#) set to [sp](#) and its [Event Selector Base \(SEL<sub>EVNT</sub>\)](#) set to [evt](#).<sup>†</sup>
- The created [EC](#) is permanently bound to the [CPU](#) [cpu](#) and to the required spaces of the [PD](#) referred to by  $SPC_{OBJ_{CURRENT}}[pd]$ .
- The resources for the created [EC](#) were accounted to the [PD](#) referred to by  $SPC_{OBJ_{CURRENT}}[pd]$ .
- $SPC_{OBJ_{CURRENT}}[sel]$  refers to a [CAP<sub>OBJ<sub>EC</sub></sub>](#) for the created [EC](#) with all [defined](#) permissions.

<sup>†</sup>The microhypervisor sets these values only once during [EC](#) creation. Subsequently, each [Host Execution Context](#) is responsible for maintaining its [Stack Pointer \(SP\)](#) across hypercalls. Applications can use different initial [SP](#) or [SEL<sub>EVNT</sub>](#) values as a means to identify each [Host Execution Context](#) or [Guest Execution Context](#) during concurrent [STARTUP](#) events.

**Status:****SUCCESS**

- The hypercall completed successfully.

**ABORTED**

- A required space of the PD designated by  $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{pd}]$  is being destructed concurrently.

**BAD\_CAP**

- $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{sel}]$  did not refer to a Null Capability ( $\text{CAP}_0$ ).
- $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{pd}]$  did not refer to a PD Capability ( $\text{CAP}_{\text{OBJ}_{\text{PD}}}$ ) or that capability had insufficient permissions.

**BAD\_CPU**

- The CPU number is invalid.

**BAD\_FTR**

- Virtual CPUs are not supported by the hardware architecture.

**BAD\_PAR**

- The HVA corresponding to hvp is outside the user-accessible memory range.

**MEM\_OBJ**

- The Protection Domain referred to by  $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{pd}]$  had insufficient memory resources for object creation.

**MEM\_CAP**

- The Protection Domain to which  $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}$  belongs had insufficient memory resources for capability creation.

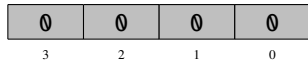


### 5.3.3 Create Scheduling Context

Parameters:

```
status = create_sc (SEL_OBJ sel,           // Created SC
                   SEL_OBJ pd,           // Owner PD
                   SEL_OBJ ec,           // Bound EC
                   SCD   scd)           // Scheduling Context Descriptor
```

Flags:



Description:

Creates a new **Scheduling Context (SC)**.

Prior to the hypercall:

- $SPC_{OBJ_{CURRENT}}[sel]$  must refer to a **Null Capability ( $CAP_0$ )**.
- $SPC_{OBJ_{CURRENT}}[pd]$  must refer to a **PD Capability ( $CAP_{OBJ_{PD}}$ )** with permission SC.
- $SPC_{OBJ_{CURRENT}}[ec]$  must refer to an **EC Capability ( $CAP_{OBJ_{EC}}$ )** with permission  $BIND_{SC}$ .
- The owner **PD** designated by  $CAP_{OBJ_{PD}}$  and the owner **PD** of the bound **EC** must be the same.

If the hypercall completed successfully:

- A new **Scheduling Context (SC)** has been created.
- The created **SC** is bound to the **EC** referred to by  $SPC_{OBJ_{CURRENT}}[ec]$  on the **CPU** of that **EC**, with its scheduling parameters set according to  $scd$ .
- The resources for the created **SC** were accounted to the **PD** referred to by  $SPC_{OBJ_{CURRENT}}[pd]$ .
- $SPC_{OBJ_{CURRENT}}[sel]$  refers to a  $CAP_{OBJ_{SC}}$  for the created **SC** with all **defined** permissions.

Status:

#### SUCCESS

- The hypercall completed successfully.

#### ABORTED

- The **EC** designated by  $SPC_{OBJ_{CURRENT}}[ec]$  is being destructed concurrently.

#### BAD\_CAP

- $SPC_{OBJ_{CURRENT}}[sel]$  did not refer to a **Null Capability ( $CAP_0$ )**.
- $SPC_{OBJ_{CURRENT}}[pd]$  did not refer to a **PD Capability ( $CAP_{OBJ_{PD}}$ )** or that capability had insufficient permissions.
- $SPC_{OBJ_{CURRENT}}[ec]$  did not refer to an **EC Capability ( $CAP_{OBJ_{EC}}$ )** or that capability had insufficient permissions.
- Binding the **SC** to the **EC** failed, e.g. because the **EC** is a local thread.

#### BAD\_PAR

- At least one **SCD** field in  $scd$  was invalid.

#### MEM\_OBJ

- The **Protection Domain** referred to by  $SPC_{OBJ_{CURRENT}}[pd]$  had insufficient memory resources for object creation.

#### MEM\_CAP

- The **Protection Domain** to which  $SPC_{OBJ_{CURRENT}}$  belongs had insufficient memory resources for capability creation.

### 5.3.4 Create Portal

#### Parameters:

```
status = create_pt (SEL_OBJ sel,          // Created PT
                   SEL_OBJ pd,          // Owner PD
                   SEL_OBJ ec,          // Bound EC
                   UINT_PTR ip)         // Instruction Pointer
```

#### Flags:

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 3 | 2 | 1 | 0 |

#### Description:

Creates a new **Portal (PT)**.

Prior to the hypercall:

- $SPC_{OBJ\_CURRENT}[sel]$  must refer to a **Null Capability ( $CAP_0$ )**.
- $SPC_{OBJ\_CURRENT}[pd]$  must refer to a **PD Capability ( $CAP_{OBJ\_PD}$ )** with permission PT.
- $SPC_{OBJ\_CURRENT}[ec]$  must refer to an **EC Capability ( $CAP_{OBJ\_EC}$ )** with permission  $BIND_{PT}$ .
- The owner **PD** designated by  $CAP_{OBJ\_PD}$  and the owner **PD** of the bound **EC** must be the same.

If the hypercall completed successfully:

- A new **Portal (PT)** has been created.
- The created **PT** is bound to the **EC** referred to by  $SPC_{OBJ\_CURRENT}[ec]$  on the **CPU** of that **EC**, with its Portal Instruction Pointer (**IP**) set to `ip`, its initial **MTD** set to 0 and its initial **PID** set to 0.
- The resources for the created **PT** were accounted to the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .
- $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_PT}$  for the created **PT** with all **defined** permissions.

#### Status:

##### SUCCESS

- The hypercall completed successfully.

##### ABORTED

- The **EC** designated by  $SPC_{OBJ\_CURRENT}[ec]$  is being destructed concurrently.

##### BAD\_CAP

- $SPC_{OBJ\_CURRENT}[sel]$  did not refer to a **Null Capability ( $CAP_0$ )**.
- $SPC_{OBJ\_CURRENT}[pd]$  did not refer to a **PD Capability ( $CAP_{OBJ\_PD}$ )** or that capability had insufficient permissions.
- $SPC_{OBJ\_CURRENT}[ec]$  did not refer to an **EC Capability ( $CAP_{OBJ\_EC}$ )** or that capability had insufficient permissions.
- Binding the **PT** to the **EC** failed, e.g. because the **EC** is not a local thread.

##### MEM\_OBJ

- The **Protection Domain** referred to by  $SPC_{OBJ\_CURRENT}[pd]$  had insufficient memory resources for object creation.

##### MEM\_CAP

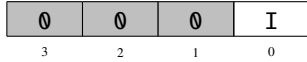
- The **Protection Domain** to which  $SPC_{OBJ\_CURRENT}$  belongs had insufficient memory resources for capability creation.

## 5.3.5 Create Semaphore

### Parameters:

```
status = create_sm (SEL_OBJ sel,           // Created SM
                   SEL_OBJ pd,           // Owner PD
                   UINT64 val)          // Initial Counter Value / ISD
```

### Flags:



### Description:

Creates a new **Semaphore (SM)**.

Prior to the hypercall:

- $SPC_{OBJ_{CURRENT}}[sel]$  must refer to a **Null Capability ( $CAP_0$ )**.
- If **I=0 (Regular Semaphore)**
  - $SPC_{OBJ_{CURRENT}}[pd]$  must refer to a **PD Capability ( $CAP_{OBJ_{PD}}$ )** with permission SM.
- If **I=1 (Interrupt Semaphore)**
  - $SPC_{OBJ_{CURRENT}}[pd]$  must refer to a **PD Capability ( $CAP_{OBJ_{PD}}$ )** with permission DC.
  - The Interrupt Semaphore Descriptor (**ISD**) must be valid.

If the hypercall completed successfully:

- If **I=0 (Regular Semaphore)**
  - A new regular **Semaphore (SM)** has been created.
  - The regular **Semaphore** has an initial counter value of `val`.
  - The regular **Semaphore** is not associated with any interrupt.
- If **I=1 (Interrupt Semaphore)**
  - A new interrupt **Semaphore (SM)** has been created.
  - The interrupt **Semaphore** has an initial counter value of 0.
  - The interrupt **Semaphore** is permanently associated with the interrupt designated by **ISD**.
- The resources for the created **SM** were accounted to the **PD** referred to by  $SPC_{OBJ_{CURRENT}}[pd]$ .
- $SPC_{OBJ_{CURRENT}}[sel]$  refers to a  $CAP_{OBJ_{SM}}$  for the created **SM** with all **defined** permissions.

### Status:

#### SUCCESS

- The hypercall completed successfully.

#### ABORTED

- The **PD** designated by  $SPC_{OBJ_{CURRENT}}[pd]$  is being destructed concurrently.

#### BAD\_CAP

- $SPC_{OBJ_{CURRENT}}[sel]$  did not refer to a **Null Capability ( $CAP_0$ )**.
- $SPC_{OBJ_{CURRENT}}[pd]$  did not refer to a **PD Capability ( $CAP_{OBJ_{PD}}$ )** or that capability had insufficient permissions.

#### BAD\_PAR

- If **I=1**: The Interrupt Semaphore Descriptor (**ISD**) was invalid.

#### MEM\_OBJ

- The **Protection Domain** referred to by  $SPC_{OBJ_{CURRENT}}[pd]$  had insufficient memory resources for object creation.

#### MEM\_CAP

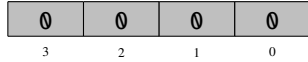
- The **Protection Domain** to which  $SPC_{OBJ_{CURRENT}}$  belongs had insufficient memory resources for capability creation.

## 5.3.6 Create Device Context

### Parameters:

```
status = create_dc (SEL_OBJ sel,           // Created DC
                   SEL_OBJ pd,           // Owner PD
                   DTD dtd)              // Device Topology Descriptor
```

### Flags:



### Description:

Creates a new **Device Context (DC)**.

Prior to the hypercall:

- $SPC_{OBJ\_CURRENT}[sel]$  must refer to a **Null Capability ( $CAP_0$ )**.
- $SPC_{OBJ\_CURRENT}[pd]$  must refer to a **PD Capability ( $CAP_{OBJ\_PD}$ )** with permission DC.

If the hypercall completed successfully:

- A new **Device Context (DC)** has been created.
- The created **DC** has its topology information configured according to **DTD**.
- The resources for the created **DC** were accounted to the **PD** referred to by  $SPC_{OBJ\_CURRENT}[pd]$ .
- $SPC_{OBJ\_CURRENT}[sel]$  refers to a  $CAP_{OBJ\_DC}$  for the created **DC** with all **defined** permissions.

### Status:

#### SUCCESS

- The hypercall completed successfully.

#### ABORTED

- The **PD** designated by  $SPC_{OBJ\_CURRENT}[pd]$  is being destructed concurrently.

#### BAD\_CAP

- $SPC_{OBJ\_CURRENT}[sel]$  did not refer to a **Null Capability ( $CAP_0$ )**.
- $SPC_{OBJ\_CURRENT}[pd]$  did not refer to a **PD Capability ( $CAP_{OBJ\_PD}$ )** or that capability had insufficient permissions.

#### MEM\_OBJ

- The **Protection Domain** referred to by  $SPC_{OBJ\_CURRENT}[pd]$  had insufficient memory resources for object creation.

#### MEM\_CAP

- The **Protection Domain** to which  $SPC_{OBJ\_CURRENT}$  belongs had insufficient memory resources for capability creation.

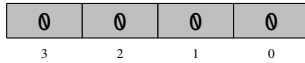
## 5.4 Object Control

### 5.4.1 Control Protection Domain

Parameters:

```
status = ctrl_pd (SEL_OBJ src,          // SRC Space
                  SEL_OBJ dst,          // DST Space
                  SEL   ssb,            // SRC Selector Base
                  SEL   dsb,            // DST Selector Base
                  UINT5 ord,            // Order
                  UINT6 pmm,            // Permission Mask
                  MAD    mad)           // Memory Attribute Descriptor
```

Flags:



Description:

Delegates capabilities from the specified selector range in the source space to the specified selector range in the destination space and thereby optionally reduces the permissions for the destination capabilities.

Capability delegation is defined as follows:

1. **Capabilities** are delegated from source **Capability Selector** range  $\text{src}[\text{ssb}, \text{ssb}+2^{\text{ord}})$ .
2. **Capabilities** are delegated to destination **Capability Selector** range  $\text{dst}[\text{dsb}, \text{dsb}+2^{\text{ord}})$ .
3. The permissions of each destination capability are the logical AND of the permissions of the respective source capability and the permission mask  $\text{pmm}$ , i.e.
  - a) for bits set (1) in  $\text{pmm}$ , the respective permissions are *inherited* from the source capability.
  - b) for bits clear (0) in  $\text{pmm}$ , the respective permissions are *removed* for the destination capability.
4. The destination capability is always a **Null Capability** ( $\text{CAP}_0$ ) in the following cases:
  - a) if the source capability is a **Null Capability** ( $\text{CAP}_0$ ).
  - b) if the destination capability obtains zero permissions from the source capability.
  - c) if the object that the source capability refers to is being destructed concurrently.

Prior to the hypercall:

- $\text{SPC}_{\text{OBJCURRENT}}[\text{src}]$  must refer to a  $\text{CAP}_{\text{OBJ}}$  for the source space with permission TAKE.
- $\text{SPC}_{\text{OBJCURRENT}}[\text{dst}]$  must refer to a  $\text{CAP}_{\text{OBJ}}$  for the destination space with permission GRANT.
- **Capability Selectors**  $\text{ssb}$  and  $\text{dsb}$  must be order-aligned:  $\text{ssb} \equiv 0 \pmod{2^{\text{ord}}}$  and  $\text{dsb} \equiv 0 \pmod{2^{\text{ord}}}$ .
- **Capability Selectors**  $\text{ssb}$  and  $\text{dsb}$  must be equal if  $\text{src}$  refers to a **PIO Space** or an **MSR Space**.

If the hypercall completed successfully:

- If both  $\text{src}$  and  $\text{dst}$  refer to **Object Spaces**:
  - All  $\text{CAP}_{\text{OBJ}}$  (or  $\text{CAP}_0$ ) from the source selector range were delegated to the destination selector range. Any pre-existing  $\text{CAP}_{\text{OBJ}}$  in the destination selector range were revoked.
  - The parameter  $\text{mad}$  was ignored.
- If  $\text{src}$  refers to a **Host Space** and  $\text{dst}$  refers to a **Host Space** or **Guest Space** or **DMA Space**:
  - All  $\text{CAP}_{\text{MEM}}$  (or  $\text{CAP}_0$ ) from the source selector range were delegated to the destination selector range. Any pre-existing  $\text{CAP}_{\text{MEM}}$  in the destination selector range were revoked.
  - If  $\text{src}$  refers to the **NOVA Host Space**, then source  $\text{SEL}_{\text{HST}}$  are **physical** page numbers and the memory attributes of each destination  $\text{CAP}_{\text{MEM}}$  were *set* according to  $\text{mad}$ .

Otherwise, source  $SEL_{HST}$  are **virtual** page numbers and the memory attributes of each destination  $CAP_{MEM}$  were *inherited* from the respective source  $CAP_{MEM}$ , i.e. the parameter mad was ignored.

- If both src and dst refer to **PIO Spaces**:
  - All  $CAP_{PIO}$  (or  $CAP_0$ ) from the source selector range were delegated to the destination selector range. Any pre-existing  $CAP_{PIO}$  in the destination selector range were revoked.
  - The parameter mad was ignored.
- If both src and dst refer to **MSR Spaces**:
  - All  $CAP_{MSR}$  (or  $CAP_0$ ) from the source selector range were delegated to the destination selector range. Any pre-existing  $CAP_{MSR}$  in the destination selector range were revoked.
  - The parameter mad was ignored.
- The resources for storing the granted capabilities were accounted to the **PD** to which the space referred to by  $SPC_{OBJ_{CURRENT}}[dst]$  belongs.

#### Status:

##### SUCCESS

- The hypercall completed successfully.

##### BAD\_CAP

- $SPC_{OBJ_{CURRENT}}[src]$  or  $SPC_{OBJ_{CURRENT}}[dst]$  did not refer to compatible space capabilities or had insufficient permissions.

##### BAD\_PAR

- **Capability Selector** ssb or dsb was not order-aligned.
- **Capability Selector** ssb+2<sup>ord</sup> or dsb+2<sup>ord</sup> was larger than the number of selectors.
- If src refers to a **PIO Space** or an **MSR Space**: **Capability Selectors** ssb and dsb were not equal.
- If src refers to the **NOVA Host Space**: At least one **MAD** field in mad was invalid.

##### MEM\_CAP

- The **PD** to which the space referred to by  $SPC_{OBJ_{CURRENT}}[dst]$  belongs had insufficient memory resources for allocating the storage required for granting all destination capabilities. This constitutes a partial failure of the operation, because all destination capabilities up to the first allocation failure have been granted.

## 5.4.2 Control Execution Context

### Parameters:

```
status = ctrl_ec (SEL_OBJ ec)           // Execution Context
```

### Flags:

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | S |
| 3 | 2 | 1 | 0 |

### Description:

Prior to the hypercall:

- $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{ec}]$  must refer to an **EC Capability** ( $\text{CAP}_{\text{OBJ}_{\text{EC}}}$ ) with permission CTRL.

If the hypercall completed successfully:

- The **EC** referred to by  $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{ec}]$  has been forced to enter the microhypervisor. It will generate a recall exception prior to its next exit from the microhypervisor and will traverse through the respective Event Portal (Arm, x86).
- If **S=0 (Weak Recall)**:
  - The hypercall returns as soon as the recall exception has been *pending*, i.e. the **EC** may not have entered the microhypervisor yet.
- If **S=1 (Strong Recall)**:
  - The hypercall returns as soon as the recall exception has been *observed*, i.e the **EC** will have entered the microhypervisor.

### Status:

#### SUCCESS

- The hypercall completed successfully.

#### BAD\_CAP

- $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{ec}]$  did not refer to an **EC Capability** ( $\text{CAP}_{\text{OBJ}_{\text{EC}}}$ ) or that capability had insufficient permissions.

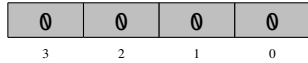


### 5.4.3 Control Scheduling Context

#### Parameters:

```
status = ctrl_sc (SEL_OBJ sc,          // Scheduling Context
                  UINT64 &stc)         // Total Consumed Execution Time
```

#### Flags:



#### Description:

Prior to the hypercall:

- `SPC_OBJ_CURRENT[sc]` must refer to an **SC Capability** (`CAP_OBJ_SC`) with permission CTRL.

If the hypercall completed successfully:

- The microhypervisor has returned the total consumed execution time as **System Time Counter (STC)** value for the **SC** referred to by `SPC_OBJ_CURRENT[sc]`.

#### Status:

##### SUCCESS

- The hypercall completed successfully.

##### BAD\_CAP

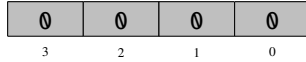
- `SPC_OBJ_CURRENT[sc]` did not refer to an **SC Capability** (`CAP_OBJ_SC`) or that capability had insufficient permissions.

## 5.4.4 Control Portal

### Parameters:

```
status = ctrl_pt (SEL_OBJ pt,           // Portal
                  UINT_PTR pid,         // Portal Identifier
                  MTD mtd)              // Message Transfer Descriptor
```

### Flags:



### Description:

Prior to the hypercall:

- $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{pt}]$  must refer to a **PT Capability** ( $\text{CAP}_{\text{OBJ}_{\text{PT}}}$ ) with permission CTRL.

If the hypercall completed successfully:

- The microhypervisor has set the Portal Identifier (**PID**) to `pid` and the **Message Transfer Descriptor** (**MTD**) to `mtd` for the **Portal** referred to by  $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{pt}]$ .
- Subsequent portal traversals will observe the new **PID** and new **MTD** as follows:

#### On the ambient CPU

- After the hypercall has returned.

#### On all other CPUs

- After receiving a notification from the ambient **CPU**, denoting that the hypercall has returned. Such a notification must be implemented in user-mode with ACQUIRE/RELEASE or stronger memory ordering.

### Status:

#### SUCCESS

- The hypercall completed successfully.

#### BAD\_CAP

- $\text{SPC}_{\text{OBJ}_{\text{CURRENT}}}[\text{pt}]$  did not refer to a **PT Capability** ( $\text{CAP}_{\text{OBJ}_{\text{PT}}}$ ) or that capability had insufficient permissions.

## 5.4.5 Control Semaphore

### Parameters:

```
status = ctrl_sm (SEL_OBJ sm,           // Semaphore
                  UINT64 stc)           // Absolute Timeout
```

### Flags:

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | Z | D |
| 3 | 2 | 1 | 0 |

### Description:

Prior to the hypercall:

- If **D=0** (Semaphore Up):
  - $SPC_{OBJ_{CURRENT}}[sm]$  must refer to an **SM Capability** ( $CAP_{OBJ_{SM}}$ ) with permission  $CTRL_{UP}$ .
- If **D=1** (Semaphore Down):
  - $SPC_{OBJ_{CURRENT}}[sm]$  must refer to an **SM Capability** ( $CAP_{OBJ_{SM}}$ ) with permission  $CTRL_{DN}$ .

If the hypercall completed successfully:

- If **D=0** (Semaphore Up):
  - If there were **ECs** blocked on the semaphore, then the microhypervisor has released one of those blocked **ECs**. Otherwise, the microhypervisor has incremented the semaphore counter. The timeout value and the Z-flag were ignored.
- If **D=1** (Semaphore Down):
  - If the semaphore counter was larger than zero, then the microhypervisor has decremented the semaphore counter (**Z=0**) or set it to zero (**Z=1**). Otherwise, the microhypervisor has blocked  $EC_{CURRENT}$  on the semaphore. If the timeout value was non-zero,  $EC_{CURRENT}$  unblocks with a timeout status when the **System Time Counter** (STC) reaches or exceeds the specified value.

Blocking and releasing of **ECs** on a semaphore uses the FIFO queueing discipline.

### Status:

#### SUCCESS

- The hypercall completed successfully.

#### TIMEOUT

- If **D=1**: Down operation aborted when the timeout triggered.

#### ABORTED

- The **SM** designated by  $SPC_{OBJ_{CURRENT}}[sm]$  is being destructed concurrently.

#### OVRFLOW

- If **D=0**: Up operation aborted because the semaphore counter would overflow.

#### BAD\_CAP

- $SPC_{OBJ_{CURRENT}}[sm]$  did not refer to an **SM Capability** ( $CAP_{OBJ_{SM}}$ ) or that capability had insufficient permissions.

#### BAD\_CPU

- If **D=1** on an interrupt semaphore: Attempt to wait for the interrupt on a different **CPU** than the **CPU** to which that interrupt has been routed via `assign_int`.

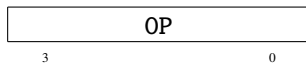
## 5.5 Platform Management

### 5.5.1 Control Hardware

#### Parameters:

```
status = ctrl_hw (UINT desc)           // Descriptor
```

#### Flags:



#### Description:

Modifies the platform hardware configuration or power management state.

Prior to the hypercall:

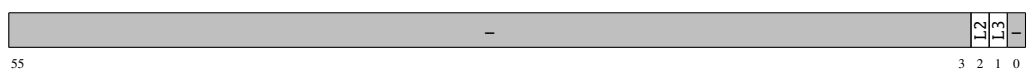
- The hypercall must be invoked by an [Execution Context](#) in the [Root Protection Domain](#) ( $PD_{ROOT}$ ).
- If **OP=0 (S-State Transition)**:
  - The descriptor desc uses the following encoding:



- The value S designates the platform-wide reset or sleep state that shall be entered. The values A and B are the first two bytes of the respective  $\backslash\_Sx$  package in the [ACPI](#) root namespace as follows:

| S   | A                   | B                   | Type    | Description          |
|-----|---------------------|---------------------|---------|----------------------|
| 0x0 | 0x0                 | 0x0                 | Reset   | Platform Reset       |
| 0x1 | $\backslash\_S1[0]$ | $\backslash\_S1[1]$ | Shallow | S1: Stop Grant       |
| 0x2 | $\backslash\_S2[0]$ | $\backslash\_S2[1]$ |         | S2: Power-On Suspend |
| 0x3 | $\backslash\_S3[0]$ | $\backslash\_S3[1]$ |         | S3: Suspend to RAM   |
| 0x4 | $\backslash\_S4[0]$ | $\backslash\_S4[1]$ | Deep    | S4: Suspend to Disk  |
| 0x5 | $\backslash\_S5[0]$ | $\backslash\_S5[1]$ |         | S5: Soft Off         |

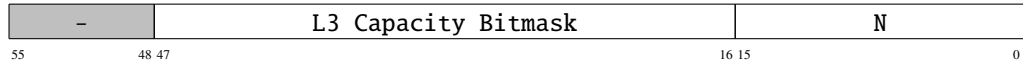
- The caller is responsible for invoking the necessary pre-sleep [ACPI](#) methods, for transitioning platform devices into a suitable Dx sleep state, and for programming wakeup events.
- If **OP=4 (QOS Configuration)**:
  - The descriptor desc uses the following encoding:



- Only Code and Data Prioritization ([CDP](#)) settings supported by the ambient [CPU](#) are valid.
  - \* The L3 bit disables (0) or enables (1)  $CDP_{L3}$  on the ambient [CPU](#).
  - \* The L2 bit disables (0) or enables (1)  $CDP_{L2}$  on the ambient [CPU](#).
- [CAT/CDP](#) or [MBA](#) settings cannot be configured for a [CPU](#) until a valid [QOS](#) configuration has been established for that [CPU](#). Subsequently, that [QOS](#) configuration cannot be changed anymore.

- If **OP=5 (CAT/CDP L3 Capacity Bitmask)**:

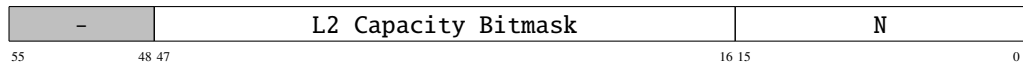
- The descriptor desc uses the following encoding:



- N designates the CPU-local Class Of Service (COS) that shall be configured. Only COS below COS<sub>L3</sub> of the ambient CPU are valid.
  - \* If CDP<sub>L3</sub> is disabled on the ambient CPU:
    - To configure the CAT<sub>L3</sub> Capacity Bitmask for a COS, use N=COS.
  - \* If CDP<sub>L3</sub> is enabled on the ambient CPU:
    - To configure the CDP<sub>L3-Data</sub> Capacity Bitmask for a COS, use N=(COS<<1).
    - To configure the CDP<sub>L3-Code</sub> Capacity Bitmask for a COS, use N=(COS<<1)+1.
- For the L3 Capacity Bitmask, all (and only) contiguous combinations of 1-bits up to the highest capacity bit supported by the ambient CPU are valid.

- If **OP=6 (CAT/CDP L2 Capacity Bitmask)**:

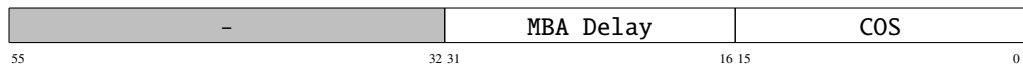
- The descriptor desc uses the following encoding:



- N designates the CPU-local Class Of Service (COS) that shall be configured. Only COS below COS<sub>L2</sub> of the ambient CPU are valid.
  - \* If CDP<sub>L2</sub> is disabled on the ambient CPU:
    - To configure the CAT<sub>L2</sub> Capacity Bitmask for a COS, use N=COS.
  - \* If CDP<sub>L2</sub> is enabled on the ambient CPU:
    - To configure the CDP<sub>L2-Data</sub> Capacity Bitmask for a COS, use N=(COS<<1).
    - To configure the CDP<sub>L2-Code</sub> Capacity Bitmask for a COS, use N=(COS<<1)+1.
- For the L2 Capacity Bitmask, all (and only) contiguous combinations of 1-bits up to the highest capacity bit supported by the ambient CPU are valid.

- If **OP=7 (MBA Delay)**:

- The descriptor desc uses the following encoding:



- COS designates the CPU-local Class Of Service (COS) that shall be configured. Only COS below COS<sub>MB</sub> of the ambient CPU are valid.
- For the MBA Delay, only values up to the highest delay supported by the ambient CPU are valid.

If the hypercall completed successfully:

- If **OP=0 (S-State Transition)**:

- The platform resets or enters the specified ACPI sleep state.
- For reset or deep sleep states, the hypercall does not return.
- For shallow sleep states, the hypercall returns upon a wakeup event. The caller is responsible for invoking the necessary post-sleep ACPI methods and for transitioning platform devices back into the D0 working state.

- If **OP=4 (QOS Configuration)**:

- The ambient CPU uses and locks down the QOS configuration.

- If **OP=5 (CAT/CDP L3 Capacity Bitmask)**:

- The ambient CPU uses the L3 Capacity Bitmask for the designated COS.

- If **OP=6 (CAT/CDP L2 Capacity Bitmask)**:

- The ambient CPU uses the L2 Capacity Bitmask for the designated COS.
- If OP=7 (MBA Delay):
  - The ambient CPU uses the MBA Delay for the designated COS.

**Status:**

**SUCCESS**

- The hypercall completed successfully.

**BAD\_HYP**

- The hypercall was not invoked from the Root Protection Domain (PD<sub>ROOT</sub>).

**BAD\_FTR**

- The requested feature is not supported by the platform.

**BAD\_PAR**

- If OP=4: The QOS configuration is invalid.
- If OP=5/6/7: The COS, CAT/CDP capacity bitmask or MBA delay is invalid.
- Otherwise: The requested operation (OP) is invalid.

**ABORTED**

- If OP=0: A concurrent power management transition prevailed.
- If OP=4: A QOS configuration has already been established for the ambient CPU.
- If OP=5/6/7: A QOS configuration has not yet been established for the ambient CPU.

## 5.5.2 Assign Device

### Parameters:

```
status = assign_dev (SEL_OBJ dc,          // DMA Source Device
                    SEL_OBJ dma)         // DMA Space
```

### Flags:

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 3 | 2 | 1 | 0 |

### Description:

Assigns a DMA-capable device to the specified **DMA Space** ( $SPC_{DMA}$ ).

Prior to the hypercall:

- $SPC_{OBJ_{CURRENT}}[dc]$  must refer to a **DC Capability** ( $CAP_{OBJ_{DC}}$ ) with permission  $ASSIGN_{DEV}$ .
- $SPC_{OBJ_{CURRENT}}[dma]$  must refer to a **DMA Space Capability** ( $CAP_{OBJ_{DMA}}$ ) with permission  $ASSIGN$ .

If the hypercall completed successfully:

- The device designated by  $SPC_{OBJ_{CURRENT}}[dc]$  has been assigned to the **DMA Space** designated by  $SPC_{OBJ_{CURRENT}}[dma]$ .
- DMA transactions of that device will be managed by the **SMMU** and its resources that were encoded in the **Device Context** during its creation. Prior users of those **SMMU** resources have been unconfigured.

### Status:

#### SUCCESS

- The hypercall completed successfully.

#### TIMEOUT

- An **SMMU** command timed out at the hardware level.

#### ABORTED

- A concurrent **assign\_dev** for the designated device prevailed.

#### BAD\_CAP

- $SPC_{OBJ_{CURRENT}}[dc]$  did not refer to a **DC Capability** ( $CAP_{OBJ_{DC}}$ ) or that capability had insufficient permissions.
- $SPC_{OBJ_{CURRENT}}[dma]$  did not refer to a **DMA Space Capability** ( $CAP_{OBJ_{DMA}}$ ) or that capability had insufficient permissions.

#### BAD\_DEV

- The **SMMU** configured in the **Device Context** is invalid.

#### MEM\_OBJ

- A translation table for the **DMA Space** or a mapping table for the **SMMU** could not be allocated.

### 5.5.3 Assign Interrupt

#### Parameters:

```
status = assign_int (SEL_OBJ sm,          // Interrupt Semaphore (GSI)
                    SEL_OBJ dc,          // MSI Source Device
                    UINT_4  cfg,         // Interrupt Configuration
                    UINT_16 cpu,         // Destination CPU Number
                    UINT_16 idx,         // Table Index
                    UINT_PTR &msi_addr,  // OUT: MSI Address
                    UINT_PTR &msi_data) // OUT: MSI Data
```

#### Flags:

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | A |
| 3 | 2 | 1 | 0 |

#### Description:

Attaches or Detaches an Interrupt [Semaphore](#) in a semaphore table slot and configures the associated [GSI](#).

Prior to the hypercall:

- $SPC_{OBJ\_CURRENT}[sm]$  must refer to an [SM Capability](#) ( $CAP_{OBJ\_SM}$ ) for an Interrupt [Semaphore](#) with permission ASSIGN and thereby designates the [GSI](#).
- The destination [CPU](#) must be valid ( $cpu \leq CPU_{MAX}$ ).
- The [Semaphore](#) ([SM](#)) table slot is determined as follows:
  - **Arm**: The [SM](#) table is shared by all [CPUs](#) and indexed by [GSI](#).
  - **x86**: The [SM](#) table is individual for each [CPU](#), designated by `cpu` and indexed by `idx`.
- If **A=0 (Detach)**
  - The [SM](#) table slot must be attached to the Interrupt [Semaphore](#) designated by  $SPC_{OBJ\_CURRENT}[sm]$ .
  - $SPC_{OBJ\_CURRENT}[dc]$  must refer to a [Null Capability](#) ( $CAP_0$ ).
- If **A=1 (Attach)**
  - The [SM](#) table slot must be attached to the Interrupt [Semaphore](#) designated by  $SPC_{OBJ\_CURRENT}[sm]$  or must be detached.
  - If the [GSI](#) type is **PIN**
    - \*  $SPC_{OBJ\_CURRENT}[dc]$  must refer to a [Null Capability](#) ( $CAP_0$ ).
  - If the [GSI](#) type is **MSI**
    - \*  $SPC_{OBJ\_CURRENT}[dc]$  must refer to a [DC Capability](#) ( $CAP_{OBJ\_DC}$ ) with permission  $ASSIGN_{INT}$ .

If the hypercall completed successfully:

- If **A=0 (Detach)**
  - The Interrupt [Semaphore](#) designated by  $SPC_{OBJ\_CURRENT}[sm]$  has been detached from the [SM](#) table slot and will no longer signal interrupt assertions arriving at that [SM](#) table slot.
- If **A=1 (Attach)**
  - The Interrupt [Semaphore](#) designated by  $SPC_{OBJ\_CURRENT}[sm]$  has been attached to the [SM](#) table slot and will signal interrupt assertions arriving at that [SM](#) table slot.
  - The [GSI](#) associated with that Interrupt [Semaphore](#) has been routed to the designated [CPU](#) and [SM](#) table slot and configured according to `cfg`, which uses the following encoding:

|                                                                                                                   |                  |                    |                                  |   |   |   |   |   |             |              |                         |
|-------------------------------------------------------------------------------------------------------------------|------------------|--------------------|----------------------------------|---|---|---|---|---|-------------|--------------|-------------------------|
| <table><tr><td>O</td><td>P</td><td>T</td><td>M</td></tr><tr><td>3</td><td>2</td><td>1</td><td>0</td></tr></table> | O                | P                  | T                                | M | 3 | 2 | 1 | 0 | <b>Mask</b> | 0 = Unmasked | 1 = Masked <sup>†</sup> |
| O                                                                                                                 | P                | T                  | M                                |   |   |   |   |   |             |              |                         |
| 3                                                                                                                 | 2                | 1                  | 0                                |   |   |   |   |   |             |              |                         |
|                                                                                                                   | <b>Trigger</b>   | 0 = Edge-Triggered | 1 = Level-Triggered <sup>†</sup> |   |   |   |   |   |             |              |                         |
|                                                                                                                   | <b>Polarity</b>  | 0 = Active-High    | 1 = Active-Low <sup>†</sup>      |   |   |   |   |   |             |              |                         |
|                                                                                                                   | <b>Ownership</b> | 0 = Host-Owned     | 1 = Guest-Owned <sup>‡</sup>     |   |   |   |   |   |             |              |                         |

<sup>†</sup>Only valid for GSI type PIN

<sup>‡</sup>Only valid for the Arm architecture



- If the **GSI** type is **PIN**
  - \* Any device that is connected to that interrupt pin is able to generate the interrupt.
  - \* The returned `msi_{addr, data}` values are 0 and meaningless.
- If the **GSI** type is **MSI**
  - \* **Arm:** **GITS** translates the device-specific EventID `idx` to the **MSI**.
  - \* Only the device designated by `SPC_OBJ_CURRENT[dc]` is authorized to generate the **MSI**.
  - \* The device driver must program the returned `msi_{addr, data}` values into the **MSI** registers of that device to ensure proper interrupt operation.

Prior to the first invocation of `assign_int` for a **GSI**, the state of that interrupt is as follows:

- The interrupt is masked.
- The configuration (trigger, polarity, ownership) is undefined.
- The source device, destination **CPU** and semaphore table index are undefined.

#### Status:

##### SUCCESS

- The hypercall completed successfully.

##### BAD\_CPU

- The specified CPU number was invalid.

##### BAD\_CAP

- `SPC_OBJ_CURRENT[sm]` did not refer to an **SM Capability** (`CAP_OBJ_SM`) for an Interrupt **Semaphore** or that capability had insufficient permissions.
- If Attach (PIN) or Detach: `SPC_OBJ_CURRENT[dc]` did not refer to a **Null Capability** (`CAP_0`).
- If Attach (MSI): `SPC_OBJ_CURRENT[dc]` did not refer to a **DC Capability** (`CAP_OBJ_DC`) or that capability had insufficient permissions.

##### BAD\_DEV

- The **Device Context (DC)** configuration was invalid.

##### BAD\_PAR

- The semaphore table index was invalid.
- The interrupt configuration was invalid.

##### ABORTED

- A concurrent operation destroyed the designated interrupt semaphore.
- The semaphore table slot was not in the required state.

# 6 Booting

## 6.1 NOVA Microhypervisor

### 6.1.1 NOVA Image

The bootloader must place all loadable (PT\_LOAD) program segments of the [NOVA](#) microhypervisor into physical memory (RAM) according to the physical addresses (`p_paddr`) and memory sizes (`p_memsz`) defined in the [NOVA](#) microhypervisor [ELF](#) executable. The following is an example:

```
readelf -l nova.elf
```

Elf file type is EXEC (Executable file)

**Entry point 0x404000**

There are 2 program headers, starting at offset 64

Program Headers:

| Type | Offset<br>FileSiz                          | VirtAddr<br>MemSiz                          | PhysAddr<br>Flags Align          |
|------|--------------------------------------------|---------------------------------------------|----------------------------------|
| LOAD | 0x00000000000000e8<br>0x0000000000000091c  | 0x00000000000404000<br>0x0000000000000091c  | 0x00000000000404000<br>RW 0x8    |
| LOAD | 0x0000000000000191c<br>0x00000000000014d4c | 0xffffffff80000491c<br>0x000000000000ffb6e4 | 0x0000000000040491c<br>RW 0x1000 |

If the physical memory range defined in the [ELF](#) executable is unsuitable for a particular platform, the bootloader may shift all loadable program segments lower or higher in physical memory, by applying an offset, subject to the following constraints:

- The same offset must be applied to each loadable program segment and to the entry point.
- The offset must comply with the load range and image alignment specified in the Multiboot2 [6] relocatable header tag that is embedded in the [NOVA](#) microhypervisor [ELF](#) executable.
- The entire physical memory region occupied by the [NOVA](#) microhypervisor must be RAM.

After loading the [NOVA](#) microhypervisor into physical memory, the bootloader must invoke the entry point of the [ELF](#) executable with architecture-specific preconditions ([Arm](#), [x86](#)).

### 6.1.2 NOVA Integrity Measurement

The Reference Integrity Measurement ([RIM](#)) of the [NOVA](#) microhypervisor is printed during **make install**.

On hardware platforms with support for establishing a Dynamic Root of Trust for Measurement ([DRTM](#)), the [NOVA](#) microhypervisor initiates a measured launch of itself. The Core Root of Trust for Measurement ([CRTM](#)) extends a Launch Integrity Measurement ([LIM](#)) of the attestable region of the [NOVA](#) microhypervisor into [PCR 17](#) of the Trusted Platform Module ([TPM](#)). The Launch Integrity Measurement is

- sensitive to ([LIM](#) ≠ [RIM](#))
  - any changes in the attestable region
  - the bootloader passing command-line parameters to [NOVA](#)
- insensitive to ([LIM](#) = [RIM](#))
  - the hardware platform and [NOVA](#) patching its code to adapt to available hardware features
  - the physical memory range into which [NOVA](#) has been loaded
  - the memory map and other software modules that have been loaded

## 6.1.3 NOVA Spaces

### 6.1.3.1 NOVA Object Space

The **NOVA Object Space** ( $SPC_{OBJ_{NOVA}}$ ) contains the following **CAP<sub>OBJ</sub>**:

| <b>SEL<sub>OBJ</sub></b> | <b>Capability Type</b>                 | <b>Capability Resource</b>  | <b>Capability Permissions</b>  |
|--------------------------|----------------------------------------|-----------------------------|--------------------------------|
| $2^{SBW_{OBJ}-1}$        | <b>CAP<sub>OBJ<sub>SM</sub></sub></b>  | Console Semaphore           | All <b>defined</b> permissions |
| $2^{SBW_{OBJ}-2}$        | <b>CAP<sub>OBJ<sub>OBJ</sub></sub></b> | NOVA Object Space           | <b>TAKE</b>                    |
| $2^{SBW_{OBJ}-3}$        | <b>CAP<sub>OBJ<sub>HST</sub></sub></b> | NOVA Host Space             | <b>TAKE</b>                    |
| $2^{SBW_{OBJ}-4}$        | <b>CAP<sub>OBJ<sub>PIO</sub></sub></b> | NOVA PIO Space <sup>†</sup> | <b>TAKE</b>                    |
| $2^{SBW_{OBJ}-5}$        | <b>CAP<sub>OBJ<sub>MSR</sub></sub></b> | NOVA MSR Space <sup>†</sup> | <b>TAKE</b>                    |
| $2^{SBW_{OBJ}-6}$        | <b>CAP<sub>OBJ<sub>OBJ</sub></sub></b> | Root Object Space           | All <b>defined</b> permissions |
| $2^{SBW_{OBJ}-7}$        | <b>CAP<sub>OBJ<sub>HST</sub></sub></b> | Root Host Space             | All <b>defined</b> permissions |
| $2^{SBW_{OBJ}-8}$        | <b>CAP<sub>OBJ<sub>PIO</sub></sub></b> | Root PIO Space <sup>†</sup> | All <b>defined</b> permissions |
| $0 \dots CPU_{MAX}$      | <b>CAP<sub>OBJ<sub>SC</sub></sub></b>  | Idle Scheduling Contexts    | <b>CTRL</b>                    |

Using  $SPC_{OBJ_{NOVA}}$  as source space for the **ctrl\_pd** hypercall facilitates the delegation of **CAP<sub>OBJ</sub>** from the **NOVA Object Space** to another **Object Space**.

### 6.1.3.2 NOVA Host Space

In the **NOVA Host Space** ( $SPC_{HST_{NOVA}}$ ), **SEL<sub>HST</sub>** N refers to

- **CAP<sub>0</sub>** for memory protected by the NOVA microhypervisor (**Arm**, **x86**).
- **CAP<sub>MEM</sub>** for the 4 KiB page frame at **physical** address  $N \ll 12$  with all **defined** permissions otherwise.

Using  $SPC_{HST_{NOVA}}$  as source space for the **ctrl\_pd** hypercall facilitates the delegation of **CAP<sub>MEM</sub>** from the **NOVA Host Space** to another **Host Space**, **Guest Space** or **DMA Space**.

### 6.1.3.3 NOVA PIO Space

In the **NOVA PIO Space** ( $SPC_{PIO_{NOVA}}$ ), **SEL<sub>PIO</sub>** N refers to

- **CAP<sub>0</sub>** for an I/O port protected by the NOVA microhypervisor (**x86**).
- **CAP<sub>PIO</sub>** for the I/O port number N with certain permissions otherwise (**x86**).

Using  $SPC_{PIO_{NOVA}}$  as source space for the **ctrl\_pd** hypercall facilitates the delegation of **CAP<sub>PIO</sub>** from the **NOVA PIO Space** to another **PIO Space**.

### 6.1.3.4 NOVA MSR Space

In the **NOVA MSR Space** ( $SPC_{MSR_{NOVA}}$ ), **SEL<sub>MSR</sub>** N refers to

- **CAP<sub>0</sub>** for an **MSR** protected by the NOVA microhypervisor (**x86**).
- **CAP<sub>MSR</sub>** for the **MSR** number N with certain permissions otherwise (**x86**).

Using  $SPC_{MSR_{NOVA}}$  as source space for the **ctrl\_pd** hypercall facilitates the delegation of **CAP<sub>MSR</sub>** from the **NOVA MSR Space** to another **MSR Space**.

<sup>†</sup>Only on architectures, where this type of space **exists**, otherwise **Null Capability** (**CAP<sub>0</sub>**).

## 6.2 Root Protection Domain

After the [NOVA](#) microhypervisor has initialized the system, it creates the following initial kernel objects:

- [Root Protection Domain](#) ( $PD_{ROOT}$ )
  - with [Root Object Space](#) ( $SPC_{OBJ_{ROOT}}$ )
  - with [Root Host Space](#) ( $SPC_{HST_{ROOT}}$ )
  - with [Root PIO Space](#) ( $SPC_{PIO_{ROOT}}$ )<sup>†</sup>
- [Root Execution Context](#) ( $EC_{ROOT}$ ) on  $CPU_{BSP}$ 
  - bound to  $SPC_{OBJ_{ROOT}}$ ,  $SPC_{HST_{ROOT}}$ ,  $SPC_{PIO_{ROOT}}$ <sup>†</sup>
  - with  $SEL_{EVT} = 0$
- [Root Scheduling Context](#) ( $SC_{ROOT}$ ) on  $CPU_{BSP}$ 
  - bound to  $EC_{ROOT}$
  - with  $COS = 0$
  - with Priority = highest priority
  - with Budget = 1000 ms

The [Root Protection Domain](#) is responsible for bootstrapping the other components of the user-mode framework by creating additional kernel objects, loading additional images, assigning resources, etc.

### 6.2.1 Root Image

The image of the [Root Protection Domain](#) ( $PD_{ROOT}$ ) must be a valid [ELF](#) executable (ET\_EXEC) that has been compiled for the respective hardware architecture and

- linked such that  $p\_filesz = p\_memsz$
- loaded such that  $p\_vaddr \equiv LOAD\_ADDR^* + p\_offset \pmod{PAGE\_SIZE}$

holds for each loadable (PT\_LOAD) program segment. These constraints ensure that the [NOVA](#) microhypervisor can map all program segments directly from physical into virtual memory without any additional memory allocation or copying. The following is an example:

```
readelf -l root.elf
```

Elf file type is EXEC (Executable file)

Entry point 0x10000120

There are 2 program headers, starting at offset 64

Program Headers:

| Type | Offset             | VirtAddr           | PhysAddr           |
|------|--------------------|--------------------|--------------------|
|      | FileSiz            | MemSiz             | Flags Align        |
| LOAD | 0x0000000000000000 | 0x0000000010000000 | 0x0000000010000000 |
|      | 0x0000000000000a75 | 0x0000000000000a75 | R E 0x1000         |
| LOAD | 0x0000000000000100 | 0x0000000010001000 | 0x0000000010001000 |
|      | 0x000000000000f004 | 0x000000000000f004 | RW 0x1000          |

<sup>†</sup>Only on architectures, where this type of space [exists](#).

\*The address in physical memory at which the bootloader has placed the ELF image.

## 6.2.2 Root Integrity Measurement

On hardware platforms with support for establishing a Dynamic Root of Trust for Measurement (DRTM), the NOVA microhypervisor extends a Launch Integrity Measurement (LIM) of the attestable region of the Root Protection Domain into PCR 19 of the Trusted Platform Module (TPM).

The attestable region is defined as the entire contents of the first loadable (PT\_LOAD) program segment of the PD<sub>ROOT</sub> ELF image that is readable and/or executable, but not writable.

The attestable region of the Root Protection Domain should cover:

- the ELF header – to include the entry point of PD<sub>ROOT</sub> in the measurement
- the ELF program headers – to include the virtual-memory layout of PD<sub>ROOT</sub> in the measurement
- the code and read-only data sections of PD<sub>ROOT</sub> – to reflect their integrity

## 6.2.3 Root Spaces

Prior to the Root Execution Context (EC<sub>ROOT</sub>) invoking the entry point of the Root Protection Domain (PD<sub>ROOT</sub>) ELF image, the NOVA microhypervisor sets up the spaces for PD<sub>ROOT</sub> as described in the following subsections.

### 6.2.3.1 Root Object Space

The Root Object Space (SPC<sub>OBJ<sub>ROOT</sub></sub>) contains the following initial CAP<sub>OBJ</sub>:

| SEL <sub>OBJ</sub>               | Capability Type                  | Capability Resource     | Capability Permissions  |
|----------------------------------|----------------------------------|-------------------------|-------------------------|
| 2 <sup>SBW<sub>OBJ</sub>-1</sup> | CAP <sub>OBJ<sub>OBJ</sub></sub> | NOVA Object Space       | TAKE                    |
| 2 <sup>SBW<sub>OBJ</sub>-2</sup> | CAP <sub>OBJ<sub>OBJ</sub></sub> | Root Object Space       | All defined permissions |
| 2 <sup>SBW<sub>OBJ</sub>-3</sup> | CAP <sub>OBJ<sub>PD</sub></sub>  | Root Protection Domain  | All defined permissions |
| 2 <sup>SBW<sub>OBJ</sub>-4</sup> | CAP <sub>OBJ<sub>EC</sub></sub>  | Root Execution Context  | All defined permissions |
| 2 <sup>SBW<sub>OBJ</sub>-5</sup> | CAP <sub>OBJ<sub>SC</sub></sub>  | Root Scheduling Context | All defined permissions |

All other SEL<sub>OBJ</sub> in SPC<sub>OBJ<sub>ROOT</sub></sub> initially refer to a Null Capability (CAP<sub>0</sub>).

### 6.2.3.2 Root Host Space

#### ELF Program Segments

The microhypervisor maps the Root Protection Domain (PD<sub>ROOT</sub>) into the Root Host Space according to the virtual addresses (p\_vaddr), memory sizes (p\_memsz) and page attributes (p\_flags) of all loadable (PT\_LOAD) program segments defined in the PD<sub>ROOT</sub> ELF image.

#### Hypervisor Information Page

The microhypervisor maps the Hypervisor Information Page read-only into the Root Host Space 4 KiB below the end of user-accessible virtual memory. The virtual address of the HIP is passed to EC<sub>ROOT</sub> at the entry point (Arm, x86).

#### UTCB

The microhypervisor maps the User Thread Control Block of EC<sub>ROOT</sub> into the Root Host Space 4 KiB below the address of the Hypervisor Information Page.

All other SEL<sub>HST</sub> in SPC<sub>HST<sub>ROOT</sub></sub> initially refer to a Null Capability (CAP<sub>0</sub>).

### 6.2.3.3 Root PIO Space

All SEL<sub>PIO</sub> in SPC<sub>PIO<sub>ROOT</sub></sub> initially refer to a Null Capability (CAP<sub>0</sub>).

## 6.3 Hypervisor Information Page

The [Hypervisor Information Page \(HIP\)](#) is mapped in the [Host Space](#) of the [Root Protection Domain](#) and the initial Stack Pointer (SP) of the [Root Execution Context](#) points to the [HIP](#). Because the [HIP](#) is allocated and owned by the microhypervisor, it cannot be delegated using `ctrl_pd`.

The [HIP](#) conveys information about the platform and configuration and has the following layout:

|                          |       |       |       |       |       |       |     |   |         |
|--------------------------|-------|-------|-------|-------|-------|-------|-----|---|---------|
| 63                       | 56 55 | 48 47 | 40 39 | 32 31 | 24 23 | 16 15 | 8 7 | 0 | +Length |
| Architecture-Dependent   |       |       |       |       |       |       |     |   |         |
| Architecture-Dependent   |       |       |       |       |       |       |     |   |         |
| Platform Features        |       |       |       |       |       |       |     |   | +0xa0   |
| SEL <sub>GST/NOVA</sub>  |       |       |       |       |       |       |     |   | +0x98   |
| SEL <sub>GST/ARCH</sub>  |       |       |       |       |       |       |     |   | +0x90   |
| KID <sub>MAX</sub>       |       |       |       |       |       |       |     |   | +0x88   |
| CPU <sub>MAX</sub>       |       |       |       |       |       |       |     |   | +0x80   |
| CPU <sub>BSP</sub>       |       |       |       |       |       |       |     |   | +0x78   |
| MCO <sub>MSR</sub>       |       |       |       |       |       |       |     |   | +0x70   |
| MCO <sub>PIO</sub>       |       |       |       |       |       |       |     |   | +0x68   |
| MCO <sub>DMA</sub>       |       |       |       |       |       |       |     |   | +0x60   |
| MCO <sub>GST</sub>       |       |       |       |       |       |       |     |   | +0x58   |
| MCO <sub>HST</sub>       |       |       |       |       |       |       |     |   | +0x50   |
| MCO <sub>OBJ</sub>       |       |       |       |       |       |       |     |   | +0x48   |
| SBW <sub>MSR</sub>       |       |       |       |       |       |       |     |   | +0x40   |
| SBW <sub>PIO</sub>       |       |       |       |       |       |       |     |   | +0x38   |
| SBW <sub>DMA</sub>       |       |       |       |       |       |       |     |   | +0x30   |
| SBW <sub>GST</sub>       |       |       |       |       |       |       |     |   | +0x28   |
| SBW <sub>HST</sub>       |       |       |       |       |       |       |     |   | +0x20   |
| SBW <sub>OBJ</sub>       |       |       |       |       |       |       |     |   | +0x18   |
| STC Frequency            |       |       |       |       |       |       |     |   | +0x10   |
| UEFI Desc Version        |       |       |       |       |       |       |     |   | +0x08   |
| UEFI Desc Size           |       |       |       |       |       |       |     |   | +0x00   |
| UEFI Memory Map Size     |       |       |       |       |       |       |     |   |         |
| UEFI Memory Map Address  |       |       |       |       |       |       |     |   |         |
| Framebuffer Resolution Y |       |       |       |       |       |       |     |   |         |
| Framebuffer Resolution X |       |       |       |       |       |       |     |   |         |
| Framebuffer Pixel Pitch  |       |       |       |       |       |       |     |   |         |
| Framebuffer Pixel Format |       |       |       |       |       |       |     |   |         |
| Framebuffer Size         |       |       |       |       |       |       |     |   |         |
| Framebuffer Address      |       |       |       |       |       |       |     |   |         |
| ACPI RSDP Address        |       |       |       |       |       |       |     |   |         |
| ROOT End Address         |       |       |       |       |       |       |     |   |         |
| ROOT Start Address       |       |       |       |       |       |       |     |   |         |
| MBUF End Address         |       |       |       |       |       |       |     |   |         |
| MBUF Start Address       |       |       |       |       |       |       |     |   |         |
| NOVA End Address         |       |       |       |       |       |       |     |   |         |
| NOVA Start Address       |       |       |       |       |       |       |     |   |         |
| Length                   |       |       |       |       |       |       |     |   |         |
| Checksum                 |       |       |       |       |       |       |     |   |         |
| Signature                |       |       |       |       |       |       |     |   |         |

All HIP fields are unsigned little-endian values, unless stated otherwise, and have the following meaning:

### Signature

The value `0x41564f4e` identifies the [NOVA](#) microhypervisor.

### Checksum

The checksum is valid if 16bit-wise addition of the entire [HIP](#) contents produces a value of `0`.

### Length

Length of the entire [HIP](#) in bytes.

### NOVA Start/End Address

Physical start and end address of the [NOVA](#) microhypervisor image.

### MBUF Start/End Address

Physical start and end address of the memory buffer console region (see [C.1](#)).

### ROOT Start/End Address

Physical start and end address of the root protection domain image.

**ACPI RSDP Address**

Physical address of the [ACPI \[7\]](#) Root System Description Pointer (0xfffffffffffffff if not present).

**Framebuffer Address**

Framebuffer physical address.

**Framebuffer Size**

Framebuffer size in bytes (0x0 if not present).

**Framebuffer Pixel Format**

Pixel format, using the following encoding (bits not contained in any color bitmask are reserved):

**R<sub>L</sub>, R<sub>H</sub>**

Lowest and highest bit number of the bitmask representing color intensity for **red**.

**G<sub>L</sub>, G<sub>H</sub>**

Lowest and highest bit number of the bitmask representing color intensity for **green**.

**B<sub>L</sub>, B<sub>H</sub>**

Lowest and highest bit number of the bitmask representing color intensity for **blue**.

**N-1**

Number of bytes per pixel minus one: 0=1B (8-bit), 1=2B (16-bit), 2=3B (24-bit), 3=4B (32-bit)

| N-1      | B <sub>H</sub> | B <sub>L</sub> | G <sub>H</sub> | G <sub>L</sub> | R <sub>H</sub> | R <sub>L</sub> |
|----------|----------------|----------------|----------------|----------------|----------------|----------------|
| 31 30 29 | 25 24          | 20 19          | 15 14          | 10 9           | 5 4            | 0              |

**Framebuffer Pixel Pitch**

Number of pixels per horizontal scanline.

**Framebuffer Resolution X**

Number of pixels in horizontal direction.

**Framebuffer Resolution Y**

Number of pixels in vertical direction.

**UEFI Memory Map Address**

Physical address of the [UEFI \[8\]](#) Memory Map (0xfffffffffffffff if not present).

**UEFI Memory Map Size**

Total size of the [UEFI](#) Memory Map (0 if not present).

**UEFI Desc Size**

[UEFI](#) Memory Descriptor Size (0 if not present).

**UEFI Desc Version**

[UEFI](#) Memory Descriptor Version (0 if not present).

**STC Frequency**

Frequency of the [System Time Counter \(STC\)](#) in Hz.

**SBW<sub>OBJ</sub>**

Selector bit width: [Object Capability Selector \(SEL<sub>OBJ</sub>\)](#)

**SBW<sub>HST</sub>**

Selector bit width: [Host Capability Selector \(SEL<sub>HST</sub>\)](#)

**SBW<sub>GST</sub>**

Selector bit width: [Guest Capability Selector \(SEL<sub>GST</sub>\)](#)

**SBW<sub>DMA</sub>**

Selector bit width: [DMA Capability Selector \(SEL<sub>DMA</sub>\)](#)

**SBW<sub>PIO</sub>**

Selector bit width: [PIO Capability Selector \(SEL<sub>PIO</sub>\)](#)

**SBW<sub>MSR</sub>**

Selector bit width: [MSR Capability Selector \(SEL<sub>MSR</sub>\)](#)

**MCO<sub>OBJ</sub>**

Maximum contiguous order that avoids partial failures during capability updates in [SPC<sub>OBJ</sub>](#).

**MCO<sub>HST</sub>**

Maximum contiguous order that avoids partial failures during capability updates in [SPC<sub>HST</sub>](#).

**MCO<sub>GST</sub>**

Maximum contiguous order that avoids partial failures during capability updates in [SPC<sub>GST</sub>](#).

**MCO<sub>DMA</sub>**

Maximum contiguous order that avoids partial failures during capability updates in [SPC<sub>DMA</sub>](#).

**MCO<sub>PIO</sub>**

Maximum contiguous order that avoids partial failures during capability updates in [SPC<sub>PIO</sub>](#).

**MCO<sub>MSR</sub>**

Maximum contiguous order that avoids partial failures during capability updates in [SPC<sub>MSR</sub>](#).

**CPU<sub>BSP</sub>**

Bootstrap Processor. The [CPU](#) number on which [EC<sub>ROOT</sub>](#) and [SC<sub>ROOT</sub>](#) have been created.

**CPU<sub>MAX</sub>**

Maximum [CPU](#) number. The total number of [CPU](#)s is  $\text{CPU}_{\text{MAX}} + 1$ .

**KID<sub>MAX</sub>**

Maximum [KID](#) number. The total number of [KID](#)s is  $\text{KID}_{\text{MAX}} + 1$ .

**SEL<sub>HST/ARCH</sub>**

Number of [Capability Selectors](#) required for handling architectural host events. ([Arm, x86](#))

**SEL<sub>HST/NOVA</sub>**

Number of additional [Capability Selectors](#) required for handling microhypervisor host events. ([Arm, x86](#))

**SEL<sub>GST/ARCH</sub>**

Number of [Capability Selectors](#) required for handling architectural guest events. ([Arm, x86](#))

**SEL<sub>GST/NOVA</sub>**

Number of additional [Capability Selectors](#) required for handling microhypervisor guest events. ([Arm, x86](#))

**Platform Features**

Supported platform features. ([Arm, x86](#))

**Architecture-Dependent**

Architecture-dependent part. ([Arm, x86](#))



## **Part IV**

# **Application Binary Interface**

# 7 ABI aarch64

## 7.1 Boot State

### 7.1.1 NOVA Microhypervisor

The bootloader must set up the [CPU](#) register state according to one of the launch types listed below when it transfers control to the [NOVA](#) microhypervisor entry point. Furthermore, the following preconditions must be satisfied:

- The [CPU](#) must execute in EL2 (hypervisor mode) or in EL3 (monitor mode).
- Paging ([MMU](#)) must either be disabled (`SCTLR_ELx.M=0`) or the page tables must provide an identity (1:1) mapping with read/write/execute permissions.
- Interrupts must be disabled (`PSTATE.DAIF=0b1111`).
- The physical memory region occupied by the microhypervisor image must be clean to the PoC.
- All [DMA](#) activity targeting the physical memory region occupied by the microhypervisor must be quiesced. That physical memory region should also be protected against [DMA](#) accesses on systems with an [SMMU](#).

#### 7.1.1.1 Multiboot v2 Launch

Only this launch type supports 64-bit [UEFI](#) platforms.

| Register | Value / Description                                                                |
|----------|------------------------------------------------------------------------------------|
| IP       | Physical address of the <a href="#">NOVA ELF</a> image <a href="#">entry point</a> |
| X0       | Multiboot v2 magic value ( <code>0x36d76289</code> ) [6]                           |
| X1       | Physical address of the Multiboot v2 information structure [6]                     |
| Other    | ~                                                                                  |

The [NOVA](#) microhypervisor consumes the following multiboot tags, if present: 1, 3, 12, 15, 20.

#### 7.1.1.2 Multiboot v1 Launch

| Register | Value / Description                                                                |
|----------|------------------------------------------------------------------------------------|
| IP       | Physical address of the <a href="#">NOVA ELF</a> image <a href="#">entry point</a> |
| X0       | Multiboot v1 magic value ( <code>0x2badb002</code> ) [9]                           |
| X1       | Physical address of the Multiboot v1 information structure [9]                     |
| Other    | ~                                                                                  |

The [NOVA](#) microhypervisor consumes the following multiboot flags, if present: 2, 3.

#### 7.1.1.3 Legacy Launch

| Register | Value / Description                                                                                                         |
|----------|-----------------------------------------------------------------------------------------------------------------------------|
| IP       | Physical address of the <a href="#">NOVA ELF</a> image <a href="#">entry point</a>                                          |
| X0       | Physical address of the Flattened Device Tree ( <a href="#">FDT</a> ) for the hardware platform <sup>†</sup>                |
| X1       | Physical address of the <a href="#">Root Protection Domain</a> ( <code>PD<sub>ROOT</sub></code> ) <a href="#">ELF</a> image |
| Other    | ~                                                                                                                           |

<sup>†</sup> Due to its alignment constraint, a valid FDT address will never be equal to a Multiboot magic value.

### 7.1.2 Root Protection Domain

The [NOVA](#) microhypervisor sets up the [CPU](#) register state as follows when it transfers control to the [Root Execution Context \(EC<sub>ROOT</sub>\)](#):

| Register | Value / Description                                                                                         |
|----------|-------------------------------------------------------------------------------------------------------------|
| IP       | HVA of the <a href="#">Root Protection Domain (PD<sub>ROOT</sub>)</a> <a href="#">ELF</a> image entry point |
| SP       | HVA of the <a href="#">Hypervisor Information Page (HIP)</a>                                                |
| X0       | X0 at <a href="#">boot time</a> <sup>†</sup>                                                                |
| X1       | X1 at <a href="#">boot time</a> <sup>†</sup>                                                                |
| X2       | X2 at <a href="#">boot time</a> <sup>†</sup>                                                                |
| Other    | ~                                                                                                           |

## 7.2 Memory Map

The [Root Protection Domain \(PD<sub>ROOT</sub>\)](#) can obtain a list of available/reserved physical memory regions as follows:

- On platforms using Unified Extensible Firmware Interface, by parsing the [UEFI memory map](#).
- On platforms using Flattened Device Tree, by parsing the [FDT](#).

## 7.3 Class Of Service

Class Of Service ([COS](#)) is currently not supported.

## 7.4 Protected Resources

Certain resources protected by the [NOVA](#) microhypervisor cannot be delegated and therefore remain inaccessible to user-mode components. The following subsections enumerate these protected resources.

### 7.4.1 Physical Memory

The following physical memory regions are protected:

- [NOVA](#) microhypervisor – conveyed via [HIP](#).
- [GITS](#), [GICD](#), [GICR](#), [GICC](#), [GICH](#) devices [[10](#), [11](#)] – conveyed via [ACPI](#) MADT or via [FDT](#).
- [SMMU](#) devices [[12](#), [13](#)] – conveyed via [ACPI](#) IORT or via [FDT](#).
- Firmware runtime services – conveyed via [UEFI](#) memory map.

---

<sup>†</sup>The register contains the preserved original value from the point when control was transferred from the bootloader to the microhypervisor.

## 7.5 Event-Specific Capability Selectors

For the delivery of exception/intercept messages, the microhypervisor performs an implicit portal traversal.

The selector for the destination portal (**SEL<sub>OBJ</sub>**):

- is determined by adding the exception/intercept number to the affected **Execution Context**'s Event Selector Base (**SEL<sub>EVT</sub>**).
- indexes into the **Object Space** (**SPC<sub>OBJ</sub>**) of the affected **EC**'s **Protection Domain** (**PD**).
- must refer to a **PT Capability** (**CAP<sub>OBJPT</sub>**) with permission **EVENT** that is bound to an **EC** on the same **CPU** as the affected **EC**, otherwise the affected **EC** is killed.

### 7.5.1 Architectural Events

#### Host Exceptions and Guest Intercepts

| <b>SEL<sub>OBJ</sub></b>        | <b>Exception / Intercept</b> | <b>SEL<sub>OBJ</sub></b>        | <b>Exception / Intercept</b> |
|---------------------------------|------------------------------|---------------------------------|------------------------------|
| <b>SEL<sub>EVT</sub></b> + 0x00 | Unknown Reason               | <b>SEL<sub>EVT</sub></b> + 0x20 | Instruction Abort (lower EL) |
| <b>SEL<sub>EVT</sub></b> + 0x01 | Trapped WFI or WFE           | <b>SEL<sub>EVT</sub></b> + 0x21 | Instruction Abort (same EL)* |
| <b>SEL<sub>EVT</sub></b> + 0x02 | reserved                     | <b>SEL<sub>EVT</sub></b> + 0x22 | PC Alignment Fault           |
| <b>SEL<sub>EVT</sub></b> + 0x03 | Trapped MCR or MRC           | <b>SEL<sub>EVT</sub></b> + 0x23 | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x04 | Trapped MCRR or MRRC         | <b>SEL<sub>EVT</sub></b> + 0x24 | Data Abort (lower EL)        |
| <b>SEL<sub>EVT</sub></b> + 0x05 | Trapped MCR or MRC           | <b>SEL<sub>EVT</sub></b> + 0x25 | Data Abort (same EL)*        |
| <b>SEL<sub>EVT</sub></b> + 0x06 | Trapped LDC or STC           | <b>SEL<sub>EVT</sub></b> + 0x26 | SP Alignment Fault           |
| <b>SEL<sub>EVT</sub></b> + 0x07 | SME, SVE, SIMD, FPU          | <b>SEL<sub>EVT</sub></b> + 0x27 | Memory Operation Exception   |
| <b>SEL<sub>EVT</sub></b> + 0x08 | Trapped VMRS Access          | <b>SEL<sub>EVT</sub></b> + 0x28 | Trapped FPU (AArch32)        |
| <b>SEL<sub>EVT</sub></b> + 0x09 | Trapped PAuth Instruction    | <b>SEL<sub>EVT</sub></b> + 0x29 | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x0a | Trapped LD64B or ST64B       | <b>SEL<sub>EVT</sub></b> + 0x2a | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x0b | reserved                     | <b>SEL<sub>EVT</sub></b> + 0x2b | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x0c | Trapped MRRC                 | <b>SEL<sub>EVT</sub></b> + 0x2c | Trapped FPU (AArch64)        |
| <b>SEL<sub>EVT</sub></b> + 0x0d | Branch Target Exception      | <b>SEL<sub>EVT</sub></b> + 0x2d | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x0e | Illegal Execution State      | <b>SEL<sub>EVT</sub></b> + 0x2e | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x0f | reserved                     | <b>SEL<sub>EVT</sub></b> + 0x2f | Error                        |
| <b>SEL<sub>EVT</sub></b> + 0x10 | reserved                     | <b>SEL<sub>EVT</sub></b> + 0x30 | Breakpoint (lower EL)        |
| <b>SEL<sub>EVT</sub></b> + 0x11 | SVC (from AArch32 State)     | <b>SEL<sub>EVT</sub></b> + 0x31 | Breakpoint (same EL)*        |
| <b>SEL<sub>EVT</sub></b> + 0x12 | HVC (from AArch32 State)     | <b>SEL<sub>EVT</sub></b> + 0x32 | Software Step (lower EL)     |
| <b>SEL<sub>EVT</sub></b> + 0x13 | SMC (from AArch32 State)     | <b>SEL<sub>EVT</sub></b> + 0x33 | Software Step (same EL)*     |
| <b>SEL<sub>EVT</sub></b> + 0x14 | reserved                     | <b>SEL<sub>EVT</sub></b> + 0x34 | Watchpoint (lower EL)        |
| <b>SEL<sub>EVT</sub></b> + 0x15 | SVC (from AArch64 State)*    | <b>SEL<sub>EVT</sub></b> + 0x35 | Watchpoint (same EL)*        |
| <b>SEL<sub>EVT</sub></b> + 0x16 | HVC (from AArch64 State)     | <b>SEL<sub>EVT</sub></b> + 0x36 | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x17 | SMC (from AArch64 State)     | <b>SEL<sub>EVT</sub></b> + 0x37 | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x18 | Trapped MSR or MRS           | <b>SEL<sub>EVT</sub></b> + 0x38 | BKPT (AArch32)               |
| <b>SEL<sub>EVT</sub></b> + 0x19 | Trapped SVE                  | <b>SEL<sub>EVT</sub></b> + 0x39 | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x1a | Trapped ERET                 | <b>SEL<sub>EVT</sub></b> + 0x3a | Vector Catch (AArch32)       |
| <b>SEL<sub>EVT</sub></b> + 0x1b | TSTART Exception             | <b>SEL<sub>EVT</sub></b> + 0x3b | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x1c | PAuth Instruction Failure    | <b>SEL<sub>EVT</sub></b> + 0x3c | BRK (AArch64)                |
| <b>SEL<sub>EVT</sub></b> + 0x1d | Trapped SME                  | <b>SEL<sub>EVT</sub></b> + 0x3d | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x1e | Granule Protection Exception | <b>SEL<sub>EVT</sub></b> + 0x3e | reserved                     |
| <b>SEL<sub>EVT</sub></b> + 0x1f | reserved                     | <b>SEL<sub>EVT</sub></b> + 0x3f | reserved                     |

Please refer to [3] for more details on each of these events.

\*These events may be handled by the microhypervisor, in which case they will not cause portal traversals.

## 7.5.2 Microhypervisor Events

| <b>SEL<sub>OBJ</sub></b>                             | <b>Event</b>  |
|------------------------------------------------------|---------------|
| <b>SEL<sub>EVT</sub></b> + SEL <sub>ARCH</sub> + 0x0 | Startup       |
| <b>SEL<sub>EVT</sub></b> + SEL <sub>ARCH</sub> + 0x1 | Recall        |
| <b>SEL<sub>EVT</sub></b> + SEL <sub>ARCH</sub> + 0x2 | Virtual Timer |

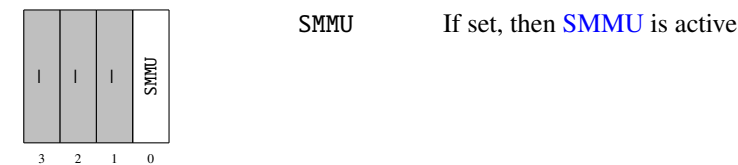
The value of SEL<sub>ARCH</sub> depends on the origin of the event:

- SEL<sub>ARCH</sub> = SEL<sub>HST/ARCH</sub> for events that occurred in the host.
- SEL<sub>ARCH</sub> = SEL<sub>GST/ARCH</sub> for events that occurred in the guest.

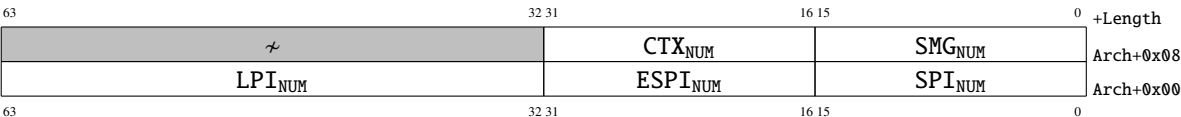
## 7.6 Architecture-Dependent Structures

### 7.6.1 Platform Features

The supported platform features are defined as follows:



### 7.6.2 Hypervisor Information Page



- SPI<sub>NUM</sub>

Total number of Shared Peripheral Interrupts (SPIs).
- ESPI<sub>NUM</sub>

Total number of Extended Shared Peripheral Interrupts (ESPIs).
- LPI<sub>NUM</sub>

Total number of Locality-Specific Peripheral Interrupts (LPIs).
- SMG<sub>NUM</sub>

Total number of SMMUv2 Stream Mapping Groups (SMGs).
- CTX<sub>NUM</sub>

Total number of SMMUv2 Translation Contexts (CTXs).

### 7.6.3 User Thread Control Block

|               |          |                |          |        |  |
|---------------|----------|----------------|----------|--------|--|
| -             |          | SEL_GST        |          | +0x2e0 |  |
| -             |          | VMCR           | ELRSR    | +0x2d0 |  |
| AP1R3         | AP1R2    | AP1R1          | AP1R0    | +0x2c0 |  |
| AP0R3         | AP0R2    | AP0R1          | AP0R0    | +0x2b0 |  |
| LR15          |          | LR14           |          | +0x2a0 |  |
| LR13          |          | LR12           |          | +0x290 |  |
| LR11          |          | LR10           |          | +0x280 |  |
| LR9           |          | LR8            |          | +0x270 |  |
| LR7           |          | LR6            |          | +0x260 |  |
| LR5           |          | LR4            |          | +0x250 |  |
| LR3           |          | LR2            |          | +0x240 |  |
| LR1           |          | LR0            |          | +0x230 |  |
| CNTVOFF_EL2   |          | CNTKCTL_EL1    |          | +0x220 |  |
| CNTV_CTL_EL0  |          | CNTV_CVAL_EL0  |          | +0x210 |  |
| -             |          | HPFAR_EL2      |          | +0x200 |  |
| FAR_EL2       |          | ESR_EL2        |          | +0x1f0 |  |
| SPSR_EL2      |          | ELR_EL2        |          | +0x1e0 |  |
| VMPIDR_EL2    |          | VPIDR_EL2      |          | +0x1d0 |  |
| HCRX_EL2      |          | HCR_EL2        |          | +0x1c0 |  |
| -             |          | MDSCR_EL1      |          | +0x1b0 |  |
| SCTLR_EL1     |          | VBAR_EL1       |          | +0x1a0 |  |
| AMAIR_EL1     |          | MAIR_EL1       |          | +0x190 |  |
| TCR_EL1       |          | TTBR1_EL1      |          | +0x180 |  |
| TTBR0_EL1     |          | AFSR1_EL1      |          | +0x170 |  |
| AFSR0_EL1     |          | FAR_EL1        |          | +0x160 |  |
| ESR_EL1       |          | SPSR_EL1       |          | +0x150 |  |
| ELR_EL1       |          | CONTEXTIDR_EL1 |          | +0x140 |  |
| TPIDR_EL1     |          | SP_EL1         |          | +0x130 |  |
| -             | HSTR     | IFSR           | DACR     | +0x120 |  |
| SPSR_und      | SPSR_irq | SPSR_fiq       | SPSR_abt | +0x110 |  |
| TPIDRRO_EL0   |          | TPIDR_EL0      |          | +0x100 |  |
| SP_EL0        |          | X30 (LR_fiq)   |          | +0x0f0 |  |
| X29 (SP_fiq)  |          | X28 (R12_fiq)  |          | +0x0e0 |  |
| X27 (R11_fiq) |          | X26 (R10_fiq)  |          | +0x0d0 |  |
| X25 (R9_fiq)  |          | X24 (R8_fiq)   |          | +0x0c0 |  |
| X23 (SP_und)  |          | X22 (LR_und)   |          | +0x0b0 |  |
| X21 (SP_abt)  |          | X20 (LR_abt)   |          | +0x0a0 |  |
| X19 (SP_svc)  |          | X18 (LR_svc)   |          | +0x090 |  |
| X17 (SP_irq)  |          | X16 (LR_irq)   |          | +0x080 |  |
| X15 (SP_hyp)  |          | X14 (LR_usr)   |          | +0x070 |  |
| X13 (SP_usr)  |          | X12 (R12_usr)  |          | +0x060 |  |
| X11 (R11_usr) |          | X10 (R10_usr)  |          | +0x050 |  |
| X9 (R9_usr)   |          | X8 (R8_usr)    |          | +0x040 |  |
| X7 (R7)       |          | X6 (R6)        |          | +0x030 |  |
| X5 (R5)       |          | X4 (R4)        |          | +0x020 |  |
| X3 (R3)       |          | X2 (R2)        |          | +0x010 |  |
| X1 (R1)       |          | X0 (R0)        |          | +0x000 |  |

## 7.6.4 Message Transfer Descriptor

The [Message Transfer Descriptor \(MTD\)](#), which controls the subset of the architectural state transferred during exceptions and intercepts, as described in Section 4.4.2, has the following layout:

|        |     |     |   |           |             |              |         |         |   |           |           |          |          |         |          |          |             |              |         |        |   |         |          |   |         |        |     |     |     |        |
|--------|-----|-----|---|-----------|-------------|--------------|---------|---------|---|-----------|-----------|----------|----------|---------|----------|----------|-------------|--------------|---------|--------|---|---------|----------|---|---------|--------|-----|-----|-----|--------|
| SPACES | GIC | TMR | — | EL2_HPFAR | EL2_ESR_FAR | EL2_ELR_SPSR | EL2_IDR | EL2_HCR | — | EL1_MDSCR | EL1_SCTLR | EL1_VBAR | EL1_MAIR | EL1_TCR | EL1_TTBR | EL1_AFSR | EL1_ESR_FAR | EL1_ELR_SPSR | EL1_IDR | EL1_SP | — | A32_DIH | A32_SPSR | — | EL0_IDR | EL0_SP | FPR | GPR | ICI | POISON |
| 31     | 30  | 29  |   | 27        | 26          | 25           | 24      | 23      |   | 20        | 19        | 18       | 17       | 16      | 15       | 14       | 13          | 12           | 11      | 10     |   | 8       | 7        |   | 5       | 4      | 3   | 2   | 1   | 0      |

Each [MTD](#) bit controls the transfer of the listed architectural state to/from the respective fields in the [UTCB](#) (7.6.3) as follows:

- State with access **r** can be read from the architectural state into the [UTCB](#).
- State with access **w** can be written from the [UTCB](#) into the architectural state.

| MTD Bit          | Access  | Host Execution Context State               | Guest Execution Context State                           |
|------------------|---------|--------------------------------------------|---------------------------------------------------------|
| POISON           | w       | Kills the <a href="#">EC<sub>HST</sub></a> | Kills the <a href="#">EC<sub>GST</sub></a>              |
| ICI <sup>†</sup> | w       | Invalidates the entire I-Cache             | Invalidates the entire I-Cache                          |
| GPR              | rw      | X0 ... X30                                 | X0 ... X30                                              |
| EL0_SP           | rw      | SP_EL0                                     | SP_EL0                                                  |
| EL0_IDR          | rw      | TPIDR_EL0, TPIDRRO_EL0                     | TPIDR_EL0, TPIDRRO_EL0                                  |
| A32_SPSR         | rw      | —                                          | SPSR_ABT, SPSR_FIQ, SPSR_IRQ, SPSR_UND                  |
| A32_DIH          | rw      | —                                          | DACR, IFSR, HSTR                                        |
| EL1_SP           | rw      | —                                          | SP_EL1                                                  |
| EL1_IDR          | rw      | —                                          | TPIDR_EL1, CONTEXTIDR_EL1                               |
| EL1_ELR_SPSR     | rw      | —                                          | ELR_EL1, SPSR_EL1                                       |
| EL1_ESR_FAR      | rw      | —                                          | ESR_EL1, FAR_EL1                                        |
| EL1_AFSR         | rw      | —                                          | AFSR0_EL1, AFSR1_EL1                                    |
| EL1_TTBR         | rw      | —                                          | TTBR0_EL1, TTBR1_EL1                                    |
| EL1_TCR          | rw      | —                                          | TCR_EL1                                                 |
| EL1_MAIR         | rw      | —                                          | MAIR_EL1, AMAIR_EL1                                     |
| EL1_VBAR         | rw      | —                                          | VBAR_EL1                                                |
| EL1_SCTLR        | rw      | —                                          | SCTLR_EL1                                               |
| EL1_MDSCR        | rw      | —                                          | MDSCR_EL1                                               |
| EL2_HCR          | rw      | —                                          | HCR_EL2, HCRX_EL2                                       |
| EL2_IDR          | rw      | —                                          | VPIDR_EL2, VMPIDR_EL2                                   |
| EL2_ELR_SPSR     | rw      | ELR_EL2, SPSR_EL2*                         | ELR_EL2, SPSR_EL2                                       |
| EL2_ESR_FAR      | r       | ESR_EL2, FAR_EL2                           | ESR_EL2, FAR_EL2                                        |
| EL2_HPFAR        | r       | —                                          | HPFAR_EL2                                               |
| TMR              | rw      | —                                          | CNTV_CVAL_EL0, CNTV_CTL_EL0<br>CNTKCTL_EL1, CNTVOFF_EL2 |
| GIC              | rw<br>r | —                                          | LR0 ... LR15, APxR0 ... APxR3<br>ELRSR, VMCR            |
| SPACES           | w       | —                                          | SEL_GST                                                 |

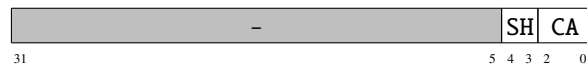
\*Only the condition flags are writable.

<sup>†</sup>Only affects a VIPT instruction cache of the local [CPU](#). Has no effect on PIPT instruction caches, data caches, or caches of other [CPUs](#).



## 7.6.5 Memory Attribute Descriptor

The Memory Attribute Descriptor (**MAD**) describes page attributes for a **CAP<sub>MEM</sub>** delegation from **SPC<sub>HSTNOVA</sub>**.



The fields are defined as follows:

### CA

Specifies the cacheability attributes of the page according to the following table:

| Encoding | Cacheability | Description                              |
|----------|--------------|------------------------------------------|
| 0x0      | DEV          | Device                                   |
| 0x1      | DEV_E        | Device, Early Ack                        |
| 0x2      | DEV_RE       | Device, Early Ack, Reordering            |
| 0x3      | DEV_GRE      | Device, Early Ack, Reordering, Gathering |
| 0x4      | –            | <i>reserved</i>                          |
| 0x5      | MEM_NC       | Memory, Inner/Outer Non-Cacheable        |
| 0x6      | MEM_WT       | Memory, Inner/Outer Write-Through        |
| 0x7      | MEM_WB       | Memory, Inner/Outer Write-Back           |

### SH

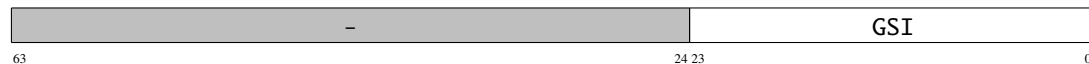
Specifies the shareability attributes of the page according to the following table:

| Encoding | Shareability | Description     |
|----------|--------------|-----------------|
| 0x0      | NONE         | Not Shareable   |
| 0x1      | –            | <i>reserved</i> |
| 0x2      | OUTER        | Outer Shareable |
| 0x3      | INNER        | Inner Shareable |

Please refer to [3] for the architectural behavior of these attributes.

## 7.6.6 Interrupt Semaphore Descriptor

The Interrupt Semaphore Descriptor (**ISD**) describes the interrupt associated with an Interrupt **Semaphore**.



The fields are defined as follows:

### GSI

Designates the interrupt via its **GSI** number (INTID) in one of the following ranges:

- [32, 32+**SPI<sub>NUM</sub>**) – Shared Peripheral Interrupt (**SPI**)
- [4096, 4096+**ESPI<sub>NUM</sub>**) – Extended Shared Peripheral Interrupt (**ESPI**)
- [8192, 8192+**LPI<sub>NUM</sub>**) – Locality-Specific Peripheral Interrupt (**LPI**)

## 7.6.7 Device Topology Descriptor

The Device Topology Descriptor (DTD) describes the topology of a device as follows:

### SID

Stream Identifier (SID) – if the device is subject to DMA translation.

### SID Mask

Specifies which bits of that SID should be matched (0) or ignored (1) by the Stream Mapping Group.

### SMG

Specifies the SMMU Stream Mapping Group (SMG) to use for that SID – must be < SMG<sub>NUM</sub>.

### CTX

Specifies the SMMU Translation Context (CTX) to use for that SMG – must be < CTX<sub>NUM</sub>.

### DID

Device Identifier (DID) – if the device is subject to MSI translation.

### SMMU

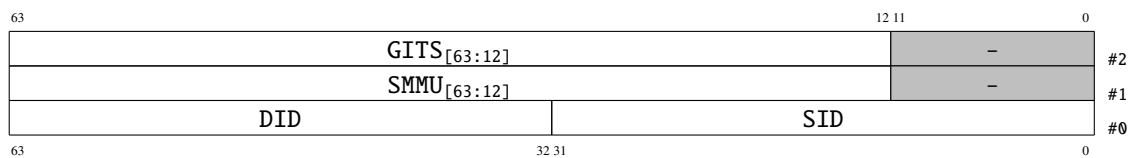
Host-Physical Address (HPA) of the SMMU that performs DMA translation for the device.

### GITS

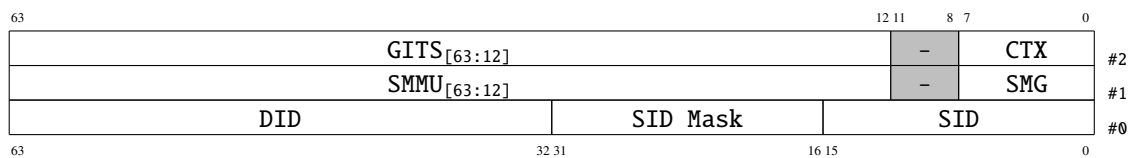
Host-Physical Address (HPA) of the GITS that performs MSI translation for the device.

Reserved fields and fields that do not apply to a particular platform must be zero.

### 7.6.7.1 SMMUv3



### 7.6.7.2 SMMUv2



System software must ensure an unambiguous assignment of Stream Identifiers to Stream Mapping Groups, i.e. it must configure the SID/Mask fields across all Stream Mapping Groups such that no SID multi-matches can occur.

## 7.7 Calling Convention

The following pages describes the calling convention for each hypercall. A [Host Execution Context](#) calls into the microhypervisor by loading the hypercall identifier and other parameters into the specified [CPU](#) registers and then executes the `svc #0` instruction [3].

The hypercall identifier consists of the [hypercall number](#) and hypercall-specific flags, as illustrated in Figure 7.1.

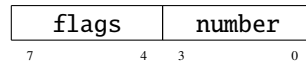


Figure 7.1: Hypercall Identifier

The [status code](#) returned from a hypercall has the format shown in Figure 7.2.

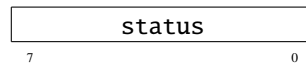
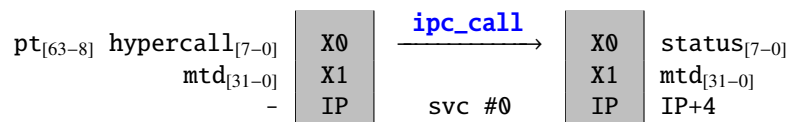


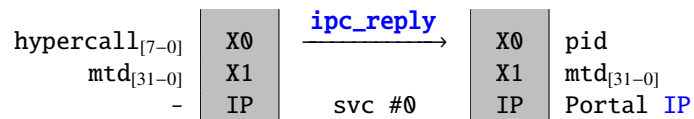
Figure 7.2: Status Code

The assignment of hypercall parameters to [CPU](#) registers is shown on the left side, the contents of the [CPU](#) registers after the hypercall is shown on the right side.

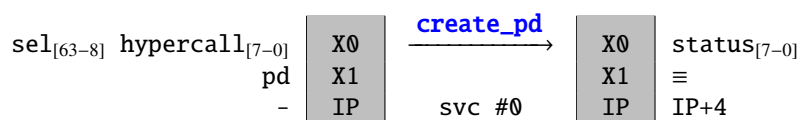
### IPC Call



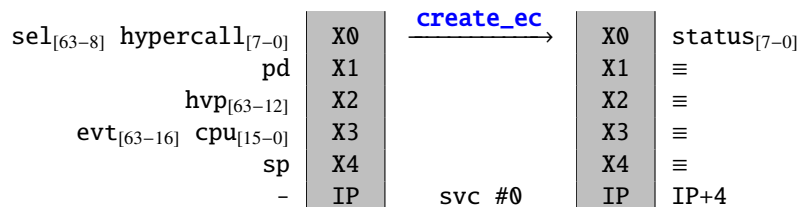
### IPC Reply



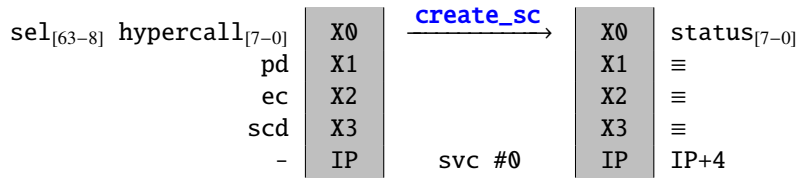
### Create Protection Domain



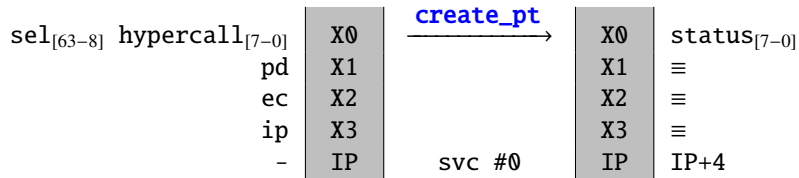
### Create Execution Context



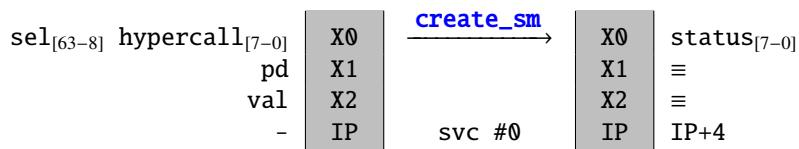
### Create Scheduling Context



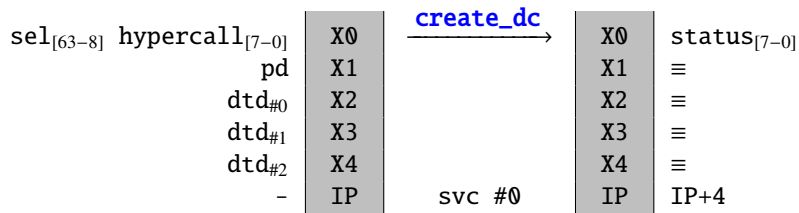
### Create Portal



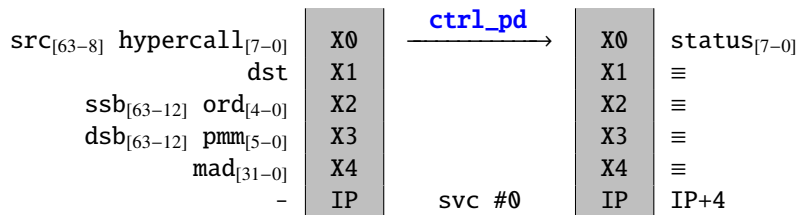
### Create Semaphore



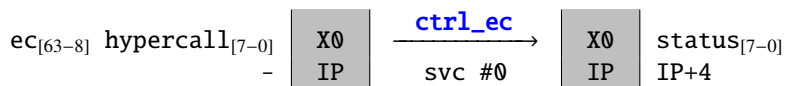
### Create Device Context



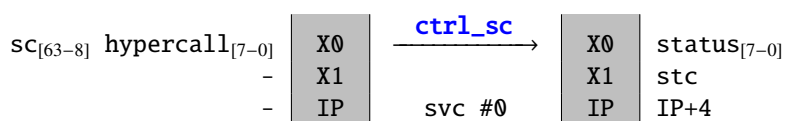
### Control Protection Domain



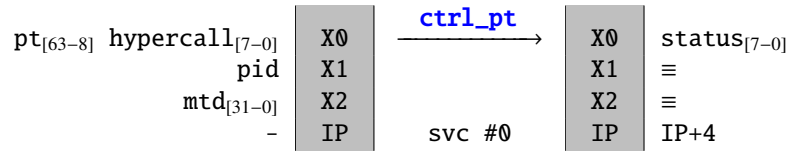
### Control Execution Context



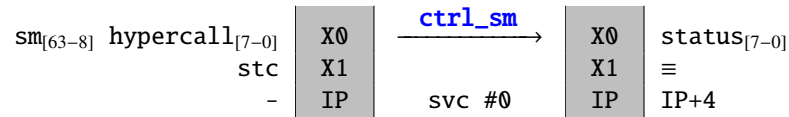
### Control Scheduling Context



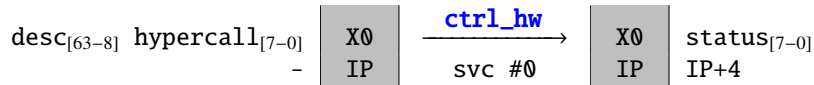
## Control Portal



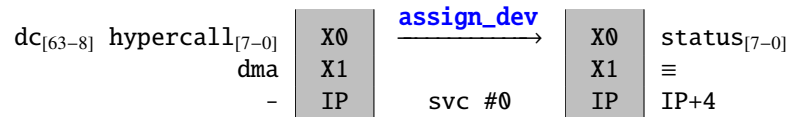
## Control Semaphore



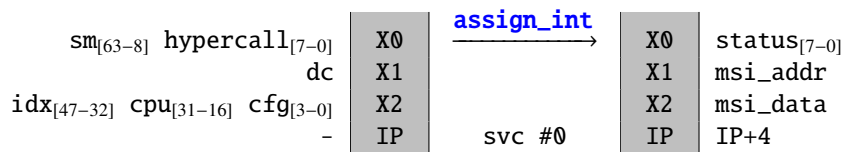
## Control Hardware



## Assign Device



## Assign Interrupt



## 7.8 Supplementary Functionality

This section describes functions that do **not** conform to the calling convention for hypercalls. Because these functions cannot perform capability-based access control, their invocation is restricted to the [Root Protection Domain](#) (PD<sub>ROOT</sub>). Invocation of these functions from any other [Protection Domain](#) generates an exception.

### Secure Monitor Call

| identifier <sub>[31-0]</sub> |     | proxy_smc → |     |      |
|------------------------------|-----|-------------|-----|------|
| -                            | X0  |             | X0  | ~    |
| -                            | X1  |             | X1  | ~    |
| -                            | X2  |             | X2  | ~    |
| -                            | X3  |             | X3  | ~    |
| -                            | X4  |             | X4  | ~    |
| -                            | X5  |             | X5  | ~    |
| -                            | X6  |             | X6  | ~    |
| -                            | X7  |             | X7  | ~    |
| -                            | X8  |             | X8  | ~    |
| -                            | X9  |             | X9  | ~    |
| -                            | X10 |             | X10 | ~    |
| -                            | X11 |             | X11 | ~    |
| -                            | X12 |             | X12 | ~    |
| -                            | X13 |             | X13 | ~    |
| -                            | X14 |             | X14 | ~    |
| -                            | X15 |             | X15 | ~    |
| -                            | X16 |             | X16 | ~    |
| -                            | X17 |             | X17 | ~    |
| -                            | IP  | svc #1      | IP  | IP+4 |

This call is proxy-filtered by the microhypervisor. If the combination of invoked service (identifier<sub>[29-24]</sub>) and function (identifier<sub>[15-0]</sub>) is listed in the table below, then the microhypervisor issues the corresponding SMC to platform firmware on behalf of the caller. Otherwise, this function generates an exception. Register allocation conforms to the Arm SMCCC [14].

| Service   | Description                   | Function      | Description         |
|-----------|-------------------------------|---------------|---------------------|
| 0x2       | SIP Service Calls             | 0x0000-0xffff | All functions       |
| 0x4       | Standard Secure Service Calls | 0x0050-0x005f | TRNG functions [15] |
|           |                               | 0x0130-0x013f | PCI functions [16]  |
| 0x30-0x31 | Trusted Application Calls     | 0x0000-0xffff | All functions       |
| 0x32-0x3f | Trusted OS Calls              | 0x0000-0xffff | All functions       |

## 8 ABI x86-64

### 8.1 Boot State

#### 8.1.1 NOVA Microhypervisor

The bootloader must set up the [CPU](#) register state according to one of the launch types listed below when it transfers control to the [NOVA](#) microhypervisor entry point. Furthermore, the following preconditions must be satisfied:

- The [CPU](#) state must conform to a machine state defined in the Multiboot Specification v2 [6] or v1 [9].
- All [DMA](#) activity targeting the physical memory region occupied by the microhypervisor must be quiesced. That physical memory region should also be protected against [DMA](#) accesses on systems with an [IOMMU](#).

##### 8.1.1.1 Multiboot v2 Launch

Only this launch type supports 64-bit [UEFI](#) platforms.

| Register | Value / Description                                                                |
|----------|------------------------------------------------------------------------------------|
| EIP      | Physical address of the <a href="#">NOVA ELF</a> image <a href="#">entry point</a> |
| EAX      | Multiboot v2 magic value (0x36d76289) [6]                                          |
| EBX      | Physical address of the Multiboot v2 information structure [6]                     |
| Other    | ↯                                                                                  |

The [NOVA](#) microhypervisor consumes the following multiboot tags, if present: 1, 3, 12, 15, 20.

##### 8.1.1.2 Multiboot v1 Launch

| Register | Value / Description                                                                |
|----------|------------------------------------------------------------------------------------|
| EIP      | Physical address of the <a href="#">NOVA ELF</a> image <a href="#">entry point</a> |
| EAX      | Multiboot v1 magic value (0x2badb002) [9]                                          |
| EBX      | Physical address of the Multiboot v1 information structure [9]                     |
| Other    | ↯                                                                                  |

The [NOVA](#) microhypervisor consumes the following multiboot flags, if present: 2, 3.

#### 8.1.2 Root Protection Domain

The [NOVA](#) microhypervisor sets up the [CPU](#) register state as follows when it transfers control to the [Root Execution Context](#) ( $EC_{\text{ROOT}}$ ):

| Register | Value / Description                                                                                                            |
|----------|--------------------------------------------------------------------------------------------------------------------------------|
| RIP      | <a href="#">HVA</a> of the <a href="#">Root Protection Domain</a> ( $PD_{\text{ROOT}}$ ) <a href="#">ELF</a> image entry point |
| RSP      | <a href="#">HVA</a> of the <a href="#">Hypervisor Information Page</a> ( <a href="#">HIP</a> )                                 |
| RDI      | EAX at <a href="#">boot time</a> <sup>†</sup>                                                                                  |
| RSI      | EBX at <a href="#">boot time</a> <sup>†</sup>                                                                                  |
| RDX      | ECX at <a href="#">boot time</a> <sup>†</sup>                                                                                  |
| Other    | ↯                                                                                                                              |

<sup>†</sup>The register contains the preserved original value from the point when control was transferred from the bootloader to the microhypervisor.

## 8.2 Memory Map

The **Root Protection Domain** ( $PD_{ROOT}$ ) can obtain a list of available/reserved physical memory regions as follows:

- On platforms using Multiboot v2 (UEFI boot services enabled), by parsing the **UEFI memory map** [8].
- On platforms using Multiboot v2, by parsing the **Multiboot v2** memory map [6].
- On platforms using Multiboot v1, by parsing the **Multiboot v1** memory map [9].

## 8.3 Class Of Service

Class Of Service (**COS**) support is indicated by CPUID leaf  $0x7$ , sub-leaf  $0x0$ :  $EBX_{[15]}$ .

The **Root Protection Domain** ( $PD_{ROOT}$ ) must perform the following steps on each **CPU** to configure **QOS** settings:

1. Invoke **ctrl\_hw** to establish a valid **QOS** configuration:
  - **CDP<sub>L3</sub>** support is indicated by CPUID leaf  $0x10$ , sub-leaf  $0x1$ :  $ECX_{[2]}$
  - **CDP<sub>L2</sub>** support is indicated by CPUID leaf  $0x10$ , sub-leaf  $0x2$ :  $ECX_{[2]}$
2. Determine **COS<sub>NUM</sub>** as the maximum of the following:
  - **COS<sub>L3</sub>** from CPUID leaf  $0x10$ , sub-leaf  $0x1$ :  $(1+EDX_{[15-0]}) >> X$ , where
    - $X=0$  if **CDP<sub>L3</sub>** is disabled.
    - $X=1$  if **CDP<sub>L3</sub>** is enabled.
  - **COS<sub>L2</sub>** from CPUID leaf  $0x10$ , sub-leaf  $0x2$ :  $(1+EDX_{[15-0]}) >> X$ , where
    - $X=0$  if **CDP<sub>L2</sub>** is disabled.
    - $X=1$  if **CDP<sub>L2</sub>** is enabled.
  - **COS<sub>MB</sub>** from CPUID leaf  $0x10$ , sub-leaf  $0x3$ :  $(1+EDX_{[15-0]})$
3. Invoke **ctrl\_hw** to configure the following:
  - For each **COS<sub>L3</sub>**: **CAT/CDP** L3 Capacity Bitmask(s)
  - For each **COS<sub>L2</sub>**: **CAT/CDP** L2 Capacity Bitmask(s)
  - For each **COS<sub>MB</sub>**: **MBA** Delay

## 8.4 Protected Resources

Certain resources protected by the **NOVA** microhypervisor cannot be delegated and therefore remain inaccessible to user-mode components. The following subsections enumerate these protected resources.

### 8.4.1 Physical Memory

The following physical memory regions are protected:

- **NOVA** microhypervisor – conveyed via **HIP**.
- **LAPIC**, **IOAPIC** devices – conveyed via **ACPI** MADT.
- **IOMMU** devices [17, 18] – conveyed via **ACPI** DMAR or IVRS.
- **DPR** and **TXT** memory [19], except for **TPM** localities 0 and 1.
- **MSI** range – at  $0xfeef00000$
- Firmware runtime services – conveyed via **UEFI** memory map.



## 8.4.2 I/O Ports

The **CAP<sub>PIO</sub>** column in the table below details the permissions granted for I/O ports by the **NOVA PIO Space**.

| I/O Port                                   | CAP <sub>PIO</sub> | Exception or I/O Exit | Enumeration | VMM Handling           |
|--------------------------------------------|--------------------|-----------------------|-------------|------------------------|
| PM1a_CNT<br>PM1b_CNT<br>PM2_CNT<br>SMI_CMD | CAP <sub>0</sub>   | Always                | ACPI FADT   | Emulate                |
| Other                                      | A                  | If A is not set       | –           | Emulate or Passthrough |

## 8.4.3 Model-Specific Registers

The **CAP<sub>MSR</sub>** column in the table below details the permissions granted for **MSRs** by the **NOVA MSR Space**.

| Model-Specific Register                                                                                                                                                                                                                                                                                                                                                                                  | CAP <sub>MSR</sub> | RDMSR Exit      | WRMSR Exit      | VMM Handling           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-----------------|-----------------|------------------------|
| IA32_SYSENTER_CS*<br>IA32_SYSENTER_ESP*<br>IA32_SYSENTER_EIP*<br>IA32_PAT*<br>IA32_EFER*<br>IA32_FS_BASE*<br>IA32_GS_BASE*<br>IA32_KERNEL_GS_BASE*                                                                                                                                                                                                                                                       | RW                 | If R is not set | If W is not set | Emulate or Passthrough |
| IA32_PRED_CMD<br>IA32_FLUSH_CMD                                                                                                                                                                                                                                                                                                                                                                          | W                  | Always          | If W is not set |                        |
| IA32_TSC<br>IA32_PLATFORM_ID<br>IA32_BIOS_SIGN_ID<br>IA32_SGXLEPUBKEYHASH{0-3}*<br>IA32_CORE_CAPABILITIES<br>IA32_MPERF<br>IA32_APERF<br>IA32_MTRR_CAP<br>IA32_ARCH_CAPABILITIES<br>IA32_MCU_OPT_CTRL<br>IA32_OVERCLOCKING_STATUS<br>IA32_PERF_STATUS<br>IA32_THERM_STATUS<br>IA32_PACKAGE_THERM_STATUS<br>IA32_SGX_SVN_STATUS<br>IA32_XSS*<br>IA32_STAR*<br>IA32_LSTAR*<br>IA32_FMASK*<br>IA32_TSC_AUX* | R                  | If R is not set | Always          |                        |
| Other                                                                                                                                                                                                                                                                                                                                                                                                    | CAP <sub>0</sub>   | Always          | Always          | Emulate                |

\*The VMM can read/write the guest-effective **MSR** value via the **UTCB**.

## 8.5 Event-Specific Capability Selectors

For the delivery of exception/intercept messages, the microhypervisor performs an implicit portal traversal.

The selector for the destination portal ( $SEL_{OBJ}$ ):

- is determined by adding the exception/intercept number to the affected [Execution Context](#)'s Event Selector Base ( $SEL_{EVT}$ ).
- indexes into the [Object Space](#) ( $SPC_{OBJ}$ ) of the affected [EC](#)'s [Protection Domain](#) ([PD](#)).
- must refer to a [PT Capability](#) ( $CAP_{OBJPT}$ ) with permission `EVENT` that is bound to an [EC](#) on the same [CPU](#) as the affected [EC](#), otherwise the affected [EC](#) is killed.

### 8.5.1 Architectural Events

#### Host Exceptions

| $SEL_{OBJ}$        | Exception | $SEL_{OBJ}$        | Exception |
|--------------------|-----------|--------------------|-----------|
| $SEL_{EVT} + 0x00$ | #DE       | $SEL_{EVT} + 0x10$ | #MF       |
| $SEL_{EVT} + 0x01$ | #DB       | $SEL_{EVT} + 0x11$ | #AC       |
| $SEL_{EVT} + 0x02$ | reserved  | $SEL_{EVT} + 0x12$ | #MC*      |
| $SEL_{EVT} + 0x03$ | #BP       | $SEL_{EVT} + 0x13$ | #XM       |
| $SEL_{EVT} + 0x04$ | #OF       | $SEL_{EVT} + 0x14$ | #VE       |
| $SEL_{EVT} + 0x05$ | #BR       | $SEL_{EVT} + 0x15$ | #CP       |
| $SEL_{EVT} + 0x06$ | #UD       | $SEL_{EVT} + 0x16$ | reserved  |
| $SEL_{EVT} + 0x07$ | #NM*      | $SEL_{EVT} + 0x17$ | reserved  |
| $SEL_{EVT} + 0x08$ | #DF*      | $SEL_{EVT} + 0x18$ | reserved  |
| $SEL_{EVT} + 0x09$ | reserved  | $SEL_{EVT} + 0x19$ | reserved  |
| $SEL_{EVT} + 0x0a$ | #TS*      | $SEL_{EVT} + 0x1a$ | reserved  |
| $SEL_{EVT} + 0x0b$ | #NP       | $SEL_{EVT} + 0x1b$ | reserved  |
| $SEL_{EVT} + 0x0c$ | #SS       | $SEL_{EVT} + 0x1c$ | reserved  |
| $SEL_{EVT} + 0x0d$ | #GP       | $SEL_{EVT} + 0x1d$ | reserved  |
| $SEL_{EVT} + 0x0e$ | #PF       | $SEL_{EVT} + 0x1e$ | reserved  |
| $SEL_{EVT} + 0x0f$ | reserved  | $SEL_{EVT} + 0x1f$ | reserved  |

\*These events may be handled by the microhypervisor, in which case they will not cause portal traversals.

†These events may be force-enabled by the microhypervisor, in which case they will cause portal traversals.

## Guest Intercepts (VMX)

| SEL <sub>OBJ</sub>        | Intercept                 | SEL <sub>OBJ</sub>        | Intercept                   |
|---------------------------|---------------------------|---------------------------|-----------------------------|
| SEL <sub>EVT</sub> + 0x00 | Exception or NMI*         | SEL <sub>EVT</sub> + 0x30 | EPT Violation†              |
| SEL <sub>EVT</sub> + 0x01 | External Interrupt*       | SEL <sub>EVT</sub> + 0x31 | EPT Misconfiguration        |
| SEL <sub>EVT</sub> + 0x02 | Triple Fault†             | SEL <sub>EVT</sub> + 0x32 | INVEPT                      |
| SEL <sub>EVT</sub> + 0x03 | INIT†                     | SEL <sub>EVT</sub> + 0x33 | RDTSCP                      |
| SEL <sub>EVT</sub> + 0x04 | SIPI†                     | SEL <sub>EVT</sub> + 0x34 | Preemption Timer            |
| SEL <sub>EVT</sub> + 0x05 | I/O SMI                   | SEL <sub>EVT</sub> + 0x35 | INVVPID                     |
| SEL <sub>EVT</sub> + 0x06 | Other SMI                 | SEL <sub>EVT</sub> + 0x36 | WBINVD, WBNOINVD            |
| SEL <sub>EVT</sub> + 0x07 | Interrupt Window          | SEL <sub>EVT</sub> + 0x37 | XSETBV                      |
| SEL <sub>EVT</sub> + 0x08 | NMI Window                | SEL <sub>EVT</sub> + 0x38 | APIC Write                  |
| SEL <sub>EVT</sub> + 0x09 | Task Switch†              | SEL <sub>EVT</sub> + 0x39 | RDRAND                      |
| SEL <sub>EVT</sub> + 0x0a | CPUID†                    | SEL <sub>EVT</sub> + 0x3a | INVPCID                     |
| SEL <sub>EVT</sub> + 0x0b | GETSEC†                   | SEL <sub>EVT</sub> + 0x3b | VMFUNC                      |
| SEL <sub>EVT</sub> + 0x0c | HLT†                      | SEL <sub>EVT</sub> + 0x3c | ENCLS                       |
| SEL <sub>EVT</sub> + 0x0d | INVD†                     | SEL <sub>EVT</sub> + 0x3d | RDSEED                      |
| SEL <sub>EVT</sub> + 0x0e | INVLPG                    | SEL <sub>EVT</sub> + 0x3e | PML Log Full                |
| SEL <sub>EVT</sub> + 0x0f | RDPMC                     | SEL <sub>EVT</sub> + 0x3f | XSAVES                      |
| SEL <sub>EVT</sub> + 0x10 | RDTSC                     | SEL <sub>EVT</sub> + 0x40 | XRSTORS                     |
| SEL <sub>EVT</sub> + 0x11 | RSM                       | SEL <sub>EVT</sub> + 0x41 | PCONFIG                     |
| SEL <sub>EVT</sub> + 0x12 | VMCALL                    | SEL <sub>EVT</sub> + 0x42 | SPP Miss / Misconfiguration |
| SEL <sub>EVT</sub> + 0x13 | VMCLEAR                   | SEL <sub>EVT</sub> + 0x43 | UMWAIT                      |
| SEL <sub>EVT</sub> + 0x14 | VMLAUNCH                  | SEL <sub>EVT</sub> + 0x44 | TPAUSE                      |
| SEL <sub>EVT</sub> + 0x15 | VMPTRLD                   | SEL <sub>EVT</sub> + 0x45 | LOADIWKEY                   |
| SEL <sub>EVT</sub> + 0x16 | VMPTRST                   | SEL <sub>EVT</sub> + 0x46 | ENCLV                       |
| SEL <sub>EVT</sub> + 0x17 | VMREAD                    | SEL <sub>EVT</sub> + 0x47 | SGX Conflict                |
| SEL <sub>EVT</sub> + 0x18 | VMRESUME                  | SEL <sub>EVT</sub> + 0x48 | ENQCMD PASID Failure        |
| SEL <sub>EVT</sub> + 0x19 | VMWRITE                   | SEL <sub>EVT</sub> + 0x49 | ENQCMDS PASID Failure       |
| SEL <sub>EVT</sub> + 0x1a | VMXOFF                    | SEL <sub>EVT</sub> + 0x4a | Bus Lock                    |
| SEL <sub>EVT</sub> + 0x1b | VMXON                     | SEL <sub>EVT</sub> + 0x4b | Instruction Timeout         |
| SEL <sub>EVT</sub> + 0x1c | CR Access*                | SEL <sub>EVT</sub> + 0x4c | SEAMCALL                    |
| SEL <sub>EVT</sub> + 0x1d | DR Access                 | SEL <sub>EVT</sub> + 0x4d | TDCALL                      |
| SEL <sub>EVT</sub> + 0x1e | I/O Access†               | SEL <sub>EVT</sub> + 0x4e | RDMSRLIST                   |
| SEL <sub>EVT</sub> + 0x1f | RDMSR†                    | SEL <sub>EVT</sub> + 0x4f | WRMSRLIST                   |
| SEL <sub>EVT</sub> + 0x20 | WRMSR†                    | SEL <sub>EVT</sub> + 0x50 | URDMSR                      |
| SEL <sub>EVT</sub> + 0x21 | VM Entry Failure (State)† | SEL <sub>EVT</sub> + 0x51 | UWRMSR                      |
| SEL <sub>EVT</sub> + 0x22 | VM Entry Failure (MSR)    | SEL <sub>EVT</sub> + 0x52 | reserved                    |
| SEL <sub>EVT</sub> + 0x23 | reserved                  | SEL <sub>EVT</sub> + 0x53 | reserved                    |
| SEL <sub>EVT</sub> + 0x24 | MWAIT                     | SEL <sub>EVT</sub> + 0x54 | reserved                    |
| SEL <sub>EVT</sub> + 0x25 | MTF                       | SEL <sub>EVT</sub> + 0x55 | reserved                    |
| SEL <sub>EVT</sub> + 0x26 | reserved                  | SEL <sub>EVT</sub> + 0x56 | reserved                    |
| SEL <sub>EVT</sub> + 0x27 | MONITOR                   | SEL <sub>EVT</sub> + 0x57 | reserved                    |
| SEL <sub>EVT</sub> + 0x28 | PAUSE                     | SEL <sub>EVT</sub> + 0x58 | reserved                    |
| SEL <sub>EVT</sub> + 0x29 | VM Entry Failure (MCE)    | SEL <sub>EVT</sub> + 0x59 | reserved                    |
| SEL <sub>EVT</sub> + 0x2a | reserved                  | SEL <sub>EVT</sub> + 0x5a | reserved                    |
| SEL <sub>EVT</sub> + 0x2b | TPR Below Threshold       | SEL <sub>EVT</sub> + 0x5b | reserved                    |
| SEL <sub>EVT</sub> + 0x2c | APIC Access               | SEL <sub>EVT</sub> + 0x5c | reserved                    |
| SEL <sub>EVT</sub> + 0x2d | Virtualized EOI           | SEL <sub>EVT</sub> + 0x5d | reserved                    |
| SEL <sub>EVT</sub> + 0x2e | GDTR/IDTR Access          | SEL <sub>EVT</sub> + 0x5e | reserved                    |
| SEL <sub>EVT</sub> + 0x2f | LDTR/TR Access            | SEL <sub>EVT</sub> + 0x5f | reserved                    |

Please refer to [4] for more details on each of these events.

## 8.5.2 Microhypervisor Events

| <b>SEL<sub>OBJ</sub></b>                             | <b>Event</b> |
|------------------------------------------------------|--------------|
| <b>SEL<sub>EVT</sub></b> + SEL <sub>ARCH</sub> + 0x0 | Startup      |
| <b>SEL<sub>EVT</sub></b> + SEL <sub>ARCH</sub> + 0x1 | Recall       |

The value of SEL<sub>ARCH</sub> depends on the origin of the event:

- SEL<sub>ARCH</sub> = SEL<sub>HST/ARCH</sub> for events that occurred in the host.
- SEL<sub>ARCH</sub> = SEL<sub>GST/ARCH</sub> for events that occurred in the guest.

## 8.6 Architecture-Dependent Structures

### 8.6.1 Platform Features

The supported platform features are defined as follows:

|   |     |     |       |       |                                              |
|---|-----|-----|-------|-------|----------------------------------------------|
| I | SVM | VMX | IOMMU | IOMMU | If set, then <a href="#">IOMMU</a> is active |
|   |     |     |       | VMX   | If set, then Intel VMX is active             |
|   |     |     |       | SVM   | If set, then AMD SVM is active               |
| 3 | 2   | 1   | 0     |       |                                              |

### 8.6.2 Hypervisor Information Page

|                            |       |  |  |  |                    |  |     |   |                    |           |
|----------------------------|-------|--|--|--|--------------------|--|-----|---|--------------------|-----------|
| 63                         | 32 31 |  |  |  | 0                  |  |     |   | +Length            |           |
| Event Log Offset           |       |  |  |  | Event Log Size     |  |     |   |                    | Arch+0x10 |
| Event Log Physical Address |       |  |  |  |                    |  |     |   |                    | Arch+0x08 |
| GSI <sub>NUM</sub>         |       |  |  |  | PIN <sub>NUM</sub> |  |     | ⌘ | VEC <sub>NUM</sub> | Arch+0x00 |
| 63                         | 32 31 |  |  |  | 16 15              |  | 8 7 |   | 0                  |           |

#### VEC<sub>NUM</sub>

Total number of per-CPU interrupt vectors.

#### PIN<sub>NUM</sub>

Total number of interrupts that are type PIN in [GSI](#) range [0, PIN<sub>NUM</sub>). All other interrupts are type [MSI](#).

#### GSI<sub>NUM</sub>

Total number of Global System Interrupts ([GSIs](#)).

#### Event Log Physical Address

Page-aligned physical address of the [TPM](#) event log [20].

#### Event Log Size

Size of the [TPM](#) event log in bytes.

#### Event Log Offset

Offset of the byte after the last [TPM](#) event log entry.

### 8.6.3 User Thread Control Block

|                       |                    |                       |                    |          |        |
|-----------------------|--------------------|-----------------------|--------------------|----------|--------|
| SEL_MSR               |                    | SEL_PIO               |                    | +0x280   |        |
| SEL_GST               |                    | IA32_TSC_AUX          |                    | +0x270   |        |
| IA32_KERNEL_GS_BASE   |                    | IA32_FMASK            |                    | +0x260   |        |
| IA32_LSTAR            |                    | IA32_STAR             |                    | +0x250   |        |
| IA32_EFER             |                    | IA32_PAT              |                    | +0x240   |        |
| IA32_SYSENTER_EIP     |                    | IA32_SYSENTER_ESP     |                    | +0x230   |        |
| IA32_SYSENTER_CS      |                    | IA32_APIC_BASE        |                    | +0x220   |        |
| IA32_SGXLEPUBKEYHASH3 |                    | IA32_SGXLEPUBKEYHASH2 |                    | +0x210   |        |
| IA32_SGXLEPUBKEYHASH1 |                    | IA32_SGXLEPUBKEYHASH0 |                    | +0x200   |        |
| IA32_XSS              |                    | XCR0                  |                    | +0x1f0   |        |
| DR7                   |                    | CR8                   |                    | +0x1e0   |        |
| CR4                   |                    | CR3                   |                    | +0x1d0   |        |
| CR2                   |                    | CR0                   |                    | +0x1c0   |        |
| PDPTE3                |                    | PDPTE2                |                    | +0x1b0   |        |
| PDPTE1                |                    | PDPTE0                |                    | +0x1a0   |        |
| Base IDTR             |                    | Limit IDTR            | -                  |          | +0x190 |
| Base GDTR             |                    | Limit GDTR            | -                  |          | +0x180 |
| Base LDTR             |                    | Limit LDTR            | AR LDTR*           | SEL LDTR | +0x170 |
| Base TR               |                    | Limit TR              | AR TR*             | SEL TR   | +0x160 |
| Base GS               |                    | Limit GS              | AR GS*             | SEL GS   | +0x150 |
| Base FS               |                    | Limit FS              | AR FS*             | SEL FS   | +0x140 |
| Base ES               |                    | Limit ES              | AR ES*             | SEL ES   | +0x130 |
| Base DS               |                    | Limit DS              | AR DS*             | SEL DS   | +0x120 |
| Base SS               |                    | Limit SS              | AR SS*             | SEL SS   | +0x110 |
| Base CS               |                    | Limit CS              | AR CS*             | SEL CS   | +0x100 |
| IDT Vectoring Error   | IDT Vectoring Info | Interrupt Error       | Interrupt Info†    |          | +0x0f0 |
| TPR Threshold         | PF Error Match     | PF Error Mask         | EXC Intercepts     |          | +0x0e0 |
| CR4 Intercepts        |                    | CR0 Intercepts        |                    |          | +0x0d0 |
| 3rd Exec Controls     |                    | 2nd Exec Controls     | 1st Exec Controls  |          | +0x0c0 |
| -                     |                    | 3rd Qualification     |                    |          | +0x0b0 |
| 2nd Qualification     |                    | 1st Qualification     |                    |          | +0x0a0 |
| Activity              | Interruptibility   | Instruction Info      | Instruction Length |          | +0x090 |
| RIP                   |                    | RFLAGS                |                    |          | +0x080 |
| R15                   |                    | R14                   |                    |          | +0x070 |
| R13                   |                    | R12                   |                    |          | +0x060 |
| R11                   |                    | R10                   |                    |          | +0x050 |
| R9                    |                    | R8                    |                    |          | +0x040 |
| R7 (RDI)              |                    | R6 (RSI)              |                    |          | +0x030 |
| R5 (RBP)              |                    | R4 (RSP)              |                    |          | +0x020 |
| R3 (RBX)              |                    | R2 (RDX)              |                    |          | +0x010 |
| R1 (RCX)              |                    | R0 (RAX)              |                    |          | +0x000 |

\*See Section 8.6.3.1 for encoding details.

†See Section 8.6.3.2 for encoding details.

#### 8.6.3.1 Encoding: Segment Access Rights

|              |                                            |    |     |   |     |   |     |   |      |
|--------------|--------------------------------------------|----|-----|---|-----|---|-----|---|------|
| –            | U                                          | G  | D/B | L | AVL | P | DPL | S | Type |
|              | 12                                         | 11 | 10  | 9 | 8   | 7 | 6   | 5 | 4    |
|              |                                            |    |     |   |     |   | 3   |   | 0    |
| <b>Field</b> | <b>Description</b>                         |    |     |   |     |   |     |   |      |
| U            | 0 = Segment Usable<br>1 = Segment Unusable |    |     |   |     |   |     |   |      |
| G            | Granularity                                |    |     |   |     |   |     |   |      |
| D/B          | 0 = 16-bit segment<br>1 = 32-bit segment   |    |     |   |     |   |     |   |      |
| L            | 64-bit mode active (CS only)               |    |     |   |     |   |     |   |      |
| AVL          | Available for use by system software       |    |     |   |     |   |     |   |      |
| P            | Segment Present                            |    |     |   |     |   |     |   |      |
| DPL          | Descriptor Privilege Level                 |    |     |   |     |   |     |   |      |
| S            | 0 = System<br>1 = Code or Data             |    |     |   |     |   |     |   |      |
| Type         | Segment Type                               |    |     |   |     |   |     |   |      |

### 8.6.3.2 Encoding: Interruption Information

|              |  |  |  |  |  |  |  |  |  |  |                                                                                                                                                                                                                        |   |   |      |        |  |  |  |  |  |  |
|--------------|--|--|--|--|--|--|--|--|--|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|------|--------|--|--|--|--|--|--|
| V            |  |  |  |  |  |  |  |  |  |  | N                                                                                                                                                                                                                      | I | E | Type | Vector |  |  |  |  |  |  |
| 31           |  |  |  |  |  |  |  |  |  |  | 13 12 11 10 8 7 0                                                                                                                                                                                                      |   |   |      |        |  |  |  |  |  |  |
| <b>Field</b> |  |  |  |  |  |  |  |  |  |  | <b>Description</b>                                                                                                                                                                                                     |   |   |      |        |  |  |  |  |  |  |
| V            |  |  |  |  |  |  |  |  |  |  | 0 = Fields E, Type, Vector are invalid<br>1 = Fields E, Type, Vector are valid                                                                                                                                         |   |   |      |        |  |  |  |  |  |  |
| N            |  |  |  |  |  |  |  |  |  |  | 0 = Do not request an NMI window<br>1 = Request an NMI window                                                                                                                                                          |   |   |      |        |  |  |  |  |  |  |
| I            |  |  |  |  |  |  |  |  |  |  | 0 = Do not request an interrupt window<br>1 = Request an interrupt window                                                                                                                                              |   |   |      |        |  |  |  |  |  |  |
| E            |  |  |  |  |  |  |  |  |  |  | 0 = Do not deliver the error code from the <b>UTCB</b> Interruption Error field<br>1 = Deliver the error code from the <b>UTCB</b> Interruption Error field                                                            |   |   |      |        |  |  |  |  |  |  |
| Type         |  |  |  |  |  |  |  |  |  |  | 0 = External Interrupt<br>2 = Non-Maskable Interrupt<br>3 = Hardware Exception<br>4 = Software Interrupt<br>5 = Privileged Software Exception<br>6 = Software Exception<br>7 = Other Event (not delivered through IDT) |   |   |      |        |  |  |  |  |  |  |
| Vector       |  |  |  |  |  |  |  |  |  |  | IDT Vector of Interrupt or Exception                                                                                                                                                                                   |   |   |      |        |  |  |  |  |  |  |

## 8.6.4 Message Transfer Descriptor

The [Message Transfer Descriptor \(MTD\)](#), which controls the subset of the architectural state transferred during exceptions and intercepts, as described in [Section 4.4.2](#), has the following layout:

|        |     |     |     |           |      |     |          |         |      |     |       |    |    |       |      |      |      |    |       |       |       |     |     |      |      |     |     |        |                     |                    |        |
|--------|-----|-----|-----|-----------|------|-----|----------|---------|------|-----|-------|----|----|-------|------|------|------|----|-------|-------|-------|-----|-----|------|------|-----|-----|--------|---------------------|--------------------|--------|
| SPACES | FPU | TLB | TSC | KERNEL_GS | EFER | PAT | SYSENTER | SYSCALL | APIC | SGX | XSAVE | DR | CR | PDPTE | IDTR | GDTR | LDTR | TR | FS/GS | DS/ES | CS/SS | INJ | TPR | CTRL | QUAL | STA | RIP | RFLAGS | GPR <sub>8-15</sub> | GPR <sub>0-7</sub> | POISON |
| 31     | 30  | 29  | 28  | 27        | 26   | 25  | 24       | 23      | 22   | 21  | 20    | 19 | 18 | 17    | 16   | 15   | 14   | 13 | 12    | 11    | 10    | 9   | 8   | 7    | 6    | 5   | 4   | 3      | 2                   | 1                  | 0      |

Each [MTD](#) bit controls the transfer of the listed architectural state to/from the respective fields in the [UTCB](#) ([8.6.3](#)) as follows:

- State with access **r** can be read from the architectural state into the [UTCB](#).
- State with access **w** can be written from the [UTCB](#) into the architectural state.

| MTD Bit             | Access  | Host Execution Context State               | Guest Execution Context State                                                    |
|---------------------|---------|--------------------------------------------|----------------------------------------------------------------------------------|
| POISON              | w       | Kills the <a href="#">EC<sub>HST</sub></a> | Kills the <a href="#">EC<sub>GST</sub></a>                                       |
| GPR <sub>0-7</sub>  | rw      | R0 ... R7                                  | R0 ... R7                                                                        |
| GPR <sub>8-15</sub> | rw      | R8 ... R15                                 | R8 ... R15                                                                       |
| RFLAGS              | rw      | RFLAGS*                                    | RFLAGS                                                                           |
| RIP                 | rw      | RIP                                        | RIP, Instruction Length, Instruction Info                                        |
| STA                 | rw      | -                                          | Interruptibility State, Activity State                                           |
| QUAL                | r       | Qualifications <sup>†</sup>                | Qualifications <sup>‡</sup>                                                      |
| CTRL                | w       | -                                          | Execution Controls, CR Intercepts<br>EXC Intercepts, PF Error Mask/Match         |
| TPR                 | w       | -                                          | TPR Threshold                                                                    |
| INJ                 | rw<br>r | -                                          | Interruption Info, Interruption Error<br>IDT Vectoring Info, IDT Vectoring Error |
| CS/SS               | rw      | -                                          | CS, SS (Selector, Base, Limit, AR)                                               |
| DS/ES               | rw      | -                                          | DS, ES (Selector, Base, Limit, AR)                                               |
| FS/GS               | rw      | -                                          | FS, GS (Selector, Base, Limit, AR)                                               |
| TR                  | rw      | -                                          | TR (Selector, Base, Limit, AR)                                                   |
| LDTR                | rw      | -                                          | LDTR (Selector, Base, Limit, AR)                                                 |
| GDTR                | rw      | -                                          | GDTR (Base, Limit)                                                               |
| IDTR                | rw      | -                                          | IDTR (Base, Limit)                                                               |
| PDPTE               | rw      | -                                          | PDPTE{0-3}                                                                       |
| CR                  | rw      | -                                          | CR0, CR2, CR3, CR4, CR8                                                          |
| DR                  | rw      | -                                          | DR7                                                                              |
| XSAVE               | rw      | -                                          | XCR0, IA32_XSS                                                                   |
| SGX                 | rw      | -                                          | IA32_SGXLEPUBKEYHASH{0-3}                                                        |
| APIC                | w       | -                                          | IA32_APIC_BASE <sup>§</sup>                                                      |
| SYSCALL             | rw      | -                                          | IA32_{STAR, LSTAR, FMASK}                                                        |
| SYSENTER            | rw      | -                                          | IA32_SYSENTER_{CS, ESP, EIP}                                                     |
| PAT                 | rw      | -                                          | IA32_PAT                                                                         |
| EFER                | rw      | -                                          | IA32_EFER                                                                        |
| KERNEL_GS           | rw      | -                                          | IA32_KERNEL_GS_BASE                                                              |
| TSC                 | rw      | -                                          | IA32_TSC_AUX                                                                     |
| TLB                 | w       | -                                          | Invalidates the TLB for the vCPU                                                 |
| SPACES              | w       | -                                          | SEL_GST, SEL_PIO, SEL_MSR                                                        |

\*Only the status and control flags are writable.

<sup>†</sup>Qualification fields contain exception error code (1st), page-fault linear address (2nd).

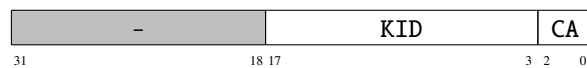
<sup>‡</sup>Qualification fields contain exit qualification (1st), guest-linear address (2nd), guest-physical address (3rd).

<sup>§</sup>APIC access page guest-physical address in the vCPU's currently assigned guest space.



### 8.6.5 Memory Attribute Descriptor

The Memory Attribute Descriptor (**MAD**) describes page attributes for a  $\text{CAP}_{\text{MEM}}$  delegation from  $\text{SPC}_{\text{HST}_{\text{NOVA}}}$ .



The fields are defined as follows:

**CA**

Specifies the cacheability attributes of the page according to the following table:

| Encoding | Cacheability | Description        |
|----------|--------------|--------------------|
| 0x0      | WB           | Write Back         |
| 0x1      | WT           | Write Through      |
| 0x2      | WC           | Write Combining    |
| 0x3      | UC           | Strong Uncacheable |
| 0x4      | WP           | Write Protected    |
| 0x5      | –            | <i>reserved</i>    |
| 0x6      | –            | <i>reserved</i>    |
| 0x7      | –            | <i>reserved</i>    |

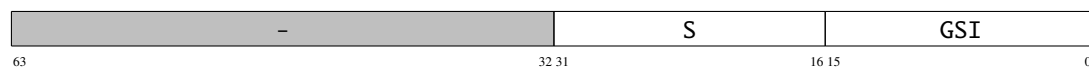
**KID**

Specifies the key identifier of the cryptographic key to be used for memory encryption – must be  $\leq \text{KID}_{\text{MAX}}$ .

Please refer to [4, 5] for the architectural behavior of these attributes.

### 8.6.6 Interrupt Semaphore Descriptor

The Interrupt Semaphore Descriptor (ISD) describes a GSI associated with an Interrupt Semaphore.



The fields are defined as follows:

**S**

## PCI Segment

**GSI**

Designates the interrupt via its [GSI](#) number in the following range:

- $[0, \text{GSI}_{\text{NUM}})$  – Global System Interrupt (GSI)

### 8.6.7 Device Topology Descriptor

The Device Topology Descriptor (DTD) describes the topology of a device as follows:

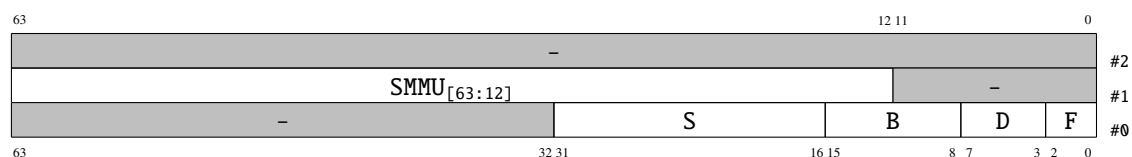
**S, B, D, F**

PCI Segment/Bus/Device/Function (SBDF).

## SMMU

Host-Physical Address (HPA) of the SMMU that performs DMA and MSI translation for the device.

Reserved fields and fields that do not apply to a particular platform must be zero.



## 8.7 Calling Convention

The following pages describes the calling convention for each hypercall. A [Host Execution Context](#) calls into the microhypervisor by loading the hypercall identifier and other parameters into the specified [CPU](#) registers and then executes the `syscall` instruction [4, 5].

The hypercall identifier consists of the [hypercall number](#) and hypercall-specific flags, as illustrated in Figure 8.1.

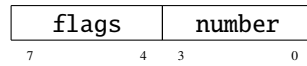


Figure 8.1: Hypercall Identifier

The [status code](#) returned from a hypercall has the format shown in Figure 8.2.

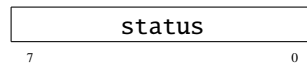
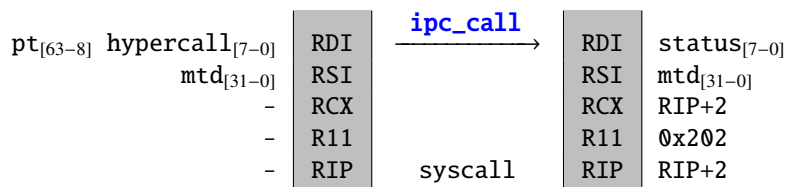


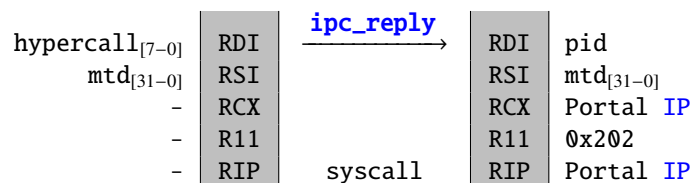
Figure 8.2: Status Code

The assignment of hypercall parameters to [CPU](#) registers is shown on the left side, the contents of the [CPU](#) registers after the hypercall is shown on the right side.

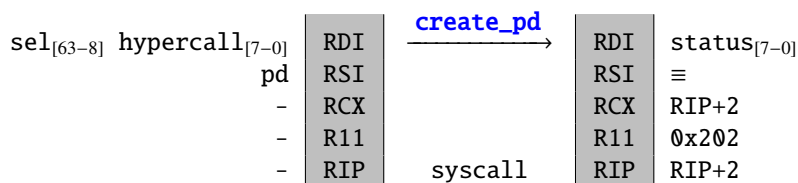
### IPC Call



### IPC Reply



### Create Protection Domain



### Create Execution Context

|                        |                            |     |                           |     |                         |
|------------------------|----------------------------|-----|---------------------------|-----|-------------------------|
| sel <sub>[63-8]</sub>  | hypercall <sub>[7-0]</sub> | RDI | <a href="#">create_ec</a> | RDI | status <sub>[7-0]</sub> |
|                        | pd                         | RSI |                           | RSI | ≡                       |
|                        | hvp <sub>[63-12]</sub>     | RDX |                           | RDX | ≡                       |
| evt <sub>[63-16]</sub> | cpu <sub>[15-0]</sub>      | RAX |                           | RAX | ≡                       |
|                        | sp                         | R8  |                           | R8  | ≡                       |
|                        | -                          | RCX |                           | RCX | RIP+2                   |
|                        | -                          | R11 |                           | R11 | 0x202                   |
|                        | -                          | RIP | syscall                   | RIP | RIP+2                   |

### Create Scheduling Context

|                       |                            |     |                           |     |                         |
|-----------------------|----------------------------|-----|---------------------------|-----|-------------------------|
| sel <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | <a href="#">create_sc</a> | RDI | status <sub>[7-0]</sub> |
|                       | pd                         | RSI |                           | RSI | ≡                       |
|                       | ec                         | RDX |                           | RDX | ≡                       |
|                       | scd                        | RAX |                           | RAX | ≡                       |
|                       | -                          | RCX |                           | RCX | RIP+2                   |
|                       | -                          | R11 |                           | R11 | 0x202                   |
|                       | -                          | RIP | syscall                   | RIP | RIP+2                   |

### Create Portal

|                       |                            |     |                           |     |                         |
|-----------------------|----------------------------|-----|---------------------------|-----|-------------------------|
| sel <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | <a href="#">create_pt</a> | RDI | status <sub>[7-0]</sub> |
|                       | pd                         | RSI |                           | RSI | ≡                       |
|                       | ec                         | RDX |                           | RDX | ≡                       |
|                       | ip                         | RAX |                           | RAX | ≡                       |
|                       | -                          | RCX |                           | RCX | RIP+2                   |
|                       | -                          | R11 |                           | R11 | 0x202                   |
|                       | -                          | RIP | syscall                   | RIP | RIP+2                   |

### Create Semaphore

|                       |                            |     |                           |     |                         |
|-----------------------|----------------------------|-----|---------------------------|-----|-------------------------|
| sel <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | <a href="#">create_sm</a> | RDI | status <sub>[7-0]</sub> |
|                       | pd                         | RSI |                           | RSI | ≡                       |
|                       | val                        | RDX |                           | RDX | ≡                       |
|                       | -                          | RCX |                           | RCX | RIP+2                   |
|                       | -                          | R11 |                           | R11 | 0x202                   |
|                       | -                          | RIP | syscall                   | RIP | RIP+2                   |

### Create Device Context

|                       |                            |     |                           |     |                         |
|-----------------------|----------------------------|-----|---------------------------|-----|-------------------------|
| sel <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | <a href="#">create_dc</a> | RDI | status <sub>[7-0]</sub> |
|                       | pd                         | RSI |                           | RSI | ≡                       |
|                       | dtd#0                      | RDX |                           | RDX | ≡                       |
|                       | dtd#1                      | RAX |                           | RAX | ≡                       |
|                       | dtd#2                      | R8  |                           | R8  | ≡                       |
|                       | -                          | RCX |                           | RCX | RIP+2                   |
|                       | -                          | R11 |                           | R11 | 0x202                   |
|                       | -                          | RIP | syscall                   | RIP | RIP+2                   |

## Control Protection Domain

|                        |                            |     |                  |     |                         |
|------------------------|----------------------------|-----|------------------|-----|-------------------------|
| src <sub>[63-8]</sub>  | hypercall <sub>[7-0]</sub> | RDI | → <b>ctrl_pd</b> | RDI | status <sub>[7-0]</sub> |
|                        | dst                        | RSI |                  | RSI | ≡                       |
| ssb <sub>[63-12]</sub> | ord <sub>[4-0]</sub>       | RDX |                  | RDX | ≡                       |
| dsb <sub>[63-12]</sub> | pmu <sub>[5-0]</sub>       | RAX |                  | RAX | ≡                       |
|                        | mad <sub>[31-0]</sub>      | R8  |                  | R8  | ≡                       |
|                        | -                          | RCX |                  | RCX | RIP+2                   |
|                        | -                          | R11 |                  | R11 | 0x202                   |
|                        | -                          | RIP | syscall          | RIP | RIP+2                   |

## Control Execution Context

|                      |                            |     |                  |     |                         |
|----------------------|----------------------------|-----|------------------|-----|-------------------------|
| ec <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | → <b>ctrl_ec</b> | RDI | status <sub>[7-0]</sub> |
|                      | -                          | RCX |                  | RCX | RIP+2                   |
|                      | -                          | R11 |                  | R11 | 0x202                   |
|                      | -                          | RIP | syscall          | RIP | RIP+2                   |

## Control Scheduling Context

|                      |                            |     |                  |     |                         |
|----------------------|----------------------------|-----|------------------|-----|-------------------------|
| sc <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | → <b>ctrl_sc</b> | RDI | status <sub>[7-0]</sub> |
|                      | -                          | RSI |                  | RSI | stc                     |
|                      | -                          | RCX |                  | RCX | RIP+2                   |
|                      | -                          | R11 |                  | R11 | 0x202                   |
|                      | -                          | RIP | syscall          | RIP | RIP+2                   |

## Control Portal

|                      |                            |     |                  |     |                         |
|----------------------|----------------------------|-----|------------------|-----|-------------------------|
| pt <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | → <b>ctrl_pt</b> | RDI | status <sub>[7-0]</sub> |
|                      | pid                        | RSI |                  | RSI | ≡                       |
|                      | mtd <sub>[31-0]</sub>      | RDX |                  | RDX | ≡                       |
|                      | -                          | RCX |                  | RCX | RIP+2                   |
|                      | -                          | R11 |                  | R11 | 0x202                   |
|                      | -                          | RIP | syscall          | RIP | RIP+2                   |

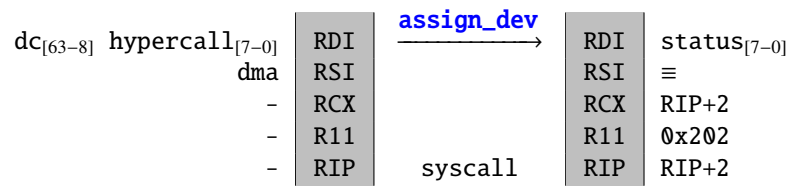
## Control Semaphore

|                      |                            |     |                  |     |                         |
|----------------------|----------------------------|-----|------------------|-----|-------------------------|
| sm <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | → <b>ctrl_sm</b> | RDI | status <sub>[7-0]</sub> |
|                      | stc                        | RSI |                  | RSI | ≡                       |
|                      | -                          | RCX |                  | RCX | RIP+2                   |
|                      | -                          | R11 |                  | R11 | 0x202                   |
|                      | -                          | RIP | syscall          | RIP | RIP+2                   |

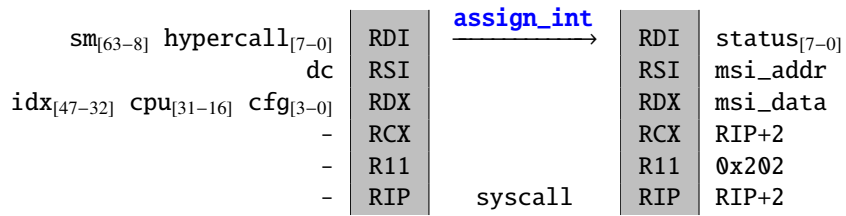
## Control Hardware

|                        |                            |     |                  |     |                         |
|------------------------|----------------------------|-----|------------------|-----|-------------------------|
| desc <sub>[63-8]</sub> | hypercall <sub>[7-0]</sub> | RDI | → <b>ctrl_hw</b> | RDI | status <sub>[7-0]</sub> |
|                        | -                          | RCX |                  | RCX | RIP+2                   |
|                        | -                          | R11 |                  | R11 | 0x202                   |
|                        | -                          | RIP | syscall          | RIP | RIP+2                   |

## Assign Device



## Assign Interrupt



# **Part V**

## **Appendix**

# A Acronyms

|                             |                                                |
|-----------------------------|------------------------------------------------|
| <b>ACPI</b>                 | Advanced Configuration and Power Interface [7] |
| <b>BSP</b>                  | Bootstrap Processor                            |
| <b>CAP</b>                  | Capability                                     |
| <b>CAP<sub>0</sub></b>      | Null Capability                                |
| <b>CAP<sub>OBJ</sub></b>    | Object Capability                              |
| <b>CAP<sub>OBJOBJ</sub></b> | Object Space Capability                        |
| <b>CAP<sub>OBJHST</sub></b> | Host Space Capability                          |
| <b>CAP<sub>OBJGST</sub></b> | Guest Space Capability                         |
| <b>CAP<sub>OBJDMA</sub></b> | DMA Space Capability                           |
| <b>CAP<sub>OBJPIO</sub></b> | PIO Space Capability                           |
| <b>CAP<sub>OBJMSR</sub></b> | MSR Space Capability                           |
| <b>CAP<sub>OBJPD</sub></b>  | PD Capability                                  |
| <b>CAP<sub>OBJEC</sub></b>  | EC Capability                                  |
| <b>CAP<sub>OBJSC</sub></b>  | SC Capability                                  |
| <b>CAP<sub>OBJPT</sub></b>  | PT Capability                                  |
| <b>CAP<sub>OBJSM</sub></b>  | SM Capability                                  |
| <b>CAP<sub>OBJDC</sub></b>  | DC Capability                                  |
| <b>CAP<sub>MEM</sub></b>    | Memory Capability                              |
| <b>CAP<sub>PIO</sub></b>    | PIO Capability                                 |
| <b>CAP<sub>MSR</sub></b>    | MSR Capability                                 |
| <b>CAT</b>                  | Cache Allocation Technology [4]                |
| <b>CDP</b>                  | Code and Data Prioritization [4]               |
| <b>COS</b>                  | Class Of Service [Arm, x86] [4]                |
| <b>CPU</b>                  | Central Processing Unit [3, 4, 5]              |
| <b>CRTM</b>                 | Core Root of Trust for Measurement [21, 22]    |
| <b>CTX</b>                  | Translation Context [12, 13]                   |
| <b>DID</b>                  | Device Identifier [11]                         |
| <b>DMA</b>                  | Direct Memory Access                           |
| <b>DPR</b>                  | DMA Protected Range                            |
| <b>DRTM</b>                 | Dynamic Root of Trust for Measurement [21, 22] |
| <b>DTD</b>                  | Device Topology Descriptor [Arm, x86]          |
| <b>DC</b>                   | Device Context                                 |
| <b>DVA</b>                  | DMA-Virtual Address                            |
| <b>EC</b>                   | Execution Context                              |
| <b>EC<sub>HST</sub></b>     | Host Execution Context                         |
| <b>EC<sub>GST</sub></b>     | Guest Execution Context                        |
| <b>EC<sub>CURRENT</sub></b> | Current Execution Context                      |

|                          |                                                        |
|--------------------------|--------------------------------------------------------|
| <b>EC<sub>ROOT</sub></b> | <a href="#">Root Execution Context</a>                 |
| <b>ELF</b>               | Executable and Linkable Format [23]                    |
| <b>ESPI</b>              | Extended Shared Peripheral Interrupt [10, 11]          |
| <b>FDT</b>               | Flattened Device Tree [24]                             |
| <b>FPU</b>               | Floating Point Unit [3, 4, 5]                          |
| <b>GIC</b>               | Generic Interrupt Controller [10, 11]                  |
| <b>GICC</b>              | <a href="#">GIC CPU Interface</a>                      |
| <b>GICD</b>              | <a href="#">GIC Distributor</a>                        |
| <b>GICH</b>              | <a href="#">GIC Hypervisor Interface</a>               |
| <b>GICR</b>              | <a href="#">GIC Redistributor</a>                      |
| <b>GITS</b>              | <a href="#">GIC Interrupt Translation Service</a>      |
| <b>GPA</b>               | <a href="#">Guest-Physical Address</a>                 |
| <b>GSI</b>               | Global System Interrupt [7]                            |
| <b>HIP</b>               | <a href="#">Hypervisor Information Page</a> [Arm, x86] |
| <b>HPA</b>               | Host-Physical Address                                  |
| <b>HVA</b>               | <a href="#">Host-Virtual Address</a>                   |
| <b>IOAPIC</b>            | I/O Advanced Programmable Interrupt Controller         |
| <b>IOMMU</b>             | I/O Memory Management Unit [17, 18]                    |
| <b>IP</b>                | Instruction Pointer                                    |
| <b>IPC</b>               | <a href="#">Inter-Process Communication</a>            |
| <b>ISD</b>               | Interrupt Semaphore Descriptor [Arm, x86]              |
| <b>KID</b>               | Key Identifier                                         |
| <b>LAPIC</b>             | Local Advanced Programmable Interrupt Controller       |
| <b>LIM</b>               | Launch Integrity Measurement                           |
| <b>LPI</b>               | Locality-Specific Peripheral Interrupt [10, 11]        |
| <b>MAD</b>               | Memory Attribute Descriptor [Arm, x86]                 |
| <b>MBA</b>               | Memory Bandwidth Allocation [4]                        |
| <b>MMU</b>               | Memory Management Unit [3, 4, 5]                       |
| <b>MSI</b>               | Message-Signaled Interrupt [25, 26]                    |
| <b>MSR</b>               | Model-Specific Register [4, 5]                         |
| <b>MTD</b>               | <a href="#">Message Transfer Descriptor</a> [Arm, x86] |
| <b>NOVA</b>              | NOVA OS Virtualization Architecture [2]                |
| <b>PCI</b>               | Peripheral Component Interconnect [25, 26]             |
| <b>PCR</b>               | Platform Configuration Register [21, 22]               |
| <b>PIO</b>               | Port-Based I/O                                         |
| <b>PD</b>                | <a href="#">Protection Domain</a>                      |
| <b>PD<sub>ROOT</sub></b> | <a href="#">Root Protection Domain</a>                 |
| <b>PID</b>               | Portal Identifier                                      |
| <b>PT</b>                | <a href="#">Portal</a>                                 |
| <b>QOS</b>               | Quality Of Service                                     |
| <b>RIM</b>               | Reference Integrity Measurement                        |
| <b>SBDF</b>              | <a href="#">PCI Segment/Bus/Device/Function</a>        |



|                                 |                                                                                         |
|---------------------------------|-----------------------------------------------------------------------------------------|
| <b>SC</b>                       | <a href="#">Scheduling Context</a>                                                      |
| <b>SC<sub>CURRENT</sub></b>     | Current Scheduling Context                                                              |
| <b>SC<sub>ROOT</sub></b>        | <a href="#">Root Scheduling Context</a>                                                 |
| <b>SCD</b>                      | <a href="#">Scheduling Context Descriptor</a>                                           |
| <b>SEL</b>                      | <a href="#">Capability Selector</a>                                                     |
| <b>SEL<sub>EVT</sub></b>        | Event Selector Base [ <a href="#">Arm</a> , <a href="#">x86</a> ]                       |
| <b>SEL<sub>OBJ</sub></b>        | <a href="#">Object Capability Selector</a>                                              |
| <b>SEL<sub>HST</sub></b>        | <a href="#">Host Capability Selector</a>                                                |
| <b>SEL<sub>GST</sub></b>        | <a href="#">Guest Capability Selector</a>                                               |
| <b>SEL<sub>DMA</sub></b>        | <a href="#">DMA Capability Selector</a>                                                 |
| <b>SEL<sub>PIO</sub></b>        | <a href="#">PIO Capability Selector</a>                                                 |
| <b>SEL<sub>MSR</sub></b>        | <a href="#">MSR Capability Selector</a>                                                 |
| <b>SID</b>                      | Stream Identifier [ <a href="#">12</a> , <a href="#">13</a> ]                           |
| <b>SM</b>                       | <a href="#">Semaphore</a>                                                               |
| <b>SMG</b>                      | Stream Mapping Group [ <a href="#">12</a> , <a href="#">13</a> ]                        |
| <b>SMMU</b>                     | System Memory Management Unit [ <a href="#">12</a> , <a href="#">13</a> ]               |
| <b>SP</b>                       | Stack Pointer                                                                           |
| <b>SPC<sub>OBJ</sub></b>        | <a href="#">Object Space</a>                                                            |
| <b>SPC<sub>HST</sub></b>        | <a href="#">Host Space</a>                                                              |
| <b>SPC<sub>GST</sub></b>        | <a href="#">Guest Space</a>                                                             |
| <b>SPC<sub>DMA</sub></b>        | <a href="#">DMA Space</a>                                                               |
| <b>SPC<sub>PIO</sub></b>        | <a href="#">PIO Space</a>                                                               |
| <b>SPC<sub>MSR</sub></b>        | <a href="#">MSR Space</a>                                                               |
| <b>SPC<sub>OBJCURRENT</sub></b> | Current <a href="#">Object Space</a>                                                    |
| <b>SPC<sub>OBJNOVA</sub></b>    | <a href="#">NOVA Object Space</a>                                                       |
| <b>SPC<sub>HSTNOVA</sub></b>    | <a href="#">NOVA Host Space</a>                                                         |
| <b>SPC<sub>PIONOVA</sub></b>    | <a href="#">NOVA PIO Space</a>                                                          |
| <b>SPC<sub>MSRNOVA</sub></b>    | <a href="#">NOVA MSR Space</a>                                                          |
| <b>SPC<sub>OBJROOT</sub></b>    | <a href="#">Root Object Space</a>                                                       |
| <b>SPC<sub>HSTROOT</sub></b>    | <a href="#">Root Host Space</a>                                                         |
| <b>SPC<sub>PIOROOT</sub></b>    | <a href="#">Root PIO Space</a>                                                          |
| <b>SPI</b>                      | Shared Peripheral Interrupt [ <a href="#">10</a> , <a href="#">11</a> ]                 |
| <b>STC</b>                      | <a href="#">System Time Counter</a>                                                     |
| <b>TPM</b>                      | Trusted Platform Module [ <a href="#">21</a> , <a href="#">22</a> ]                     |
| <b>TXT</b>                      | Trusted Execution Technology [ <a href="#">19</a> ]                                     |
| <b>UART</b>                     | Universal Asynchronous Receiver Transmitter                                             |
| <b>UEFI</b>                     | Unified Extensible Firmware Interface [ <a href="#">8</a> ]                             |
| <b>UTCB</b>                     | <a href="#">User Thread Control Block</a> [ <a href="#">Arm</a> , <a href="#">x86</a> ] |
| <b>VMM</b>                      | Virtual-Machine Monitor                                                                 |

|                   |                                                                                    |
|-------------------|------------------------------------------------------------------------------------|
| <b>ipc_call</b>   | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">IPC Call</a>                   |
| <b>ipc_reply</b>  | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">IPC Reply</a>                  |
| <b>create_pd</b>  | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Create Protection Domain</a>   |
| <b>create_ec</b>  | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Create Execution Context</a>   |
| <b>create_sc</b>  | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Create Scheduling Context</a>  |
| <b>create_pt</b>  | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Create Portal</a>              |
| <b>create_sm</b>  | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Create Semaphore</a>           |
| <b>create_dc</b>  | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Create Device Context</a>      |
| <b>ctrl_pd</b>    | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Control Protection Domain</a>  |
| <b>ctrl_ec</b>    | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Control Execution Context</a>  |
| <b>ctrl_sc</b>    | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Control Scheduling Context</a> |
| <b>ctrl_pt</b>    | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Control Portal</a>             |
| <b>ctrl_sm</b>    | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Control Semaphore</a>          |
| <b>ctrl_hw</b>    | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Control Hardware</a>           |
| <b>assign_dev</b> | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Assign Device</a>              |
| <b>assign_int</b> | Hypercall [ <a href="#">Arm, x86</a> ]: <a href="#">Assign Interrupt</a>           |

## B Bibliography

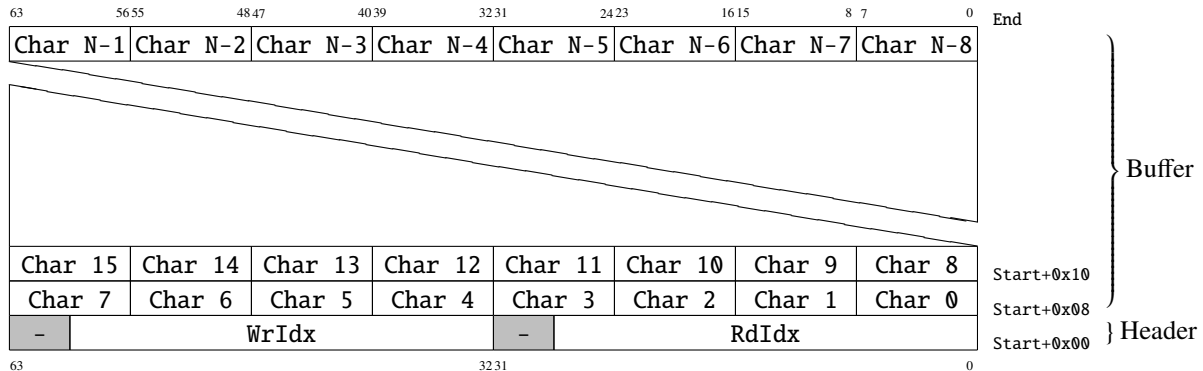
- [1] *RFC 2119*. Internet Engineering Task Force (IETF), 1997. URL <https://tools.ietf.org/html/rfc2119>. v
- [2] Udo Steinberg and Bernhard Kauer. NOVA: A Microhypervisor-Based Secure Virtualization Architecture. In *Proceedings of the 5th ACM SIGOPS/EuroSys European Conference on Computer Systems*, pages 209–222. ACM, 2010. ISBN 978-1-60558-577-2. URL <https://doi.acm.org/10.1145/1755913.1755935>. 2, 80
- [3] *Arm Architecture Reference Manual ARMv8, for ARMv8-A Architecture Profile*. Arm Limited, 2025. URL <https://developer.arm.com/documentation/ddi0487/>. Document Number: DDI0487M.a.a. 8, 52, 57, 59, 79, 80
- [4] *Intel 64 and IA-32 Architectures Software Developer’s Manual, Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*. Intel Corporation, 2026. URL <https://software.intel.com/en-us/articles/intel-sdm>. Document Number: 325462, Revision 91. 8, 67, 73, 74, 79, 80
- [5] *AMD64 Architecture Programmer’s Manual: Volumes 1–5*. Advanced Micro Devices, Inc., 2026. URL <https://developer.amd.com/resources/developer-guides-manuals>. Document Number: 40332, Revision 4.09. 8, 73, 74, 79, 80
- [6] Yoshinori K. Okuji, Bryan Ford, Erich Stefan Boleyn, Kunihiro Ishiguro, Vladimir Serbinenko, and Daniel Kiper. *The Multiboot2 Specification*, 2016. URL <https://www.gnu.org/software/grub/manual/multiboot2/multiboot.pdf>. Version 2.0. 42, 50, 63, 64
- [7] *Advanced Configuration and Power Interface (ACPI) Specification*. UEFI Forum, Inc., 2025. URL <https://uefi.org/specifications>. Version 6.6. 47, 79, 80
- [8] *Unified Extensible Firmware Interface (UEFI) Specification*. UEFI Forum, Inc., 2024. URL <https://uefi.org/specifications>. Version 2.11. 47, 64, 81
- [9] Yoshinori K. Okuji, Bryan Ford, Erich Stefan Boleyn, and Kunihiro Ishiguro. *The Multiboot Specification*, 2010. URL <https://www.gnu.org/software/grub/manual/multiboot/multiboot.pdf>. Version 0.6.96. 50, 63, 64
- [10] *Arm Generic Interrupt Controller Architecture Specification Version 2*. Arm Limited, 2013. URL <https://developer.arm.com/documentation/ihl0048/>. Document Number: IHI0048B.b. 51, 80, 81
- [11] *Arm Generic Interrupt Controller Architecture Specification Version 3 and Version 4*. Arm Limited, 2024. URL <https://developer.arm.com/documentation/ihl0069/>. Document Number: IHI0069H.b. 51, 79, 80, 81
- [12] *Arm System Memory Management Unit Architecture Specification Version 2*. Arm Limited, 2016. URL <https://developer.arm.com/documentation/ihl0062/>. Document Number: IHI0062D.c. 51, 79, 81
- [13] *Arm System Memory Management Unit Architecture Specification Version 3*. Arm Limited, 2026. URL <https://developer.arm.com/documentation/ihl0070/>. Document Number: IHI0070H.a. 51, 79, 81
- [14] *Arm SMC Calling Convention*. Arm Limited, 2026. URL <https://developer.arm.com/documentation/den0028/>. Document Number: DEN0028H. 62
- [15] *Arm True Random Number Generator Firmware Interface*. Arm Limited, 2022. URL <https://developer.arm.com/documentation/den0098/>. Document Number: DEN0098. 62
- [16] *Arm PCI Configuration Space Access Firmware Interface*. Arm Limited, 2022. URL <https://developer.arm.com/documentation/den0115/>. Document Number: DEN0115. 62

- [17] *Intel Virtualization Technology for Directed I/O Architecture Specification*. Intel Corporation, 2025. URL <https://www.intel.com/content/www/us/en/develop/download/intel-virtualization-technology-for-directed-io-architecture-specification.html>. Document Number: D51397-018, Revision 5.10. 64, 80
- [18] *AMD I/O Virtualization Technology (IOMMU) Specification*. Advanced Micro Devices, Inc., 2026. URL <https://www.amd.com/en/support/tech-docs/amd-io-virtualization-technology-iommu-specification>. Document Number: 48882, Revision 3.11. 64, 80
- [19] *Intel Trusted Execution Technology (Intel TXT) Software Development Guide*. Intel Corporation, 2025. Document Number: 315168-017, Revision 17.7. 64, 81
- [20] *TCG PC Client Platform, Firmware Profile Specification*. Trusted Computing Group, 2023. URL <https://trustedcomputinggroup.org/resources>. Version 1.06, Revision 52. 69
- [21] *TCG PC Client Platform, TPM Profile Specification for TPM 2.0*. Trusted Computing Group, 2026. URL <https://trustedcomputinggroup.org/resources>. Version 1.07. 79, 80, 81
- [22] *TCG Trusted Platform Module 2.0 Library Specification*. Trusted Computing Group, 2025. URL <https://trustedcomputinggroup.org/resources>. Revision 1.84. 79, 80, 81
- [23] *ELF Object File Format*. XinuOS, Inc., 2025. URL <https://gabi.xinuOS.com/>. Version 4.3. 80
- [24] *Devicetree Specification*. Linaro Limited, 2023. URL <https://www.devicetree.org/specifications>. Version 0.4. 80
- [25] *PCI Local Bus Specification*. PCI-SIG, 2004. URL <https://pcisig.com/specifications>. Revision 3.0. 80
- [26] *PCI Express Base Specification*. PCI-SIG, 2025. URL <https://pcisig.com/specifications>. Revision 7.0. 80

# C Consoles

## C.1 Memory-Buffer Console

The [NOVA](#) microhypervisor implements a memory-buffer console that provides boot-time and run-time debug output. The memory-buffer console consists of a signaling semaphore (see [6.1.3.1](#)) and an in-memory data structure with a header and a buffer as follows:



The start address and end address of the memory-buffer console are conveyed in the [HIP](#).

The buffer size (N characters) can be computed as:

$$N = \text{MBUF End Address} - \text{MBUF Start Address} - \text{MBUF Header Size}$$

The fields of the header are used as follows:

- **RdIdx** is in range  $[0, N-1]$  and must be accessed with **ACQUIRE/RELEASE** or stronger memory ordering. It points to the next character in the buffer that the console consumer will read and is typically advanced by the console consumer.
- **WrIdx** is in range  $[0, N-1]$  and must be accessed with **ACQUIRE/RELEASE** or stronger memory ordering. It points to the next character in the buffer that the [NOVA](#) microhypervisor will write and is only advanced by the [NOVA](#) microhypervisor.
- The buffer is empty if  $\text{RdIdx} = \text{WrIdx}$ .
- Otherwise **WrIdx** is ahead of **RdIdx**, wrapping around the buffer size N accordingly, i.e. character  $N+x$  will be stored in the same buffer slot as character  $x$ .
- If the buffer becomes full, the [NOVA](#) microhypervisor advances **RdIdx**, forcing the oldest character to be discarded from the buffer.
- Upon completing a line of output, the [NOVA](#) microhypervisor invokes **ctrl\_sm** (Up) on the signaling semaphore. To optimize performance, the console consumer should use **ctrl\_sm** (Down) on the signaling semaphore instead of polling **WrIdx**.

### Note

The memory-buffer console remains always enabled. The [Root Protection Domain](#) can delegate the data structure to any [Protection Domain](#), which can subsequently act as console consumer, read the debug output from the buffer, and write it to a suitable output device.

## C.2 Framebuffer Console

The framebuffer console requires a [UEFI](#) platform with **Graphics Output Protocol** and an EDID-capable display. It provides boot-time-only debug output of the microhypervisor until ownership of platform hardware transitions to the [Root Protection Domain](#).

### Note

The framebuffer console is enabled by default. It can be disabled by specifying the **nofbuf** parameter on the microhypervisor command line. Disabling the framebuffer console can slightly improve the time it takes to boot the system.

## C.3 UART Consoles

The [UART](#) consoles require a platform with an RS232 port/header connected to another system via a serial cable. They provide boot-time-only debug output of the microhypervisor until ownership of platform hardware transitions to the [Root Protection Domain](#). The receiver terminal must be configured for 115200 baud and 8N1 mode.

### Note

All UART consoles are enabled by default. They can be disabled by specifying the **nouart** parameter on the microhypervisor command line. Disabling the UART consoles can significantly improve the time it takes to boot the system.

## D Download

The source code of the [NOVA](#) microhypervisor and the latest version of this document can be downloaded from GitHub: <https://github.com/udosteinberg/NOVA>